

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Department of Computer Science and Engineering
Artificial Intelligence

**GUIDING MOE REASONING VIA ROUTING
INTERVENTION**

Master Thesis in
Big Data Analytics and Text Mining

Supervisor

Prof. Gianluca Moro

Candidate

Leonardo Tassinari

Co-supervisors

Dr. Giacomo Frisoni

Dr. Lorenzo Molfetta

Second Session
Academic Year 2025 – 2026

KEYWORDS

Mixture-of-Experts Language Models

Mathematical Reasoning

Reasoning Episode Annotation

Representation Probing

Expert Routing Intervention

Abstract

Sparse mixture-of-experts language models select a subset of experts for each token, making routing a possible point of intervention during reasoning. This thesis investigates whether observable reasoning activities and internal routing measurements can inform policies that improve mathematical problem solving without updating model weights. The experimental pipeline connects four stages: automatic annotation of seven reasoning episode classes, prospective probing of hidden representations, observational analysis of routing and correctness, and interventions during free generation. Annotations and probes support measurement and target exploration; the evaluated controllers use calibrated expert identities or router margins, without online episode predictions.

The correlation study covers Qwen3.5-35B-A3B, Gemma-4-26B-A4B, and GPT-OSS-20B, with 4,647 attempts from 1,547 source problems per model across six mathematical benchmarks. Probe results indicate that information about upcoming episode labels is accessible on held-out responses within the released SAT corpus: response-grouped macro-AUROC is 0.852 for Qwen, 0.780 for GPT-OSS, and 0.788 for Gemma, exceeding a simple prefix-position baseline. This evaluation does not establish external-domain or online-detector validity. Routing–correctness associations vary with model, benchmark, and reasoning class, motivating controls for difficulty, response length, and termination.

Completed stochastic intervention comparisons each use 1,395 matched attempts from 465 held-out problems. A consistent offline answer-equivalence audit rescues both conditions without new generation. No complete intervention has a positive 95% paired problem-bootstrap interval excluding zero under either attempt or equal-problem weighting. Under the primary attempt-weighted metric, Qwen and GPT-OSS fixed positive strength 2 lose 2.80 and

3.30 percentage points, respectively. GPT-OSS paper-style promotion loses 61.65 points for positive targets and 59.57 for negative targets and substantially increases token-limit hits. All margin intervals contain zero. These exploratory intervals omit multiplicity adjustment. Separately, the interrupted Qwen paper-style intervention retains 0.81% accuracy versus 70.45% on 494 matched IDs, all reaching the guided token limit; its partial AIME cohort does not estimate the full evaluation effect. The thesis provides an auditable framework for connecting reasoning annotations to sparse computation while distinguishing representational evidence, observational associations, and the measured effects of routing interventions.

Introduction

An answer to a mathematical problem provides a clear outcome, but it gives only a partial account of how a language model produced that outcome. A generated solution may alternate between interpreting the problem, planning, calculating, exploring alternatives, and checking conclusions. These activities can accompany both successful and unsuccessful solutions. Understanding their role requires measurements of the generated reasoning process as well as its final correctness.

Sparse mixture-of-experts (MoE) models offer a computational perspective on this question. Their routers select a subset of expert networks to process each token. Expert selection and routing weights can therefore vary as a solution develops. This creates an opportunity to examine whether particular routing patterns accompany particular reasoning activities, and whether changing those patterns affects subsequent generation. It also presents an interpretation problem: an expert frequently used in correct solutions may reflect the problems being solved, the position in the response, or its length, rather than a mechanism whose promotion improves correctness.

The objective of this thesis is to investigate guidance of MoE reasoning through interventions on routing, with the model weights held fixed. The central question is whether signals obtained from reasoning annotations and internal measurements can support useful intervention policies. Addressing this question requires separating three kinds of evidence. A probe can establish that a representation contains information predictive of a label. A correlation can establish that a routing statistic varies with an outcome in a given population. An intervention comparison measures what happens when a specified routing policy is applied during generation. Success at one of these levels does not

establish success at the next.

Research questions. The investigation is organised around three questions:

1. Can functional reasoning episodes be annotated automatically, and is information about an upcoming episode accessible from the hidden representation of the preceding response prefix?
2. How do expert selection, routing confidence, and representation measurements relate to reasoning episodes and answer correctness across models, benchmarks, and positions in a response?
3. Do policies calibrated from these observational measurements improve held-out mathematical accuracy or change token use when applied to expert routing during generation?

Approach and scope. The proposed pipeline connects four stages. First, an LLM-based annotator assigns sentences to seven adapted Schoenfeld episode classes: Read, Analyze, Plan, Implement, Explore, Verify, and Monitor. Prompt optimisation and validation assess the annotator as a measurement instrument; its predictions on generated solutions are not treated as human ground truth. Second, prospective probes test the accessibility of episode information before the next labelled unit and identify locations for internal measurement. These labels describe the function of visible text, without assuming that the text exhaustively represents the model’s internal computation.

Third, saved benchmark solutions are replayed through frozen models to extract routing and representation features. Analyses distinguish the whole continuation, reasoning tokens, labelled episode classes, and positional windows. Repeated attempts are grouped by source problem, and the reported bootstrap intervals preserve that dependence. Generation and replay configurations are recorded separately because corresponding checkpoints under different quantisation settings are not assumed to produce identical internal activations. Fourth, calibrated policies promote or suppress selected expert identities, or modify the selection-boundary margin during free generation. Evaluation

compares each policy with its own matched baseline on problems excluded from calibration.

The observational study covers Qwen3.5-35B-A3B, Gemma-4-26B-A4B, and GPT-OSS-20B on six mathematical benchmarks. Gemma and GPT-OSS share a stratified annotation procedure but retain model-specific selected cohorts; Qwen uses a separate annotation protocol. Completed intervention evidence concerns Qwen and GPT-OSS identity and margin guiding. The interrupted Qwen paper-style run supplies a separate descriptive analysis of completed attempts.

Contributions and findings. The thesis contributes an integrated workflow linking functional descriptions of reasoning to measurements of sparse expert computation and to explicit routing interventions. Saved prompts, trace identities, sentence offsets, model configurations, and artifact fingerprints make the connection between these stages inspectable. The empirical contribution is a comparison of routing–correctness associations across models and reasoning classes, together with matched tests of whether selected associations support effective steering. The tested contribution is the connection between observational routing signals and generation-time intervention outcomes. Episode labels organise the measurements and probes inform measurement locations, but the evaluated controllers do not predict episodes online or condition routing on those predictions. Episode-aware control remains an extension requiring separate detector and policy validation.

The grouped results show prospective episode information on held-out responses within the probe corpus, beyond a simple prefix-position baseline. They do not establish transfer to other domains or the encoders’ own generations. Routing associations depend on the model and benchmark, including changes of sign within a reasoning class. Under the primary attempt-weighted objective, stronger fixed promotion reduces accuracy for Qwen and GPT-OSS, and paper-style promotion severely degrades GPT-OSS accuracy. After consistent offline rescoring, all margin intervals contain zero and no complete intervention has a positive interval excluding zero under either reported weighting. These findings motivate a qualified view of routing-guided reasoning: observational

structure provides testable hypotheses, while a reliable improvement requires additional evidence from controlled interventions. Annotation uncertainty, problem difficulty, response length, termination, and deployment differences remain important limits on interpretation.

Thesis organisation. Chapter 1 introduces the theoretical framework, and Chapter 2 reviews related research. Chapter 3 develops the pipeline, while Chapter 4 specifies the data, model configurations, and evaluation protocols. Chapter 5 presents the findings, and Chapter 6 discusses their implications and limitations.

Contents

1	Theoretical Framework	1
1.1	Reasoning as a measurable process	1
1.2	Sparse mixture-of-experts computation	2
1.2.1	Experts, routers, and conditional computation	2
1.2.2	Interpreting expert identity	4
1.3	A vocabulary for reasoning episodes	4
1.3.1	Adapting episode theory to model-generated text	4
1.3.2	Automatic episode annotation	5
1.4	Prospective representation probes	6
1.5	Routing measurements and statistical analysis	7
1.5.1	Concentration, selection, and temporal dynamics	7
1.5.2	Binary correctness and repeated-attempt accuracy	9
1.5.3	Dependence, uncertainty, and confounding	10
1.6	From observations to routing intervention	11
2	Related Work	13
2.1	Reasoning traces, evaluation, and functional annotation	13
2.2	Prompt optimisation for episode annotation	14
2.3	Probing representations of reasoning	15
2.4	Expert specialisation and direct routing control	17
2.5	Activation steering	19
2.6	Statistical evidence for metric selection	20
2.7	Position of the experimental pipeline	21
3	Method	25
3.1	Optimising the LLM-as-judge prompt	25

3.2	Identifying relevant layers with prospective probes	28
3.3	Relating routing to reasoning and correctness	29
3.3.1	Generation, annotation, and replay	29
3.3.2	Measurements and analysis views	30
3.3.3	Statistical unit and outcome association	31
3.4	Guiding experiments	32
3.4.1	Calibration and held-out evaluation	32
3.4.2	Expert-identity guiding	33
3.4.3	Margin guiding	35
3.4.4	Matched generation and evaluation	36
4	Experimental Setup	39
4.1	Computing infrastructure	39
4.2	Software and model configurations	41
4.2.1	The subject model and the annotation models	41
4.2.2	Quantisation and generation–replay provenance	42
4.3	Datasets and units of observation	42
4.3.1	Annotated reasoning traces	42
4.3.2	Mathematical reasoning benchmarks	43
4.4	Episode-annotation protocol	45
4.5	Prospective hidden-state probes	46
4.5.1	Response-grouped evaluation	47
4.6	Benchmark generation and routing extraction	48
4.6.1	Generation settings	48
4.6.2	Replay measurements and layer selection	48
4.6.3	Sentence annotation	49
4.7	Evaluation and statistical protocol	50
4.8	Artifacts, reproducibility, and experimental scope	51
5	Results	53
5.1	Automatic annotation of reasoning episodes	54
5.1.1	Prompt development and selection	54
5.1.2	Deployment sensitivity and annotation coverage	55
5.2	Prospective episode information in hidden states	56

5.2.1	Within-corpus decodability	56
5.2.2	Response-grouped validation and a position baseline . . .	58
5.2.3	External inspection of the Qwen probes	61
5.3	Routing associations in generated solutions	61
5.3.1	Population, scoring, and termination	61
5.3.2	Confidence and other routing measurements	62
5.3.3	Repeated problems, reasoning classes, and position . . .	66
5.3.4	Sampled class composition and outcome associations . .	71
5.3.5	Directed class transitions by answer correctness	73
5.4	Held-out routing interventions	74
5.4.1	Matched comparisons and uncertainty	74
5.4.2	Expert-selection comparison with SteerMoE	78
5.4.3	Consistent offline answer-equivalence audit	80
5.4.4	Identity selection and intervention strength	82
5.4.5	Interrupted Qwen paper-style intervention: partial results	84
5.4.6	Qualitative inspection of paper-style failures	85
5.4.7	Margin guiding and generation cost	87
5.4.8	Benchmark contributions and weighting sensitivity . . .	89
5.5	What the combined evidence establishes	93
6	Conclusions	95
6.1	Answers to the research questions	95
6.2	Contributions and interpretation	97
6.3	Limitations	98
6.4	Directions for further work	100
A	Additional descriptive routing plots	103
A.1	Class shares and observed outcomes	103
A.2	Benchmark-specific directed transitions	103
A.3	Expert-frequency screens across models	106
	Bibliography	109

List of Figures

3.1	Information flow through the method. Annotation and probing use the reference corpus independently. Their outputs meet in the correlation experiment; guiding policies are fitted using calibration problems only.	26
5.1	Prospective AUROC on response-disjoint outer test folds. Points are means of five fold scores and horizontal bars are 95% conditional response-bootstrap intervals. Both the hidden-state probe and the simple prefix-position baseline are fitted without the test responses.	59
5.2	Whole continuation: saved correctness correlations and pointwise source-problem bootstrap intervals. Overlap or separation of intervals is not a paired test of model differences.	64
5.3	Full reasoning: saved correctness correlations and pointwise source-problem bootstrap intervals. Overlap or separation of intervals is not a paired test of model differences.	65
5.4	Whole-continuation metric-metric Spearman correlations across all six benchmarks. Lower triangles display each association once; diagonals and upper triangles are omitted. Repeated attempts and limit hits remain included. These are descriptive metric relationships, not correctness effects.	65

- 5.5 Accuracy at observed router-metric values. Each benchmark uses up to five quantile bins; horizontal bars show actual value ranges, points show mean values and accuracy, and panel labels give attempt counts per bin (left to right, or a shared count when equal). Vertical bars are pointwise 95% percentile intervals from 500 whole-source-problem bootstrap resamples within each fixed bin (seed 42). Degenerate intervals can occur in all-correct/all-incorrect bins and do not imply certainty. All attempts and limit hits are retained. Bins can share source problems, so their estimates are not independent; neither bin selection nor length and termination confounding is addressed by these intervals. . . . 66
- 5.6 All models: whole-continuation problem means versus avg@32, with saved pointwise problem-bootstrap intervals. All 32 attempts enter each problem mean. 67
- 5.7 Class-conditioned router confidence versus whole-attempt correctness for the stratified Gemma and GPT-OSS cohorts. Cells show point-biserial coefficients and finite-attempt counts. Missing coefficients are undefined, not zero. The models share an annotation procedure but not matched labelled cohorts; this is a descriptive comparison. 68
- 5.8 Class-conditioned confidence and hidden-state step distance versus whole-attempt correctness. Cells show point-biserial r and finite trace count. Classes use sampled Qwen3.8 labels on Qwen3.5 generations, with different token-bearing cohorts. No significance markings are implied. 69
- 5.9 Router confidence versus whole-attempt correctness in absolute reasoning windows, with saved pointwise 95% source-problem bootstrap intervals. Each benchmark has its own panel. Missing estimates remain blank. Later windows use changing token-bearing cohorts; these are not within-trace causal effects. . . . 70

- 5.10 Gemma and GPT-OSS stratified label composition. Denominators are 99,999 and 100,000 valid sampled sentences. Equal sample budgets do not imply equal class or benchmark proportions. These sentence shares are neither complete-corpus token shares nor adjusted model effects. 71
- 5.11 Cumulative counts of sampled sentences by predicted class, pooled across benchmarks. Counts start at zero; a sentence is counted when its last reasoning token is reached. Progress is normalized by each attempt's full reasoning-token length, unlike the absolute windows in Figure 5.9. Legend totals equal the rightmost endpoints. These are raw sampled counts, not cumulative percentages or estimates of full-corpus counts; longer traces and benchmark sampling quotas affect them. 72
- 5.12 Sampled class proportions versus whole-attempt correctness (point-biserial r) and total continuation-token count (Spearman ρ). All labelled attempts enter each class, including observed zero proportions. These differ from correlations of router features within class-selected tokens. No significance claim is made; class shares sum to one and cannot be interpreted as independent effects. 73
- 5.13 Qwen3.5-35B-A3B: directed class transitions for correct and incorrect answers. Cells show mean observed counts per labelled attempt, including self-transitions on the diagonal. Panel headings give denominators and adjacent-pair coverage; zero-pair attempts remain in the mean. The shared colour scale is identical across all three models. Sampling and population differences limit interpretation. 75

5.14 Gemma-4-26B-A4B: directed class transitions for correct and incorrect answers. Cells show mean observed counts per labelled attempt, including self-transitions on the diagonal. Panel headings give denominators and adjacent-pair coverage; zero-pair attempts remain in the mean. The shared colour scale is identical across all three models. Sampling and population differences limit interpretation. 76

5.15 GPT-OSS-20B: directed class transitions for correct and incorrect answers. Cells show mean observed counts per labelled attempt, including self-transitions on the diagonal. Panel headings give denominators and adjacent-pair coverage; zero-pair attempts remain in the mean. The shared colour scale is identical across all three models. Sampling and population differences limit interpretation. 77

A.1 Actual outcome levels versus sampled class proportions. Fixed share bins are [0, 20), [20, 40), [40, 60), [60, 80), and [80, 100] percent. Each point is at the mean observed share and shows accuracy; small labels give attempt counts. Empty bins are omitted. Colours distinguish benchmarks; lines only connect occupied bins. Sparse bins, sentence sampling, length, and difficulty can explain apparent trends; these descriptive plots do not identify optimal class proportions or a validated accuracy-length Pareto frontier. 104

A.2 Mean total continuation tokens versus sampled class share. Bin boundaries, colours, sample-count labels, and sampling limitations are as in Figure A.1. Token count includes the final answer and token-limit completions; lines are descriptive, not intervention effects. 105

- A.3 Observed class-state transitions by benchmark. Rows are the current class, columns the next class; cells are row percentages and row labels give pair counts. Only originally consecutive sentence indices with both labels are counted. Gaps in sampling are never bridged. Self-transitions are retained; empty rows are undefined. This describes the observed adjacent-pair sample, not the complete transition process or an effect of changing state. 106
- A.4 Qwen3.5-35B-A3B: selected-pool expert frequency correlations with correctness and token count. IDs are model-local and pooled across selected layers; they do not identify matching experts across models. Missing coefficients are blank. 107
- A.5 Gemma-4-26B-A4B: selected-pool expert frequency correlations with correctness and token count. IDs are model-local and pooled across selected layers; they do not identify matching experts across models. Missing coefficients are blank. 107
- A.6 GPT-OSS-20B: selected-pool expert frequency correlations with correctness and token count. IDs are model-local and pooled across selected layers; they do not identify matching experts across models. Missing coefficients are blank. 108

List of Tables

1.1	Operational episode categories used by the project, based on the adapted framework of Li et al. [9]. The descriptions concern a unit’s function, rather than its mathematical validity.	5
2.1	Role of each stage in the routing-intervention pipeline. These are research functions, rather than claims of completed causal validation.	22
3.1	Probe-derived router locations used by the method. Indices are zero-based decoder indices and are specific to each model’s probe run.	29
4.1	Resources of the principal experimental node. GPU specifications follow the cluster guide; CPU and host-memory capacities are the scheduler values displayed by the dashboard [29, 30].	40
4.2	Software configuration declared by the Dockerfile, project meta-data, and current launchers. Version intervals are dependency constraints, not exact installed versions for every experiment.	41
4.3	Episode support in the local corpus. Only the synthetic closing units are removed for the probe experiment.	43
4.4	Benchmark coverage of the correlation corpus. Counts refer to source problems and generated attempts, not to the number of independent observations in every statistical analysis.	44
5.1	Context-aware final fits on 407 reused development units. The Qwen3.8 seed is the earlier optimised Qwen3.6 prompt; these are successive development stages, not independent model comparisons.	55

5.2	Frozen Qwen3.8 NVFP4 judge on the same 407 development units. Equal aggregate accuracy does not imply identical labels.	56
5.3	Prospective boundary probes on the same sentence-level split. Each model reports the accuracy-selected hidden-state index, F1, and AUROC. Selection uses the evaluation split; these are within-corpus estimates.	57
5.4	Response-grouped prospective probes on 38 responses and 3,087 units. AUROC is averaged equally across five outer folds and seven targets. Brackets are 95% response-bootstrap intervals (5,000 draws), conditional on saved folds, selected layers, and fits. The paired gain subtracts the prefix-position baseline. Legacy scores select layers by sentence-test accuracy, so the two protocols differ in both splitting and selection.	58
5.5	Response-grouped results by target. AUROC and the paired AUROC gain over the position baseline include 95% conditional response-bootstrap intervals. Balanced accuracy (BA) and binary F1 are mean-fold point estimates at the unchanged decision threshold; their intervals are retained in the result files. Indices list the validation-selected hidden states in outer-fold order, not a globally selected layer.	60
5.6	All-attempt accuracy in the observational generation corpora. Invalid answers with known gold are counted as incorrect, and token-limit completions remain included. These are generation populations, not the held-out guiding baselines.	62
5.7	Whole-continuation router confidence versus correctness: point-biserial coefficients, with all attempts retained. Each model has the same benchmark counts, but these marginal correlations do not estimate controlled model differences.	63
5.8	Reasoning length in tokens, all attempts including limit hits. This replaces the position-coverage count plot; lengths are observed under the generation budget.	70

5.9	Mean sampled sentence share (percent), with equal weight per labelled attempt. Empty-label attempts are excluded; absent classes in an observed sample have zero share. These are not full-corpus class proportions.	72
5.10	Selection sensitivity using saved calibration sufficient statistics. RD here uses the same problem-balanced trace weights, support filter, polarity, layer set, and per-layer budget as lift; it is not a reproduction of the paper's token-pooled contrastive estimator. Overlap counts layer-expert identities.	79
5.11	Selected changes after applying the same revised scoring rule to both conditions. Changes and 95% intervals are percentage points. Attempt weighting is primary; problem weighting is secondary. Historical values are retained here only to document the audit and are not mixed into current result tables.	81
5.12	Complete stochastic identity comparisons. B/G denotes baseline/guided accuracy in percent; changes and intervals are percentage points. Each row uses its own matched baseline and 1,395 attempts on 465 problems. Intervals use 5,000 paired, dataset-stratified problem resamples, seed 42.	83
5.13	Partial Qwen paper-style intervention. Accuracy and token counts compare only the same completed IDs in each condition. B/G denotes baseline/guided. A dash indicates no completed guided attempts, not zero accuracy.	84
5.14	Descriptions of 22 cases sampled as correctness losses under the original scorer. Capped/stopped gives the sampled termination strata. Repetition can coexist with mathematical errors; the arithmetic-error column describes the remaining case without an observed long cycle. Formatting disagreement denotes an equivalent final answer rejected by the saved scorer. Counts are conditional on the sampling design.	86

5.15	Complete stochastic margin-guiding comparisons at strength 1. B/G denotes baseline/guided accuracy in percent; changes and intervals are percentage points. Concurrency is the maximum number of active sequences. Each row has its own matched baseline; differences between rows are not a controlled concurrency experiment.	87
5.16	Generation diagnostics for the stochastic comparisons. Means count all generated tokens, including final answers; limit hits are counts among 1,395 attempts. B/G denotes baseline/guided, and c denotes concurrency. Token differences are descriptive and have no uncertainty interval here.	88
5.17	Benchmark-level accuracy changes in percentage points for the thirteen reported stochastic interventions. Positive/negative denotes expert polarity; c denotes concurrency. Entries are point estimates, not significance decisions. Benchmark counts (problems/attempts) are shown in the header. Full benchmark intervals and correct counts are saved in <code>guiding_sensitivity.json</code>	90
5.18	Sensitivity to the evaluation unit: accuracy changes and 95% intervals in percentage points. Attempt weighting is the primary statistic; equal-problem weighting first averages correctness over each problem's attempts and then averages the 465 problems. Both use the same 5,000 paired, dataset-stratified problem resamples (seed 42). Equal problem weights are not equal benchmark weights. This is a secondary exploratory analysis without multiplicity adjustment.	91
5.19	Benchmark contributions to Qwen margin guiding at concurrency 25. B/G denotes baseline/guided correct counts; net is guided minus baseline correct attempts. Changes and pointwise 95% intervals are percentage points, with 5,000 paired problem resamples (seed 42).	92

Chapter 1

Theoretical Framework

The objective of this thesis is to investigate how the reasoning behaviour of a mixture-of-experts (MoE) language model can be guided through interventions on its routing mechanism. This objective connects a behavioural question, concerning the organisation of a generated solution, with a computational question, concerning which experts process its tokens. Establishing that connection requires a vocabulary for describing reasoning, measurements of internal model activity, and a distinction between observing an association and changing the mechanism that produces it.

The project develops four connected stages toward this objective. The first constructs an automatic annotator of reasoning episodes. The second tests whether episode information is accessible from prefix hidden states and selects measurement locations. The third measures associations between routing, representations, reasoning behaviour, and answer correctness. The fourth tests expert-identity and margin policies during free generation. Chapter 3 specifies their integration. This chapter defines the concepts needed to distinguish measurement, association, and the effect of a routing intervention.

1.1 Reasoning as a measurable process

Let q denote a problem and let $x_{1:T}$ be the model’s generated response, including intermediate reasoning and the final answer. An autoregressive

language model with parameters θ defines

$$P_{\theta}(x_{1:T} \mid q) = \prod_{t=1}^T P_{\theta}(x_t \mid q, x_{<t}). \quad (1.1)$$

Each generated token becomes part of the context for later decisions. Intermediate text can therefore affect subsequent computation, even when evaluation ultimately assigns a single correctness label to the final answer. Chain-of-thought prompting makes such intermediate text explicit and has demonstrated benefits on several reasoning benchmarks [1].

For analysis, the response is segmented into units $s_1, \dots, s_J$, usually sentences or short coherent spans. A unit can restate the problem, introduce a strategy, execute a calculation, or check a previous result. These functions provide a richer description than response length or final accuracy alone. However, a verbal explanation is an observable output of the model, rather than a complete record of its internal computation. Experiments on chain-of-thought faithfulness show that a response can omit factors that influenced its answer [2]. Accordingly, this thesis uses the term *reasoning episode* for a function assigned to a text unit, without assuming that the label identifies a unique internal cognitive state.

Three levels of observation will recur throughout the thesis. At the token level, the model produces hidden representations and expert-routing decisions. At the unit level, a segment receives a functional episode label. At the attempt level, the complete response has a final answer and possibly binary indicators of particular reasoning events. Repeated attempts add a fourth level: the source problem. Keeping these levels explicit is necessary both for temporal alignment and for statistical analysis.

1.2 Sparse mixture-of-experts computation

1.2.1 Experts, routers, and conditional computation

A sparse MoE layer contains several parameterised expert networks and a router that selects a subset for each token. The sparsely gated formulation

of Shazeer et al. made it possible to increase model capacity while activating only part of that capacity for an input [3]. In language-model backbones, an MoE module commonly replaces a dense feed-forward sublayer. The expert is therefore a neural transformation within a larger network; it is not an independent model that solves the whole problem.

Let $u_t^{(\ell)} \in \mathbb{R}^d$ be the input to the router at layer ℓ , and let E_ℓ be the number of routed experts there. For a softmax-based router, scores $z_t^{(\ell)}$ define

$$z_t^{(\ell)} = W_r^{(\ell)} u_t^{(\ell)} + b_r^{(\ell)}, \quad (1.2)$$

$$p_{t,e}^{(\ell)} = \frac{\exp(z_{t,e}^{(\ell)})}{\sum_{j=1}^{E_\ell} \exp(z_{t,j}^{(\ell)})}, \quad (1.3)$$

$$S_t^{(\ell)} = \text{TopK}(p_t^{(\ell)}, k_\ell). \quad (1.4)$$

The bias can be absent, and other routing architectures use different score transformations. These equations specify the softmax case used to define the measurements below, rather than a universal implementation contract.

Writing $f_e^{(\ell)}$ for expert e , a routed output has the form

$$m_t^{(\ell)} = \sum_{e \in S_t^{(\ell)}} g_{t,e}^{(\ell)} f_e^{(\ell)}(u_t^{(\ell)}) + m_{t,\text{shared}}^{(\ell)}. \quad (1.5)$$

Here $g_{t,e}^{(\ell)}$ denotes the effective mixing weight, which may be renormalised over the selected experts. The shared contribution is present only in architectures that include shared experts. This distinction matters: the full router distribution p , the selected set S , and the effective weights g are different objects. An intervention on one need not have the same effect as an intervention on another.

Switch Transformer uses one routed expert per token, whereas Mixtral uses two of eight experts [4, 5]. DeepSeekMoE combines finer expert partitioning with shared experts [6]. These architectural choices affect how routing statistics should be interpreted. They also illustrate why total parameter count and active parameter count must be distinguished: sparse activation reduces per-token expert computation, but does not eliminate the cost of storing expert weights or dispatching tokens.

The project’s Qwen3.5-35B-A3B subject provides a concrete instance. Its official model card describes a hybrid backbone with 40 layers, combining Gated DeltaNet and gated attention, with 256 routed experts, eight selected experts per token, and a shared expert [7]. This makes the subject a hybrid sequence model. The general routing notation above applies independently of the particular sequence-mixing sublayer.

1.2.2 Interpreting expert identity

Expert identities are local to a layer. The correct identifier is the pair (ℓ, e) : expert 7 in one layer has different parameters from expert 7 in another layer. Likewise, selecting an expert frequently during a labelled episode does not establish that the expert implements that episode. Token identity, syntax, position, and problem content may all influence selection. Mixtral’s routing analysis, for example, reports syntactic regularities and locality between consecutive tokens [5].

It is useful to distinguish *statistical specialisation*, meaning that expert use differs across observed contexts, from *functional involvement*, meaning that changing the expert’s contribution changes a behaviour of interest. The first can nominate targets for study. The second requires an intervention and an appropriate comparison. Neither implies a one-to-one mapping between experts and semantic reasoning functions.

1.3 A vocabulary for reasoning episodes

1.3.1 Adapting episode theory to model-generated text

Schoenfeld’s account of mathematical problem solving emphasises the organisation and control of problem-solving activity in addition to the availability of mathematical knowledge [8]. Li et al. adapt this perspective to reasoning-model traces and release a corpus with sentence and paragraph annotations [9]. The present project uses seven sentence-level categories. Their operational meaning in the annotation experiment is summarised in Table 1.1.

Table 1.1: Operational episode categories used by the project, based on the adapted framework of Li et al. [9]. The descriptions concern a unit’s function, rather than its mathematical validity.

Episode	Function of the text unit
Read	Restates information or the goal supplied by the problem.
Analyze	Interprets information, introduces structure, or derives a relationship.
Plan	Commits to a concrete operation or strategy before executing it.
Implement	Executes a procedure, calculation, substitution, or enumeration.
Explore	Tentatively proposes a hypothesis, option, or alternative path.
Verify	Checks an existing candidate, result, or consistency claim.
Monitor	Expresses content-light self-regulation, a transition, or a structural boundary.

These labels are nominal categories. They are not ordered from weak to strong reasoning, and a solution need not pass through them in a fixed sequence. A Verify unit can contain an invalid check; an Implement unit can contain an incorrect calculation. Similarly, repeated monitoring does not necessarily improve a solution. Episode analysis and correctness evaluation therefore measure different aspects of model behaviour.

Segmentation is part of this measurement. One sentence may both announce and execute a plan, while a short structural marker may have no mathematical content. Assigning one label requires a documented precedence rule and access to enough context to distinguish, for example, a given fact from a newly derived relation. These ambiguities limit the meaning of exact agreement with a reference annotation. Per-class errors and inspection of ambiguous boundaries are consequently useful alongside aggregate classification scores.

1.3.2 Automatic episode annotation

Let $a_j \in \mathcal{A}$ denote the reference episode for unit s_j , where $\mathcal{A}$ contains the seven categories. A prompted annotator with instruction ϕ returns

$$\hat{a}_j = A_\phi(q, s_{j-1}, s_j, s_{j+1}). \quad (1.6)$$

This is the information pattern used by **gepaLLMAsJudge**: the problem and neighbouring units help identify the current unit’s function. Because the current and next units are visible, it is an offline annotation tool. It cannot be inserted unchanged into a controller that must act before s_j is generated.

GEPA searches over instructions using feedback and natural-language reflection, while keeping the underlying model weights fixed [10]. In this project, a same-model judge can provide structured optimisation feedback, but final prompt selection is assessed against reference labels. Such feedback is not independent ground truth. Splitting complete responses before their units are assigned to training, validation, and test sets prevents neighbouring passages from crossing evaluation boundaries.

Accuracy measures exact label agreement. Balanced accuracy averages recall across categories, and macro-F1 averages the category-specific F1 scores; both expose failures concealed by a dominant class. Cohen’s κ measures chance-adjusted nominal agreement. Rank-based agreement measures require an independently justified ordering and are not primary measures for these episode labels. The purpose of annotation optimisation is to establish a usable measurement instrument for later experiments, rather than to optimise the subject model’s routing directly.

1.4 Prospective representation probes

For a unit beginning at token index b_j , define a boundary representation $v_j^{(\ell)} = h_{b_j-1}^{(\ell)}$. Under causal processing, it depends on the problem and the preceding response, but not on the target unit. A binary linear probe for episode a has the form

$$\hat{P}(a_j = a \mid v_j^{(\ell)}) = \sigma \left(w_{a,\ell}^\top v_j^{(\ell)} + c_{a,\ell} \right), \quad (1.7)$$

where σ is the logistic function. Above-chance prediction indicates that information useful to this classifier is accessible at that boundary. The interpretation depends on probe capacity, train/test separation, and possible shortcuts [11].

The **probeTest** experiment trains a separate one-versus-rest logistic regression for each label and hidden-state index. It teacher-forces released

DeepSeek-R1 responses through frozen Qwen, GPT-OSS, and Gemma encoders. Consequently, its subject is each encoder’s representation of supplied prefixes, rather than its own generation of the annotated solutions. The original replication uses a sentence-level split: units from a response can appear in both partitions, and test-based layer selection makes the selected maximum optimistic. A separate response-grouped evaluation selects layers on inner validation responses and measures predictions on outer test responses (Section 4.5.1).

Temporal availability and causal influence must also be distinguished. A pre-unit representation is prospective, but predicting a future label does not establish that the probed direction causes that label. A probe can exploit problem type, position, or formatting. Independently trained, class-balanced binary scores also need not form calibrated multiclass probabilities, even when their largest value is used to propose a label for inspection.

Finally, hidden-state indices must be tied to actual locations in the architecture. Let $h_t^{(\ell)}$ denote a block-input residual state and $u_t^{(\ell)}$ the router input from Section 1.2. An intervening sequence-mixing sublayer and normalisation can make these vectors different. The current correlation extraction retains block-input hidden states alongside the router distribution from the corresponding block; it does not directly capture the exact normalised router-input vector. Geometric relationships between the two measurements must be interpreted with this distinction in mind.

1.5 Routing measurements and statistical analysis

1.5.1 Concentration, selection, and temporal dynamics

For a fixed layer and token, suppress the indices on p , and write $p_{(1)} \geq \dots \geq p_{(E)}$ for the sorted router probabilities. For $E > 1$, Shannon entropy and its normalised concentration transform are

$$H(p) = - \sum_{e=1}^E p_e \log p_e, \quad C(p) = 1 - \frac{H(p)}{\log E}. \quad (1.8)$$

$C = 0$ for a uniform distribution and approaches one as mass concentrates on one expert. Although the implementation calls C routing confidence, it is not a calibrated probability that the answer is correct. Routing probabilities describe expert allocation; vocabulary probabilities describe the next token. Their entropies concern different distributions.

For top- k routing with $k < E$, additional measures are

$$M_k(p) = \sum_{i=1}^k p_{(i)}, \quad (1.9)$$

$$B_k(p) = p_{(k)} - p_{(k+1)}, \quad (1.10)$$

$$G_k(p) = \frac{M_k(p)}{k} - \frac{1 - M_k(p)}{E - k}. \quad (1.11)$$

Selected mass M_k measures how much probability is assigned to the active set. The boundary margin B_k describes the separation between the last selected and first unselected expert. G_k compares mean selected and unselected probability. The top-1/top-2 margin alone does not describe the selection boundary when more than one expert is active.

Some of these quantities contain redundant information. For fixed E and k , G_k is an affine function of M_k ; their associations cannot be treated as independent evidence. Similarly, C is an affine sign reversal of H . Presenting a transformed measure can aid interpretation, but does not create an additional empirical finding.

For a trace of $T > 1$ tokens, the current implementation summarises temporal routing with top-1 switching and top- k set overlap:

$$W^{(\ell)} = \frac{1}{T-1} \sum_{t=2}^T \mathbf{1} \left[\arg \max_e p_{t,e}^{(\ell)} \neq \arg \max_e p_{t-1,e}^{(\ell)} \right], \quad (1.12)$$

$$O^{(\ell)} = \frac{1}{T-1} \sum_{t=2}^T \frac{|S_{t-1}^{(\ell)} \cap S_t^{(\ell)}|}{k_\ell}. \quad (1.13)$$

The overlap denominator is the number of selected experts, not the union size used by Jaccard similarity. High overlap indicates persistent selection, but its relation to useful reasoning is an empirical question.

Identity features can instead measure the proportion of tokens selecting a particular (ℓ, e) , or the frequency of a particular expert pair. An ordered

top-1/top-2 pair describes co-selection at one token; it does not describe a transition between tokens or every combination in a top- k set. Layer identity must be preserved before such features can nominate individual experts for intervention.

Hidden-state summaries provide complementary measurements, including vector norm and consecutive-token cosine distance,

$$D^{(\ell)} = \frac{1}{T-1} \sum_{t=2}^T \left(1 - \frac{\langle h_t^{(\ell)}, h_{t-1}^{(\ell)} \rangle}{\|h_t^{(\ell)}\| \|h_{t-1}^{(\ell)}\|} \right). \quad (1.14)$$

The pipeline also constructs a geometry feature by correlating pairwise cosine similarities among sampled hidden states with the corresponding similarities among router distributions. That internal construction uses Pearson correlation. It is a scalar descriptive feature, distinct from the coefficient subsequently used to associate it with correctness; the many token pairs are not treated as independent experimental observations.

1.5.2 Binary correctness and repeated-attempt accuracy

Let $Y_{ia} \in \{0, 1\}$ indicate whether attempt a on problem i produces a correct final answer, and let X_{ia} be a routing or representation feature. Their attempt-level association is measured by point-biserial correlation,

$$r_{\text{pb}} = \text{Corr}(X, Y). \quad (1.15)$$

This is exactly Pearson correlation when correctness is encoded as zero or one. A positive coefficient means that the feature tends to be larger among correct attempts. Its magnitude can depend on class prevalence and feature distributions; it is neither a causal effect nor a measure of probability calibration. The same coefficient applies to a continuous feature paired with a binary reasoning-event indicator. Liu et al. provide direct language model precedent for this continuous-versus-binary setting [12].

With N attempts per problem, define

$$\widehat{A}_i^{(N)} = \frac{1}{N} \sum_{a=1}^N Y_{ia}, \quad \overline{X}_i = \frac{1}{N} \sum_{a=1}^N X_{ia}. \quad (1.16)$$

The first quantity is the empirical per-problem $\text{avg}@N$. For $N = 32$, it takes values in $\{0, 1/32, \dots, 1\}$. It differs from $\text{pass}@N$, which concerns at least one successful attempt among N samples. The primary repeated-attempt analysis uses one pair $(\bar{X}_i, \hat{A}_i^{(N)})$ per problem and computes Spearman correlation,

$$\rho_s = \text{Corr}\left(\text{rank}(\bar{X}_i), \text{rank}(\hat{A}_i^{(N)})\right), \quad (1.17)$$

using average ranks for ties. The scientific question is whether larger feature values tend to accompany larger success fractions, without requiring a linear relation between their numerical scales. Boundedness alone does not make Pearson invalid; Spearman is selected because monotone association is the target. Small problem counts and many tied success fractions still limit precision. Section 2.6 discusses the supporting literature.

Complete groups are necessary to make $\text{avg}@N$ comparable across problems. The analysis checks declared sample identifiers, outcome availability, and feature coverage, and audits exact duplicate generations where their fingerprints are available. A known gold answer with no parseable generated answer is counted as incorrect; a task without a scoring target remains unscored. Truncation and answer-parsing failures must be reported because they can alter both trace features and the measured success fraction.

1.5.3 Dependence, uncertainty, and confounding

Attempts on the same problem share difficulty and content. Counting them as independent observations can make uncertainty appear smaller than it is. For prespecified aggregate features, the pipeline therefore estimates 95% percentile bootstrap intervals by resampling whole problems and retaining their attempts together. For problem-level Spearman coefficients, it resamples the observed problem pairs. This latter procedure quantifies variation across observed problems without separately modelling uncertainty from the finite number of attempts within each problem.

An additional descriptive comparison uses only problems that yield both correct and incorrect attempts:

$$\Delta_i = \bar{X}_{i,Y=1} - \bar{X}_{i,Y=0}. \quad (1.18)$$

Averaging these contrasts removes fixed differences between source problems, but still permits differences in response length, token composition, or generation history within a problem. It also describes the subset of mixed-outcome problems. It is not an estimate of what would happen if routing were changed. The project’s GPQA design varies answer-option positions between attempts, so it is excluded from this particular contrast.

Searching many layers, features, experts, and pairs creates a multiplicity problem. Benjamini–Hochberg adjustments provide a declared way to summarise multiple tests within a specified family [13]. They rely on valid input tests and appropriate dependence assumptions. In particular, applying the adjustment to naive attempt-level p -values does not repair their failure to account for repeated problems. Such values remain exploratory diagnostics. A confirmatory intervention should specify its outcomes, targets, and evaluation partitions independently of these scans.

1.6 From observations to routing intervention

A routing intervention modifies the computation in Equations 1.4–1.5 during inference. One possible family of policies adds a context-dependent bias to the router scores:

$$\tilde{z}_t^{(\ell)} = z_t^{(\ell)} + \alpha v^{(\ell)}(q, x_{<t}), \quad (1.19)$$

after which expert selection and mixing weights are recomputed. This equation describes a design space for the thesis, rather than an already validated implementation. The strength α , affected layers, target experts, and activation times are parts of the policy.

Alternative policies reweight experts already selected, mask particular experts, or alter the selected set. These choices test different hypotheses. For example, rescaling all logits by a positive temperature changes their concentration but preserves their ordering and therefore preserves a deterministic top- k set. Increasing the contribution of an already selected expert also differs from forcing an otherwise unselected expert to participate. In all cases, renormalisation and the contribution of shared experts must be specified to interpret the

intervention.

A controller intended to act before a reasoning unit can use only information available at that time. Offline labels may help construct or evaluate that controller, but the target unit and its future continuation cannot supply its decision at deployment. Prospective probes are relevant because they test whether useful information exists under this restriction. Their predictions still require validation on the generation distribution where the controller will operate.

Teacher-forced replay is useful for measuring activations on a fixed saved response, but an intervention during free generation changes subsequent tokens and therefore subsequent states. A replay-based change in a feature or next-token score is not sufficient evidence of improved problem solving. The intervention objective ultimately concerns a contrast such as

$$\Delta_\pi = \mathbb{E}_{q,\xi} [Y(q, \xi; \pi) - Y(q, \xi; \pi_0)] , \quad (1.20)$$

where π_0 is the unmodified router, π is the intervention, and ξ represents sampling randomness under a specified evaluation protocol. Episode frequencies, answer correctness, and computational cost should be measured together: increasing the amount of checking language is not itself evidence of better reasoning.

This formulation motivates four connected research questions. Can the episode vocabulary be applied reliably to new traces? Can relevant behaviours be detected from information available before they occur? Which routing measurements remain associated with behaviour and correctness after accounting for major confounders? Finally, does acting on selected routing targets improve outcomes on held-out problems under a controlled generation budget? The annotation, probe, and correlation stages address the measurement and observational questions; the guiding experiments specified in Chapter 3 address the effect of intervening during generation.

Chapter 2

Related Work

Research relevant to routing-guided reasoning spans several levels of model behaviour. Some approaches change the prompts or training signals that produce reasoning traces. Others annotate those traces, decode information from internal representations, or directly modify activations and expert routing. This chapter compares those approaches according to the object they measure or manipulate, and positions the four-stage experimental pipeline within that literature.

2.1 Reasoning traces, evaluation, and functional annotation

Chain-of-thought prompting demonstrated that intermediate reasoning demonstrations can improve language-model performance on arithmetic, commonsense, and symbolic tasks [1]. This established generated reasoning text as a practical object of study, but it did not make final accuracy a complete measure of the quality of a reasoning process. Lightman et al. distinguish outcome supervision from feedback on intermediate step validity and release PRM800K for studying process supervision [14]. Their distinction is relevant to this thesis because a correct final answer and a well-formed intermediate process are related but separate evaluation targets.

Functional episode annotation addresses a further question: what a step

is doing within the solution. Building on Schoenfeld’s framework, Li et al. annotate model-generated mathematical solutions at sentence and paragraph levels and analyse transitions between episode categories [9]. Their sentence-level labels supply the project’s annotation vocabulary and released reference data. A semantic episode differs from a process-reward label: Verify describes an attempt to check something, whereas a validity label evaluates whether a step is correct. The same functional category can therefore occur in both successful and failed traces.

The faithfulness experiments of Turpin et al. further limit what can be inferred from generated explanations [2]. They show settings where input cues influence a prediction without being acknowledged in the accompanying rationale. For the present project, the implication is methodological: an episode sequence is an informative description of generated text, but a claim about the mechanism producing it needs independent internal evidence and intervention.

SPIRAL studies reasoning through a different route, using self-play on zero-sum games as a reinforcement-learning training signal [15]. Its evaluation includes mathematical and knowledge-intensive benchmarks also used by the correlation pipeline, and it analyses textual reasoning patterns. The project adopts this benchmark suite with its own benchmark-specific repeated-attempt protocol; it does not claim to reproduce every evaluation setting of the SPIRAL paper. Self-play changes the training process, whereas the thesis’s intended control mechanism acts on routing during inference.

2.2 Prompt optimisation for episode annotation

An automatic episode annotator must handle closely related categories, class imbalance, and contextual ambiguity. GEPA addresses the broader problem of optimising language-model systems through reflective prompt evolution [10]. It uses execution and evaluation traces to generate natural-language feedback, proposes prompt revisions, and maintains candidates with complementary strengths through Pareto-based selection. The optimisation operates with fixed

model weights.

The `gepaLLMAsJudge` experiment applies this approach to the seven-label annotation task. It supplies the problem and neighbouring text units, evaluates predictions against the released annotations, and preserves response boundaries in its data splits. A same-model judge can supply detailed feedback during the search, while agreement with the reference labels and per-class recall constraints govern final selection. These are properties of the project’s experimental design, rather than claims about GEPA’s original evaluation.

The distinction between optimisation feedback and final evaluation is central. A candidate selected after many comparisons on the same validation responses can exploit peculiarities of that partition. Furthermore, a judge that shares the classifier’s model may share its errors. Response-grouped evaluation and an untouched test partition address a different question from whether the search found a better validation prompt. The resulting prompt is intended as a frozen annotation instrument for subsequent experiments; its offline context and reference-label provenance must remain visible when its labels are used to study routing.

2.3 Probing representations of reasoning

Probing asks whether a chosen decoder can extract a property from a model’s representations. Hewitt and Manning demonstrate that a learned transformation of contextual word representations can recover aspects of syntactic tree geometry [16]. Hewitt and Liang subsequently show how control tasks help distinguish information in representations from a probe’s capacity to learn the target task [11]. Together, these works motivate simple, inspectable probes and evaluation designs that make shortcut learning visible. Their linguistic tasks do not establish that reasoning episodes are encoded in the same way; that is an empirical question for this project.

Sun et al. study reasoning as trajectories through representation space [17]. Their analysis examines step-specific geometry and correctness signals in a Llama family, and they investigate adaptive activation changes toward reference trajectories. Their step categories describe positions in a sequence, rather than

Schoenfeld semantic functions. This distinction makes their layerwise probing protocol relevant without making the two annotation tasks equivalent. They also report more limited transfer for correctness prediction than for step-related structure.

The **probeTest** experiment combines a boundary-probe design with the released semantic episode annotations. It extracts representations immediately before the target unit, which creates a prospective prediction task. Its sentence-split replication is useful for comparison with the adopted protocol, but shared response identities and test-based layer selection limit its generalisation claims. The additional grouped study separates responses and layer selection from outer evaluation. Applying the probes to supplied reference solutions tests representation transfer under teacher forcing. Using them to control the model’s own generation requires additional validation.

Chen et al.’s recent preprint, *Beyond the Trace*, connects internal readouts directly to native MoE routing [18]. They construct J64, a 64-axis semantic frame from reasoning states, and learn a routing-based proxy, R64, from expert usage and gate entropy. They evaluate candidate selection, online stopping and resampling, and targeted router edits. This is a close precedent for connecting process interpretation to routing and test-time decisions. The distinction here is the target vocabulary: the present project starts from externally defined Schoenfeld episodes, whereas J64 is derived from model states. Reconstructing a latent readout, predicting an upcoming episode, and improving an answer through intervention remain different evaluation tasks. Their question-grouped evaluation and frozen cross-benchmark transfer also provide useful methodological comparisons.

This line of work supplies possible detection signals and candidate layers. It does not identify a router intervention by itself. A decoder may read information from a representation that is distributed across many experts, or from a component that the model does not use in the way the decoder does. The transition from decodability to controllability therefore remains a separate experimental step.

2.4 Expert specialisation and direct routing control

Early sparse MoE work establishes conditional computation as a way to scale capacity while limiting active computation [3]. Switch Transformer, Mixtral, and DeepSeekMoE explore different selection and expert-sharing designs [4, 5, 6]. These studies support the architectural motivation for treating routing as a control point. They also caution against assuming that its units already correspond to human-readable reasoning functions. In particular, Mixtral’s analysis of syntactic routing and token locality makes lexical controls relevant when comparing experts across reasoning episodes.

Olson et al. provide controlled evidence of semantic routing [19]. They compare expert overlap when a fixed word changes sense across contexts and when a fixed context contains semantically similar or dissimilar replacement words. These controls make their study relevant to separating meaning from lexical identity. However, word-sense sensitivity does not establish that experts specialise in multi-sentence reasoning episodes. For the present analysis, it motivates checking whether episode–routing associations persist beyond differences in the words used to express each episode.

Ye et al.’s preprint shifts the unit of interpretation from individual experts to paths across layers [20]. Their decomposition separates a routing control component from an orthogonal component invisible to the router. Across the architectures studied, they report that individual experts remain polysemantic while expert trajectories group tokens by semantic function across surface forms. This suggests a limitation of per-expert episode correlations: an expert can participate in several functions, with specificity emerging through its combination with other layers. Path-level analysis is consequently a possible extension of the present measurements, rather than a property demonstrated by them.

RICE is a close precedent for reasoning-directed expert intervention [21]. Wang et al. associate experts with reasoning-related marker tokens using normalised pointwise mutual information, then reinforce identified experts during inference. Their reinforcement multiplies an expert’s mixture weight when that

expert is already in the selected set; it does not necessarily make an unselected expert participate. The paper’s description of cognitive experts is a working hypothesis. Reported sensitivity to reinforcement strength and normalisation, together with mixed sampled accuracy changes on AIME, makes clear that observed specialisation does not automatically imply a robust intervention benefit.

SteerMoE investigates behaviour-associated expert activation using contrasting inputs, and promotes or suppresses experts by modifying routing scores before top- k selection [22]. Its experiments concern behaviours such as document faithfulness and safety. This provides direct evidence that expert selection can be used as an inference-time control mechanism for the studied settings. Its behavioural targets differ from mathematical episode structure, so the transfer to reasoning-guided routing remains a question. The difference from RICE is also operational: changing scores before selection can change membership of the active set, whereas reinforcing an already selected expert primarily changes its contribution.

Yoon et al. examine whether the trained router selects favourable expert combinations by comparing standard routes with sampled alternatives at equal expert compute [23]. Their counterfactual analysis uses successful continuations and also studies a final-router update. This is relevant evidence that routing quality can be investigated separately from expert capacity. However, a next-token score on a supplied continuation is a proxy for free-generation success, and layerwise route comparisons do not by themselves identify an episode-level intervention effect.

Su et al. study continuous test-time rerouting under deployment shifts [24]. Their method optimises additive router-logit vectors in selected layers during prefill and at subsequent intervals, using the available context and generated sequence for self-supervision. Generation then continues with the adapted routing until the next update. This offers a comparison with interventions whose expert targets are identified offline: the routing adjustment is optimised online, without external reference data. Its adaptation objective is not an explicit episode vocabulary, so it does not directly test episode-guided control. Comparisons would also need to account for the extra optimisation cost at

inference time.

These studies establish that routing control is an existing research direction. The question developed here is more specific: whether a validated semantic episode vocabulary, prospective internal signals, and outcome-associated routing measurements can jointly support interventions that improve reasoning on held-out problems. Demonstrating that conjunction requires measurements and evaluations beyond finding experts associated with a few textual markers.

2.5 Activation steering

Routing edits should also be compared with methods that act directly on representations. Contrastive Activation Addition constructs steering directions from differences between activations on contrasting examples and adds those directions during generation [25]. This changes the residual representation rather than explicitly specifying which experts are selected. It is consequently a useful conceptual baseline for separating the value of a behavioural direction from the value of acting at the router.

Hong et al. study a residual-stream direction associated with the balance between reasoning and memorisation [26]. Their analysis relates scalar projections to graded reasoning scores and includes interventions on the corresponding features. Sun et al.’s trajectory work similarly moves beyond representation analysis to interventions that use reference reasoning trajectories [17]. These papers show why identifying a direction and testing its manipulation belong to distinct stages of a causal investigation.

For an MoE subject, a residual edit may indirectly alter routing as well as other downstream computations. Conversely, a router edit changes the mixture of expert outputs and therefore the residual trajectory that follows. The two intervention families are connected, but their direct targets differ. A routing-focused thesis must specify that target and evaluate its effects under comparable generation conditions. A change in episode labels alone would leave open whether the method improved problem solving or only changed how the response was expressed.

2.6 Statistical evidence for metric selection

The choice of correlation coefficient follows the outcome scale and the scientific question. Liu et al. use point-biserial correlation between continuous quantities, including context length and answer probability, and binary correctness in figurative-language interpretation [12]. This is direct precedent for the attempt-level continuous-feature/0–1-outcome setting. The additional justification is algebraic: point-biserial is Pearson correlation under binary coding, with an interpretable direction given by the group difference.

For graded targets, Hong et al. report Spearman association between residual-stream projections and model-assigned reasoning scores [26]. Hewitt and Manning use rank correlation in evaluating representation-derived syntactic geometry [16]. Xu et al. relate language-model-derived conceptual representations to human behavioural ratings using Spearman correlation and employ bootstrap confidence intervals [27]. These are precedents for rank-based analysis of internal quantities, rather than direct studies of avg@32 correctness. The project’s choice of Spearman for per-problem mean correctness is therefore justified by its monotone association target, not by claiming that these papers uniquely prescribe that coefficient.

Zhang et al. report both Pearson and Spearman correlations when comparing uncertainty-based hallucination scores with graded human judgements [28]. Their use of entropy-related quantities is relevant to this project’s concentration features, although token uncertainty and expert-routing entropy refer to different distributions. Reporting both coefficients can answer distinct linear and monotone questions; it does not turn two analyses of the same data into independent replications.

For the correlation experiment, the sample hierarchy matters as much as the coefficient. Thirty-two attempts from a single problem provide repeated observations of that problem, rather than 32 independent problems. The problem-level analysis and cluster bootstrap described in Section 1.5 address that hierarchy. Xu et al.’s use of 10,000 bootstrap resamples in one analysis provides a precedent for reporting resampling uncertainty, but does not determine an appropriate resample count or sampling unit for a different experiment

[27].

Benjamini and Hochberg’s multiple-testing framework motivates declaring families of feature, layer, and expert comparisons [13]. Multiplicity adjustment cannot remove confounding, repair non-independent tests, or turn a discovered association into a causal effect. The active analysis therefore serves to prioritise hypotheses whose intervention effects must subsequently be tested on separate data.

2.7 Position of the experimental pipeline

Table 2.1 relates the active experiments to the evidence required by the thesis objective. Their contributions are complementary: a reliable annotation instrument makes behavioural comparisons possible; a prospective probe tests whether suitable signals are available before a decision; a routing analysis narrows the set of mechanisms worth changing; and guiding experiments evaluate those changes during generation.

The completed method connects these stages without requiring an episode-aware controller. The frozen annotator supplies offline behavioural labels, and the probes select measurement locations. Identity and margin policies are then calibrated from routing measurements and correctness, independently of episode labels. This design avoids relying on the transfer accuracy of the probes as online classifiers. Their layer selection remains exploratory under the replicated split and selection protocol.

Generation and measurement provenance impose a further interpretative limit. Quantised generation and replay implementations can differ, so replay measurements need not equal the generator’s internal activations. Guiding policies retain that provenance and are evaluated by changing the actual router used for free generation. Expert-identity calibration also preserves the full (ℓ, e) pair, avoiding the ambiguity of pooled screens that pooled equal numeric expert indices across layers.

The contribution pursued by the thesis is an experimentally grounded connection between semantic reasoning behaviour and controllable sparse computation. Chapter 3 specifies this connection through the four-stage pipeline.

Table 2.1: Role of each stage in the routing-intervention pipeline. These are research functions, rather than claims of completed causal validation.

Experiment	Evidence sought	Role in intervention research
<code>gepaLLMAsJudge</code>	Agreement between automatic episode labels and reference annotations on separated responses.	Defines a reproducible behavioural measurement instrument.
<code>probeTest</code>	Predictability of upcoming episodes from pre-unit hidden states.	Nominates signals and locations for detecting relevant contexts during generation.
<code>correlation_pipeline</code>	Associations of routing and hidden-state features with correctness and reasoning events.	Nominates routing properties and expert-use patterns for controlled tests.
Routing guidance	Paired changes in correctness and generated-token counts on held-out problems.	Tests calibrated expert-identity biases and margin targets during free generation.

Its empirical success is assessed by the effect of frozen routing policies on held-out outcomes and token use.

Chapter 3

Method

The method connects semantic descriptions of reasoning to controlled changes in sparse expert computation. It consists of four stages: optimising an LLM-based episode annotator with GEPA, identifying informative hidden-state locations with probes, measuring associations between routing and reasoning outcomes, and testing routing policies during generation. Model parameters remain frozen throughout. The learned objects are an annotation prompt, linear classifiers, and calibration-derived routing policies.

These stages have distinct roles. The annotator describes the function of text already produced; the probes identify locations where information about an upcoming episode is accessible; the correlation experiment characterises routing on generated solutions; and the guiding experiments test whether changing routing changes performance. Figure 3.1 shows the flow of information. Neither guiding policy requires a predicted episode label at inference time. Episode annotation supplies a behavioural analysis, while the deployed interventions use expert identity or the current router margin.

3.1 Optimising the LLM-as-judge prompt

The first stage operationalises the seven reasoning episodes defined in Section 1.3: Read, Analyze, Plan, Implement, Explore, Verify, and Monitor.

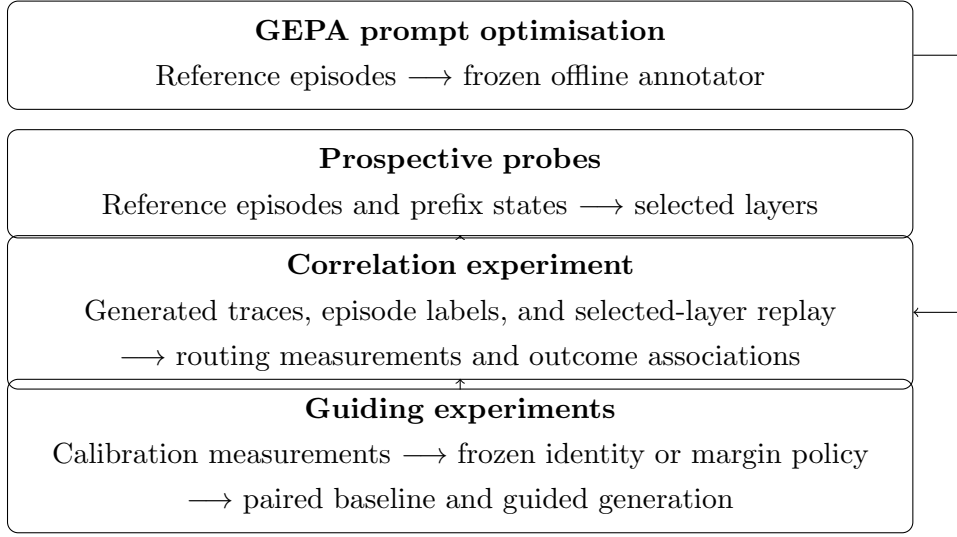


Figure 3.1: Information flow through the method. Annotation and probing use the reference corpus independently. Their outputs meet in the correlation experiment; guiding policies are fitted using calibration problems only.

For problem q and response unit u_j , the annotator receives the context

$$c_j = (q, u_{j-1}, u_j, u_{j+1}), \quad \hat{y}_j = A_\phi(c_j), \quad (3.1)$$

where ϕ is the instruction prompt. The model assigns exactly one label to the current unit. Neighbouring units help distinguish, for example, announcing an operation from performing it, or deriving a result from checking an existing result. The problem statement is visible, but reference answers and rationales are excluded. Since the next unit is available, this is an offline measurement procedure.

GEPA searches over ϕ using performance feedback while leaving the annotator’s weights fixed. The initial context-aware prompt contains the category definitions, boundary rules, and 21 audited synthetic examples, three per category. Complete responses are partitioned before their units are flattened so that neighbouring units cannot cross the annotation training and validation boundaries. Configuration comparison uses nested response-grouped cross-validation; the selected configuration is then fitted to obtain a deployable prompt. The concrete partitions and model settings are specified in Section 4.4.

Two uses of the language model must be distinguished. The episode annotator predicts $\hat{y}_j$ without access to the reference label. During prompt optimisation, a separate call to the same model evaluates that prediction with access to the reference label. This feedback judge supplies an explanation and four scores $s_d \in \{1, \dots, 5\}$: gold agreement, functional fit, contextual coherence, and boundary precision. The reward is

$$R(c_j, \hat{y}_j, y_j) = \sum_{d=1}^4 \omega_d \frac{s_d - 1}{4}, \quad (\omega_1, \omega_2, \omega_3, \omega_4) = (0.40, 0.25, 0.15, 0.20). \quad (3.2)$$

The resulting score lies in $[0, 1]$. The explanations provide information about category confusions for prompt revision. Exact-match and inverse-frequency weighted rewards provide alternative optimisation conditions.

The feedback score is not the final prompt-selection criterion. Selection compares the candidate and seed prompts against reference annotations, using balanced accuracy by default. A candidate is retained only if it improves that measure and no class recall decreases by more than 0.10. Otherwise, the seed remains the selected prompt. This distinction matters because a same-model judge can reinforce its own preferences; agreement with its feedback is not an independent estimate of annotation accuracy. The later Qwen3.8 alignment retained the imported prompt under this selection rule.

The selected instruction is frozen before downstream tagging. Each generated reasoning unit is assigned a label using the same context construction, and its text span is aligned to completion tokens. Unlabelled units remain unlabelled rather than being imputed from probe predictions. Classification quality is assessed through accuracy, balanced accuracy, macro-F1, per-category precision and recall, and valid-output coverage. Changes to the serving check-point or reasoning configuration are recorded because a frozen prompt alone does not imply unchanged annotation behaviour.

3.2 Identifying relevant layers with prospective probes

The second stage identifies hidden-state locations informative about the next reasoning episode. It uses the reference annotations directly, without substituting the automatic judge’s labels. The released problem and response are supplied to each frozen subject model through teacher forcing. For a unit whose first token has position b_j , the feature at hidden-state index ℓ is

$$v_j^{(\ell)} = h_{b_j-1}^{(\ell)}. \quad (3.3)$$

This boundary excludes the target unit and all later tokens. Consequently, the probe measures information available in the prefix before the unit is read, rather than classifying its text retrospectively. Synthetic closing `</think>` units are removed to avoid learning a structural shortcut.

For each episode c and hidden-state index ℓ , a separate logistic regression predicts the binary target $\mathbb{I}[y_j = c]$:

$$P_{c,\ell}(v_j) = \sigma\left(w_{c,\ell}^\top v_j^{(\ell)} + b_{c,\ell}\right). \quad (3.4)$$

The replication uses raw activations, balanced class weights, and L2 regularisation. The sentence-level split and classifier settings are held constant across models. Accuracy, F1, and AUROC are computed at every index. For each category, the saved protocol selects the index with greatest test accuracy, resolving ties toward the earlier index.

The output used by the correlation experiment is the union of those selected indices that correspond to actual router layers. Table 3.1 records the resulting measurement locations. Hidden-state index zero denotes the embedding output; an intermediate index denotes the state entering the decoder block with that index. The final output index has no following router. Indices are therefore filtered for router availability without shifting or renumbering them. In particular, Qwen’s Explore selection at hidden-state index 40 is omitted from routing extraction.

Layer selection narrows the measurement space but does not establish causal importance. Units from the same response occur on both sides of the probe

Table 3.1: Probe-derived router locations used by the method. Indices are zero-based decoder indices and are specific to each model’s probe run.

Subject model	Retained router indices
Qwen3.5-35B-A3B	27, 34, 35, 36, 38, 39
GPT-OSS-20B	15, 19, 21, 23
Gemma-4-26B-A4B	16, 19, 21, 22, 25, 27, 29

split, and the same test partition is used for selection and evaluation. The selected scores are therefore exploratory measures of within-corpus decodability. Moreover, a block-input state is not necessarily the vector presented directly to its router after sequence mixing and normalisation. The subsequent experiments preserve this distinction. They use the selected locations for routing measurements, without treating the probes as validated online episode detectors or optimal intervention selectors.

A separate evaluation mode tests generalisation across responses while preserving those original measurement locations. Five outer folds hold out complete responses. Within each outer development set, a grouped validation partition selects each target’s layer by AUROC; the selected classifier is refitted on the entire development set before outer testing. This produces fold-specific estimators rather than a new globally selected routing layer. A prefix-position baseline and response-level uncertainty intervals help assess how much signal extends beyond a simple progression-through-trace cue.

3.3 Relating routing to reasoning and correctness

3.3.1 Generation, annotation, and replay

The correlation stage begins with freely generated benchmark solutions. Each record identifies the dataset, source problem, attempt, generation settings, response, final-answer score, and termination status. Multiple attempts share

a source-problem identifier. Correctness is defined by the benchmark answer checker; an unparseable answer to a scored problem is incorrect, while a genuinely unscored task has a missing outcome.

Generation and internal measurement are separate operations. The saved continuation is replayed through the corresponding instrumented model to capture hidden states and router behaviour at the retained layers. The current replay path uses the original prompt and completion token IDs, together with tokenizer fingerprints and character offsets, rather than reconstructing a response from rendered text. This preserves native chat and reasoning markers and permits sentence labels to be associated with the same token positions used for feature extraction.

Generation and replay can use different quantised implementations of a checkpoint lineage. Replay measurements therefore describe the replay model on the saved continuation. Numerical equivalence with the generator is not assumed. The same provenance is carried into policy calibration: transferring an expert identity or a numeric margin range between implementations is an explicit experimental assumption.

The frozen annotator supplies class labels for the tagged analysis. When annotation is restricted to a sampled population, the sampling manifest specifies which attempts and sentences are covered. In particular, the standard tagging path selects attempt zero and allocates a sentence budget across benchmarks according to observed error rates. Such class statistics describe that selected population. Aggregate routing analyses and analyses of available tagged units retain their own denominators; missing labels do not imply an absence of the corresponding episode. Untagged runs also support separate lexical indicators of backtracking, contradiction, and self-correction, which are not interchangeable with the seven episode labels.

3.3.2 Measurements and analysis views

Let $p_{i,t,e}^{(\ell)}$ be the full router probability for expert e at token t of attempt i , and let $S_{i,t}^{(\ell)}$ be the selected expert set. The method measures concentration, probability margins, selected mass, selection-boundary gaps, and changes in

expert selection across successive tokens. Hidden-state norms, successive-state distances, and hidden/router geometry provide complementary representation measurements. Their definitions are given in Chapter 1. Expert membership is measured anywhere in the native top- k set, not only at rank one. Every intervention target retains the full pair (ℓ, e) , since equal expert numbers at different layers identify different modules.

Features are summarised over the whole continuation, the reasoning span, each available episode class, and absolute-position windows. The position view uses common token boundaries derived from the corpus mean reasoning length, with an overflow window. A short response contributes only to windows it reaches. Class views similarly require observed labelled tokens of the class. Coverage accompanies each estimate so that changes in the analysed population remain visible. Full distributions need not be retained for all analyses: compact scalar summaries and selected expert tensors preserve the measurements required for correlation and guiding calibration.

3.3.3 Statistical unit and outcome association

For a continuous feature f_i and binary correctness Y_i , the attempt-level association is the point-biserial correlation. Positive values mean that the feature is larger in correct attempts. The same convention applies to binary reasoning-event indicators. For repeated-attempt benchmarks, a separate problem-level analysis relates the mean feature to mean correctness using Spearman correlation. With n attempts, this outcome is $\text{avg}@n$, not the probability that at least one attempt succeeds. The primary problem-level analysis requires the complete declared set of unique attempts and complete binary outcomes. Problems containing both correct and incorrect attempts also permit within-problem feature contrasts.

Repeated attempts are not independent observations. For prespecified aggregate associations, uncertainty is estimated by resampling source problems and retaining their attempts together; the attempt-level bootstrap is stratified by dataset. Problem-level Spearman intervals resample complete problem summaries. Benjamini–Hochberg adjustment is applied within the declared testing

families. Broad layer, expert, and feature-pair screens remain exploratory, and conventional attempt-level significance tests do not replace uncertainty estimates that account for problem clustering.

The current analysis retains token-limit completions and reports their termination status in a generation-budget audit. Missing or nonfinite measurements are omitted for the relevant feature–outcome pair. This keeps the target population distinct from a completed-response-only analysis. Length, benchmark difficulty, and episode composition can affect both routing and correctness, so an observed association is a candidate for intervention, not evidence that changing the feature will improve an answer.

3.4 Guiding experiments

The final stage tests two policies separately: promoting expert identities associated with correctness and moving router margins toward ranges associated with correctness. Both act on native router outputs during free generation, with expert network parameters unchanged. They preserve the model’s expert count, native top- k dispatch, and shared computation. Qwen uses eight of 256 routed experts, GPT-OSS four of 32, and Gemma eight of 128. Policies are fitted separately for each model and cannot exchange expert identities across unrelated checkpoints.

3.4.1 Calibration and held-out evaluation

The global workflow partitions source problems within each benchmark into 70% calibration and 30% evaluation using seed 42. All attempts of a problem remain on the same side. Calibration consumes scored traces with the required routing measurements and records exclusions. Evaluation uses the held-out generation problems irrespective of whether their earlier traces had routing tensors or usable scores. Both correct and incorrect calibration answers are required to estimate an accuracy advantage.

For calibration trace i belonging to problem $q(i)$, let $n_{q(i)}$ be the number of available calibration attempts for that problem and define $w_i = 1/n_{q(i)}$. Each

problem then has total weight one. The common baseline for policy fitting is

$$A_0 = \frac{\sum_i w_i Y_i}{\sum_i w_i}. \quad (3.5)$$

This weighting prevents repeated-attempt benchmarks from dominating solely through their number of attempts; datasets with more problems still have more calibration weight. Policies are frozen before evaluation. Any tuning of intervention strength, support thresholds, or binning must use a separate validation partition rather than held-out outcomes.

3.4.2 Expert-identity guiding

Identity guiding uses the frequency with which an expert belongs to the selected pool. For a trace with T_i saved completion positions, define

$$r_{i,\ell,e} = \frac{1}{T_i} \sum_{t=1}^{T_i} \mathbb{1}[e \in S_{i,t}^{(\ell)}], \quad A_{\ell,e} = \frac{\sum_i w_i r_{i,\ell,e} Y_i}{\sum_i w_i r_{i,\ell,e}}. \quad (3.6)$$

The first normalisation prevents long traces from dominating through their number of tokens. $A_{\ell,e}$ is a frequency-weighted correctness estimate, not the success rate of an expert acting in isolation. Its lift is $L_{\ell,e} = A_{\ell,e} - A_0$.

Expert polarity specifies which association is targeted. Positive polarity retains experts with $L_{\ell,e} > 0$; negative polarity retains experts with $L_{\ell,e} < 0$. In either case, eligibility requires occurrence in at least four distinct calibration problems. The default target budget per layer matches native routing top- k : eight for Qwen and Gemma, four for GPT-OSS. An explicitly smaller budget is supported; larger budgets are rejected. Eligible experts are retained up to this budget, ordered by decreasing lift magnitude, then problem support, then expert identifier. Let E_ℓ denote this retained set. Scores are normalised magnitudes,

$$v_{\ell,e} = \begin{cases} |L_{\ell,e}| / \max_{j \in E_\ell} |L_{\ell,j}|, & e \in E_\ell, \\ 0, & e \notin E_\ell. \end{cases} \quad (3.7)$$

Layers without eligible experts receive zero scores, and calibration fails if no layer has an eligible target. Thus scores are nonnegative for both polarities: the selected set and the direction of intervention are separate choices.

Fixed additive bias. During generation, the fixed method adds a signed bias to each selected layer’s router logits:

$$\tilde{z}_{t,e}^{(\ell)} = z_{t,e}^{(\ell)} + \alpha v_{\ell,e}, \quad \alpha \in \mathbb{R}. \quad (3.8)$$

Positive strength promotes the selected identities; negative strength suppresses the same identities; zero leaves routing unchanged. In particular, negative strength with positive polarity suppresses success-associated experts, whereas positive strength with negative polarity promotes failure-associated experts. The completed Qwen fixed-bias evaluations use strengths $+1, +2, -1$ with positive polarity. Negative polarity is supported by the implementation.

Paper-style steering. The second method implements the extreme-score intervention used by SteerMoE, while retaining the calibration estimator above [22]. For each token and selected layer, define the original log-softmax scores $s_{t,e}^{(\ell)} = \log \text{softmax}(z_t^{(\ell)})_e$. For nonzero strength, replace them simultaneously by

$$\tilde{s}_{t,e}^{(\ell)} = \begin{cases} \max_j s_{t,j}^{(\ell)} + \varepsilon, & e \in E_\ell, \alpha = +1, \\ \min_j s_{t,j}^{(\ell)} - \varepsilon, & e \in E_\ell, \alpha = -1, \\ s_{t,e}^{(\ell)}, & e \notin E_\ell. \end{cases} \quad (3.9)$$

The extrema are computed before any replacement. The implementation uses $\varepsilon = 0.01$ and passes the adjusted scores to native dispatch in the router’s original dtype. If rounding erases the strict separation, the target is moved to the next representable value beyond the original extreme in that dtype. All selected experts receive the same target, independently of lift magnitude. This method supports strengths $+1$ and -1 , with zero as a no-op; strength $+2$ is unsupported. Layers with no selected experts are unchanged. When the target set fills all native top- k slots, promotion fixes the selected identities and gives them equal scores at the hook output. A smaller target budget leaves slots available to untargeted experts. This distinction matters when interpreting the strength of the intervention.

Paper-style steering reproduces the intervention rule, not the paper’s contrastive risk-difference expert-selection estimator. Both methods here use our

problem-balanced, routing-frequency-weighted accuracy lifts. Native dispatch selects and weights experts from the modified router outputs, so either intervention can change both membership and mixture weights. Neither uses episode labels or waits for a particular margin or reasoning event. Their purpose is to test whether acting on a calibrated association changes held-out performance, without assuming that promotion must help or suppression must hurt.

3.4.3 Margin guiding

Margin guiding instead targets a distributional property without assigning preferred expert identities. Let $p_{(r)}$ denote the probability at descending rank r before selected-expert renormalisation. The default metric is the selection-boundary margin

$$m = p_{(k)} - p_{(k+1)}. \quad (3.10)$$

The top-1/top-2 gap is an alternative configuration. For each layer, calibration partitions trace-mean margins into up to five problem-weighted quantile bins, keeping ties together. Among bins supported by at least four distinct problems and with accuracy above A_0 , it selects the bin with highest weighted accuracy, breaking ties by support and then lower margin. The observed minimum and maximum define the frozen interval $[a_\ell, b_\ell]$. Layers without an eligible bin are skipped; a policy with no eligible layer is rejected.

At a live token, the desired margin interpolates toward the nearest point of this interval:

$$d = m + \lambda(\text{clip}(m, a_\ell, b_\ell) - m), \quad 0 \leq \lambda \leq 1. \quad (3.11)$$

Let $r = k$ for the boundary margin, or $r = 1$ for the alternative gap, and let E be the number of routed experts. To decrease the margin, the policy mixes the full probability vector with the uniform distribution u_E :

$$\tilde{p} = (1 - \beta)p + \beta u_E, \quad \beta = \frac{m - d}{m} \quad (d < m). \quad (3.12)$$

To increase it, the policy mixes with a distribution u_r uniform over the current top- r experts and zero elsewhere:

$$\tilde{p} = (1 - \beta)p + \beta u_r, \quad \beta = \frac{d - m}{1/r - m} \quad (d > m). \quad (3.13)$$

If $d = m$, the original logits are returned unchanged. Otherwise the hook returns $\log \tilde{p}$ in the native logit dtype, after which the model performs its usual dispatch and weighting.

In exact arithmetic these mixtures reach the desired gap and preserve expert ordering away from singular endpoints. The implementation caps mixture coefficients below one to avoid fully uniform ties or zero probabilities. Finite precision can still introduce ties and small target errors, so the recorded diagnostics use the achieved margin after casting. Relative expert weights can change even when top- k membership does not. Learning a range from trace averages and applying it to individual token margins is a specific experimental assumption tested by the generation comparison.

3.4.4 Matched generation and evaluation

Each policy is evaluated against a matched native-routing baseline on identical held-out prompts and attempt identifiers. A baseline is generated for the comparison or reused after compatibility checks on the saved configuration and paired inputs, together with verification of the output hash. The Qwen and GPT-OSS margin comparisons at concurrency 25 reuse compatible saved baselines. The two conditions share the checkpoint, tokenised inputs, chat-template options, sampling settings, per-attempt seeds, and completion budget. The default stochastic protocol uses temperature 0.6, top- $p = 0.95$, and 32,768 output tokens; repeated benchmarks retain their original attempt counts. Matching seeds pairs the randomness settings but does not force identical sampled trajectories once the distributions change.

Interventions apply during both prompt prefill and decoding at the policy’s selected layers. The generation engine uses eager execution, disables prefix caching and speculative decoding, and installs router hooks after warmup. Native expert dispatch remains responsible for executing the chosen experts. The run verifies that every intended hook processes tokens. Diagnostics record changed top- k sets and, for margin guiding, achieved margins and distance from the target range. Hook counters can include engine padding or recomputation and are not interpreted as independent generated tokens.

The comparison reports final-answer accuracy, generated-token counts, truncations, and paired wrong-to-right and right-to-wrong transitions. All intervention evaluation outcomes Y_i use the common offline answer-equivalence rule in Section 5.4.3. Frozen policies retain their original calibration labels; no policy is refitted using evaluation outcomes. For N matched attempts, the empirical accuracy change is

$$\hat{\Delta}_\pi = \frac{1}{N} \sum_{i=1}^N (Y_i^\pi - Y_i^0) = \frac{N_{\text{wrong} \rightarrow \text{right}} - N_{\text{right} \rightarrow \text{wrong}}}{N}. \quad (3.14)$$

Token counts include reasoning and final-answer text. Benchmark-specific results accompany pooled comparisons, since pooling attempts gives more weight to repeated-attempt benchmarks. Matching manifests are checked before comparison so that changes in prompts, scoring, software, or sampling are not silently attributed to the routing policy.

Together, these experiments distinguish a calibrated association from the measured effect of acting on it. The method specifies the complete path from reference labels to held-out intervention evaluation; whether either policy improves accuracy or reduces token use is an empirical result, not an assumption of the pipeline.

Chapter 4

Experimental Setup

This chapter describes the computing environment, model implementations, datasets, and evaluation protocols used in the experimental work. The three principal studies address complementary questions: whether an automatic annotator can identify reasoning episodes, whether upcoming episodes can be predicted from hidden representations, and whether routing measurements are associated with answer correctness and reasoning events. Model weights remain fixed throughout these studies. The guiding stage additionally fits routing policies on calibration problems and evaluates them on held-out problems, as specified in Section 3.4.

4.1 Computing infrastructure

Most experiments were executed on the University of Bologna NLP group’s Slurm-managed GPU cluster, principally on `moro43`, the node equipped with one NVIDIA GeForce RTX 5090 with 32 GB of VRAM. Annotation experiments also used one RTX 3090 on `faretra`, the cluster’s master node. The cluster guide lists four RTX 3090 GPUs on that machine, each with 24 GB of VRAM; these are separate devices, rather than a single shared memory pool [29].

Table 4.1 describes node capacity, rather than a resource allocation guaranteed to each job. The GEPA and probe batch launchers request one GPU, 64 GiB of host memory, and a 48-hour time limit; the probe launcher also

Table 4.1: Resources of the principal experimental node. GPU specifications follow the cluster guide; CPU and host-memory capacities are the scheduler values displayed by the dashboard [29, 30].

Resource	Configuration
Compute node	moro43
GPU	One NVIDIA GeForce RTX 5090
GPU memory	32 GB
Scheduler CPU capacity	32 CPU resources
Scheduler host memory	Approximately 121 GB, as displayed
Execution environment	Linux, Slurm, and GPU-enabled Docker containers

explicitly requests four CPUs. Their retained default node is **faretra**, so reproducing those jobs on the main RTX 5090 machine requires overriding the node at submission. The sentence-tagging launcher requests one GPU, eight CPUs, 64 GiB, and four days. These limits are scheduler requests, not measurements of elapsed runtime or peak memory.

The cluster has independent node filesystems. Consequently, the repository, data, and Docker images must be available on the allocated node. The launchers mount the project into the container and use `/llms` as the shared model cache on that node, with `HF_HOME` pointing to the mounted directory [29]. Slurm’s assigned GPU identifiers are passed to Docker. Generation, optional annotation, and activation extraction execute sequentially so that their model weights do not occupy GPU memory together.

The 32 GB device-memory budget influenced the setup directly. Quantised weights made the chosen models practical to load, while long contexts still required space for inference caches and intermediate tensors. The extraction stages therefore retain boundary vectors or compact statistics instead of serialising every full activation tensor.

Table 4.2: Software configuration declared by the Dockerfile, project metadata, and current launchers. Version intervals are dependency constraints, not exact installed versions for every experiment.

Component	Declared configuration
Project container	Ubuntu 22.04, CUDA 12.8.1, Python 3.11
PyTorch	CUDA 12.8 wheels; no explicit version pin for <code>torch</code>
Transformers	$\geq 5.5.0$, < 6
GPTQ loading	GPTQModel 7.3.1
Replay backend	Unsloth 2026.8.18; bitsandbytes $\geq 0.46.1$
Prompt optimisation	DSPy $\geq 3.0.0$, < 4 ; GEPA $\geq 0.0.27$
Probe fitting	scikit-learn $\geq 1.5.0$
Mathematical scoring	<code>math-verify</code> $\geq 0.9.0$, < 1
vLLM server	<code>vllm/vllm-openai:v0.29.0</code>
GGUF annotation server	Locally built <code>llama.cpp:localcuda</code> image

4.2 Software and model configurations

The project runs in Python 3.11 inside a CUDA-enabled Docker image based on Ubuntu 22.04 with CUDA 12.8.1. PyTorch is installed from the CUDA 12.8 wheel index. Hugging Face Transformers provides model loading and forward execution, while NumPy, pandas, SciPy, and scikit-learn support data processing, statistical analysis, and probe training. DSPy and GEPA implement prompt optimisation. Table 4.2 records the relevant declared dependencies and image settings.

4.2.1 The subject model and the annotation models

The representation and routing studies use Qwen3.5-35B-A3B. Its language backbone has approximately 35 billion total parameters and 3 billion active parameters, 40 layers, and a hidden dimension of 2,048. Each MoE block has 256 routed experts, of which eight are selected per token, together with a shared expert. The backbone alternates Gated DeltaNet and gated-attention blocks [7]. The experiments use textual problems and continuations; they do

not evaluate visual inputs.

The annotator is a separate model. The Qwen3.6 GEPA experiments use Qwen3.6-27B in a GGUF Q4_K_XL configuration served through `llama.cpp`. The model-alignment study uses Qwen3.8-27B in UD-Q4_K_XL format. These models supply episode predictions and optimisation feedback; the routing measurements described in this chapter come from Qwen3.5-35B-A3B.

4.2.2 Quantisation and generation–replay provenance

The official Qwen/Qwen3.5-35B-A3B-GPTQ-Int4 checkpoint supplies native GPTQ 4-bit weights for the probe experiment. The correlation generation stage serves this same checkpoint identifier through vLLM. Correlation replay instead loads `unsloth/Qwen3.5-35B-A3B` through Unsloth with runtime 4-bit quantisation. Thus, the generator and replay model have corresponding base model identities, but different quantisation formats and execution engines.

Replay means supplying the saved problem and continuation to a frozen model in a teacher-forced forward pass. It does not generate a replacement answer. The resulting routing and hidden-state features describe how the replay implementation processes those tokens. They are not direct recordings of the generator’s internal activations, and numerical equivalence between the two implementations is not assumed.

4.3 Datasets and units of observation

4.3.1 Annotated reasoning traces

The annotation and probe studies use the released Schoenfeld reasoning corpus of Li et al. [9]. The repository snapshot contains 38 DeepSeek-R1 responses to SAT problems and 3,125 annotated units. Each unit has one of the seven labels introduced in Section 1.3. The SAT problem, answer choices, and available textual descriptions of tables or figures provide context. Reference answers and rationales are excluded from the annotator’s model-visible input.

The annotation study retains all 3,125 units. The probe study removes the

Table 4.3: Episode support in the local corpus. Only the synthetic closing units are removed for the probe experiment.

Episode	Annotation study	Probe study
Read	377	377
Analyze	791	791
Plan	225	225
Implement	675	675
Explore	208	208
Verify	629	629
Monitor	220	182
Total	3,125	3,087

38 synthetic `</think>` closing units, one per response, all labelled Monitor. Its retained corpus therefore contains 3,087 units. This removes a structural marker that could otherwise provide a shortcut for Monitor classification and matches the sentence count in the source study. Table 4.3 shows the resulting class distribution.

The two studies deliberately use different split protocols. GEPA partitions complete responses before flattening their units. With seed 42, the final-fit partition contains 26 training, six validation, and six test responses, corresponding to 2,382, 407, and 336 units. The probe replication instead uses a target-stratified 80/20 sentence split with seed 42: 2,469 training units and 618 test units for each binary target. All 38 response identifiers occur in both probe partitions. Consequently, the probe split measures within-corpus predictability and does not provide a response-disjoint generalisation estimate. The additional grouped probe study reuses these 3,087 units but holds entire responses out, with the same five outer folds for all three encoders and all seven targets (Section 4.5.1).

4.3.2 Mathematical reasoning benchmarks

The completed correlation run covers the six mathematical benchmarks in Table 4.4. The dataset selection and repeated-attempt convention follow

Table 4.4: Benchmark coverage of the correlation corpus. Counts refer to source problems and generated attempts, not to the number of independent observations in every statistical analysis.

Dataset	Problems	Attempts per problem	Total attempts
MATH-500 [31]	500	1	500
AIME 2024 [32]	30	32	960
AIME 2025 [33]	30	32	960
OlympiadBench [34]	675	1	675
AMC 2023 [35]	40	32	1,280
Minerva Math [36]	272	1	272
Total	1,547	—	4,647

the project’s adaptation of the SPIRAL evaluation suite [15]. There are 1,547 source problems and 4,647 generated attempts. These are corpus sizes before any analysis-specific filtering.

MATH-500 is loaded from `HuggingFaceH4/MATH-500`. AIME, AMC, and Minerva use the `math-ai` releases `aime24`, `aime25`, `amc23`, and `minervamath`. These loaders use the published `test` split. OlympiadBench uses `Hothan/OlympiadBench`, configuration `OE_T0_maths_en_COMP`, exposed by the repository loader as `train`. That split name belongs to the dataset release; these problems are used for evaluation without training the subject model. GPQA-Diamond and MMLU-Pro are supported by the pipeline but are absent from this completed six-benchmark corpus.

A separate qualitative probe-transfer stage uses 20 examples each from GSM8K, MATH, PRM800K, and ProcessBench. It teacher-forces their supplied solutions or reconstructed rated paths, rather than generating new ones. These 80 examples serve to inspect predicted episode labels and are not included in the correlation benchmark totals.

4.4 Episode-annotation protocol

Each annotation request contains the problem and the previous, current, and next response units. The model returns one episode label for the current unit. The initial context-aware instruction contains 21 audited synthetic demonstrations, three per class, chosen to illustrate ambiguous category boundaries without copying held-out annotations. Since the next unit is visible, this protocol is an offline annotation procedure.

GEPA searches over instructions while keeping the model fixed [10]. The experiments include exact-match and inverse-frequency weighted rewards, followed by an informative same-model judge reward. In the latter configuration, the judge scores gold agreement, functional fit, contextual coherence, and boundary precision from one to five. Their weights are 0.40, 0.25, 0.15, and 0.20, respectively, and the weighted score is normalised for optimisation. Gold labels are available to this feedback stage, but are not included in the classifier request.

Prompt selection remains based on reference-label performance. The default selection measure is balanced accuracy; accuracy and macro-F1 are also evaluated. A candidate is retained only if it improves the selected measure and does not lower any class recall by more than 0.10 on validation data. This gate prevents a gain in frequent classes from concealing a substantial loss in a less frequent episode.

Configuration comparisons use five-fold nested response-grouped cross-validation over the 32 development responses, with the six test responses excluded. Each outer fold has its own inner training and validation partition. The final-fit studies then use the 26/6/6 partition described above. The Qwen3.8 alignment starts from the 40-example prompt selected by the Qwen3.6 fit, uses the heavy GEPA budget and medium reasoning effort, and retains the imported seed after the validation gate rejects the new candidate. The selected instruction is subsequently frozen for downstream annotation.

Reported classification measures include exact accuracy, balanced accuracy, macro-F1, per-class precision and recall, Cohen’s κ , and valid-output coverage. Kendall’s τ_b is retained in saved artifacts but is not a primary measure for the

nominal episode categories. The final-fit runs do not evaluate the 336-unit test partition. Moreover, those responses had already been inspected in preliminary experiments, so their designation as a locked test does not make them a newly untouched evaluation set.

4.5 Prospective hidden-state probes

Probe extraction supplies each SAT problem as a user message and its released DeepSeek-R1 response as an assistant continuation. Annotated units are matched verbatim and monotonically to the response before tokenisation. For a unit beginning at token b_j , the stored feature is the hidden state at $b_j - 1$. The target unit and later text are therefore unavailable at the measurement boundary.

For Qwen, all 41 hidden-state indices are retained: index 0 is the embedding output and indices 1–40 are successive block outputs. Each feature has 2,048 dimensions. GPT-OSS supplies 25 indices with 2,880 dimensions, and Gemma supplies 31 indices with 2,816 dimensions. Adapting the layerwise binary-probe protocol of Sun et al. [17] from step-position labels to Schoenfeld episodes, a separate one-versus-rest logistic regression is fitted for each label at each index, giving $7 \times 41 = 287$ classifiers for Qwen, 175 for GPT-OSS, and 217 for Gemma in the sentence-split replication. Training uses raw activations, balanced class weights, L2 regularisation with $C = 1$, the `lbfgs` solver, and at most 2,000 iterations. No standardisation or principal-component analysis is applied.

Accuracy, binary F1, and AUROC are computed on the 618-unit test partition. For each target, the selected index maximises accuracy on that same partition. This selection and the shared response identities make the best-index scores exploratory. In the transfer inspection, the seven selected binary classifiers are applied independently. Their scores do not constitute calibrated multiclass probabilities; choosing their maximum produces an inspection label rather than a reference annotation.

Only boundary activations are persisted. At float32 precision, the $3,087 \times 41 \times 2,048$ values require approximately 0.97 GiB, before metadata and classifier files. This storage choice retains the information required for refitting probes

without preserving every token’s full hidden-state trajectory.

4.5.1 Response-grouped evaluation

The additional `probe --protocol grouped` mode operates entirely on saved activations using four CPU workers and one BLAS thread per worker. Five outer folds use `StratifiedGroupKFold`, grouping by response ID and approximately balancing the seven-class labels (seed 42). The shared plan checks exact unit text, labels, and identities across encoders. Outer test folds contain 8, 8, 7, 8, and 7 responses; each response is tested once. A single `GroupShuffleSplit` inside each development fold reserves 20% of its responses for validation, with seed $42 + f + 1$ for zero-based fold f . This leaves 24 fit responses in every fold and six or seven validation responses. Every partition contains both classes for every binary target.

For each target, all hidden-state indices are fitted on the inner fit set. The index with highest validation AUROC is selected, breaking ties toward the lowest index. That classifier is then refitted on all 30 or 31 outer development responses. The untouched test responses determine the final AUROC, balanced accuracy, binary F1, and accuracy. Regularisation and the fixed 2,000-iteration budget match the replication; there is no search over thresholds or hyperparameters. The position baseline uses the same logistic regression with two features: $\log(1 + j)$ for the number j of preceding annotated units, and $\log(1 + t)$ for the number t of preceding response tokens. It excludes total length and future content. Token counts depend on each encoder’s tokenizer, so the three baselines need not be identical.

The primary summary is the unweighted mean of five outer-fold metrics, followed by an equal average across the seven targets. Units have equal weight within each fold metric; response grouping governs separation and uncertainty rather than assigning equal sentence-metric weight to responses. The 95% percentile intervals use 5,000 draws (seed 42), resampling whole responses with replacement independently within each test fold. The same draws pair probe and position predictions and all targets. Draws without both binary classes are excluded for AUROC and balanced accuracy, with valid counts retained.

These intervals condition on the saved folds, selected layers, and fitted models; they omit refitting and split-seed variation and are not adjusted for multiple comparisons. Differences from the legacy scores change both splitting and selection criterion and do not isolate a leakage effect. External-domain transfer and online generation remain separate evaluation questions.

4.6 Benchmark generation and routing extraction

4.6.1 Generation settings

Benchmark solutions are generated zero-shot at temperature 0.6 and top- $p = 0.95$, with a completion budget of 32,768 tokens. The current launcher allocates a 49,152-token context, leaving room for the input prompt in addition to the completion. The default sampling seed is zero, with the per-request seed formed as $s_{i,a} = s_0 + 1000i + a$, where i is the problem index within a dataset and a is the attempt index. The saved trace records the actual seed and sampling settings.

The vLLM configuration permits eight concurrent requests and eight sequences, enables prefix caching, and uses a batch-token budget of 8,192. Generation uses the checkpoint’s MTP head with three speculative tokens. These are execution settings: request concurrency does not alter the declared number of attempts per problem, and long contexts can reduce the number of requests that decode simultaneously. The GPU memory utilisation setting is 0.90 and tensor parallelism is one for the main single-GPU configuration.

4.6.2 Replay measurements and layer selection

After generation, the inference server is stopped and the saved continuations are replayed through the corresponding Hugging Face model. Extraction retains decoder indices 27, 34, 35, 36, 38, and 39. This is the compatible union of indices selected by the separate probe experiment. Probe index 40 is excluded because decoder router indices run from 0 to 39.

The indexing convention is significant. A probe hidden-state index denotes a block output, while replay pairs the block-input hidden state at the retained decoder index with that block’s router distribution. The hidden vector is therefore not the exact normalised router input after the intervening sequence-mixing computation. Probe-based layer selection is a way to restrict measurement locations, not evidence that these locations are optimal intervention targets.

At each retained layer, extraction computes the routing and representation summaries defined in Chapter 1: entropy-derived concentration, top-1/top-2 margin, selected top-eight probability mass, selection-boundary margin, selected-versus-unselected mean probability gap, top-1 switching, top-eight set overlap, hidden-state norm, successive-state distance, and hidden/router geometry. Geometry calculations use at most 128 uniformly sampled tokens per trace by default. Full hidden and router tensors are reduced during extraction; compact scalar features and selected expert IDs are persisted. Original layer indices remain attached to the measurements.

The correlation analysis includes whole-continuation, full-reasoning, class-specific, and absolute-position views. Position analysis uses ten disjoint windows defined relative to the corpus mean reasoning length, followed by an overflow window. These are common absolute token windows, rather than ten equal fractions of each response. A shorter trace has no observation in a window it never reaches. Coverage is consequently reported alongside each view. Class-specific views use available sentence annotations. Full-reasoning and position views use all reasoning tokens independently of annotation coverage.

4.6.3 Sentence annotation

The pipeline applies the frozen GEPA annotator to generated reasoning. Its vLLM judge checkpoint is `unsloth/Qwen3.8-27B-NVFP4`, with temperature zero, low reasoning effort, a 4,096-token completion budget, and a 32,768-token context. The server uses eager execution and an FP8 KV cache; judge MTP is disabled in the default configuration. This deployment differs from the GGUF model and medium-effort setting of the alignment study, so agreement measured there is not automatically an estimate of downstream annotation accuracy.

Before tagging, the Qwen sampling protocol selects attempt zero for repeated problems and at most 100,000 sentences across benchmarks. Benchmark quotas are proportional to their observed error rates; sentences are sampled uniformly within each benchmark with seed 42. The sampling manifest identifies the selected units. This outcome-dependent sample has a different population from the full repeated-attempt corpus. Class-specific analyses produced from it therefore describe the labelled subset rather than the full corpus.

4.7 Evaluation and statistical protocol

Mathematical answers are checked with `math-verify`; the pipeline records when a normalised exact or numeric fallback is used. For an attempt with a known reference answer, an unparseable response is counted as incorrect in the all-attempt evaluation. This denominator is distinct from an accuracy calculated only over successfully parsed answers.

For benchmarks with one attempt per problem, the reported score is empirical pass@1. For the repeated AIME and AMC evaluations, it is avg@32: the mean correctness over 32 attempts for each problem, averaged over problems. It does not measure whether at least one of the 32 attempts succeeds. A pooled attempt-level score gives more weight to benchmarks with more attempts, so benchmark-specific scores remain necessary for interpretation.

Continuous features are related to binary correctness or event indicators using point-biserial correlation. At the problem level, mean features are related to avg@32 using Spearman’s rank correlation. The primary repeated- problem analysis requires the full set of unique attempt identifiers, complete binary outcomes, and no duplicate generations under the recorded fingerprint checks. Problems with both correct and incorrect attempts also support a paired correct-minus-incorrect feature contrast.

The saved analysis uses 500 bootstrap resamples and seed 42 for 95% percentile intervals. Resampling occurs at the source-problem level, keeping repeated attempts together. Benjamini–Hochberg adjustment is applied within the declared testing families [13]; layerwise, feature-pair, and expert-use screens remain exploratory. The initial expert-use screen pools equal numeric expert

indices across retained layers. It therefore cannot identify a particular layer-specific expert module without a subsequent analysis preserving the full (ℓ, e) identity.

Token-budget handling. The analysis retains supplied token-limit completions and records their termination status in a generation-budget audit. Missing or nonfinite features are omitted only for the relevant analysis pair. Problem-level avg@32 requires a complete set of attempts and binary outcomes. Estimates remain conditional on the configured generation budget.

4.8 Artifacts, reproducibility, and experimental scope

The implementation is organised under `src/moe_exp/gepaLLMAsJudge`, `src/moe_exp/probeTest`, and `src/moe_exp/correlation_pipeline`. Their outputs preserve the information needed to connect a numerical result to its inputs: split response identifiers, seed and selected prompts, per-call judge feedback, activation manifests, classifier parameters, generation settings, text fingerprints, retained layer indices, extraction summaries, and analysis-specific inclusion counts. Generation, annotation, and replay are checkpointed independently, allowing completed work to be reused when the recorded configuration matches.

Probe artifacts use the root `results/probeTest`, with model directories `qwen3.5-35b-a3b-gptq-int4`, `gpt-oss-20b`, and `gemma-4-26b-a4b-it-nf4`. Each keeps the original `probes` outputs and a separate `probes_grouped_s42` directory containing fold assignments, candidate validation metrics, fitted classifiers, and one outer-test prediction per unit and target. The shared plan is saved under `results/probeTest/grouped_s42`. Grouped configurations fingerprint activation shards, source code, and library versions. Tables and the figure are regenerated by `report/generate_grouped_probe_results.py`; `report/grouped_probe_sources.json` records their sources and checks. GEPA runs have separate directories under `results/gepaLLMAsJudge`. Correlation outputs

use the root `results/correlation_pipeline`, with the Qwen corpus under `reasoning-vllm-v1`. Saved fingerprints identify the generation, annotation, and replay inputs used by each analysis.

The manifests and launchers provide substantial provenance, but several dependencies still use version intervals, local image tags, or the mutable model revision `main`. These identify a configuration without fixing every underlying binary. Exact reproduction therefore also depends on the resolved model revision, Docker image digest, package versions, and matching code revision. Seeds alone do not establish bitwise equivalence across different quantisation or inference implementations.

The guiding stage is implemented in `src/moe_exp/moe_identity_guiding` and `src/moe_exp/moe_margin_guiding`. These workflows fit separate model-specific policies from the correlation artifacts and compare native and guided generation on matched held-out problems. Their split, calibration, intervention, and evaluation rules are described in Section 3.4. Split manifests, policy files, per-condition generations, and routing diagnostics provide the provenance for a guiding result; implementation checks alone do not establish an improvement in generation accuracy.

Chapter 5

Results

This chapter follows the four stages of the experimental pipeline. The annotation and probing experiments assess the available measurements of reasoning episodes. The observational analysis then examines routing and correctness on generated mathematical solutions. Finally, matched generation experiments test whether acting on selected routing measurements changes performance. These stages provide different kinds of evidence: agreement with an annotation corpus, accessibility of information in representations, association with an outcome, and the effect of an implemented intervention. They are interpreted separately before drawing conclusions about guidance.

The numerical snapshot uses the completed artifacts available on 24 September 2026. Unless stated otherwise, intervention results use stochastic decoding. Complete matched comparisons and the interrupted Qwen paper-style experiment are reported separately; missing guided attempts never enter the partial-result denominator. The report’s source manifests and the thesis companion `results_sources.json` identify the saved inputs and comparison statistics. The latter includes the additional completed margin comparisons found in the results directory. No new model inference was performed. Intervention tables use the consistent offline scoring rule in Section 5.4.3, while calibration and observational analyses retain their original labels. The qualitative inspection keeps its original sampled IDs and historical outcome frame.

5.1 Automatic annotation of reasoning episodes

5.1.1 Prompt development and selection

The judge experiments show that prompt optimisation can improve agreement on a development corpus, but the improvement depends on the starting prompt and selection protocol. The preliminary response-grouped experiments gave 66.07% test accuracy for an optimised base instruction and 69.94% for the matched few-shot condition. These runs preceded the context-aware protocol and used per-unit exact match. They therefore motivate the use of examples, without establishing the performance of the final annotator on generated benchmark traces.

The later nested cross-validation study evaluated 2,789 units from 32 responses. Macro-F1 selection with a light optimisation budget achieved accuracy 0.6734, balanced accuracy 0.6796, and macro-F1 0.6505; the heavy budget achieved 0.6669, 0.6737, and 0.6437. Increasing search effort did not improve this comparison. The class-recall gate retained the seed in two of five light-budget folds and three of five heavy-budget folds. Since the outer folds evaluated the selected prompt rather than both seed and candidate, these results describe the selection procedure and do not directly estimate the paired outer-fold benefit of optimisation.

The completed Qwen3.6 context-aware final fit used 407 development units from six responses. Its same-model feedback combined gold agreement with functional fit, contextual coherence, and boundary precision. As shown in Table 5.1, exact accuracy increased from 0.6683 to 0.7543, with corresponding improvements in balanced accuracy and macro-F1. The candidate corrected 49 seed errors and introduced 14 new errors, a net gain of 35 units. This is a substantial development-set change, but selection repeatedly used the same validation responses; it is not a fresh test estimate.

The final Qwen3.8 alignment started from the previously optimised, 40-example instruction. Its new candidate reduced accuracy from 0.7174 to 0.7125 and balanced accuracy from 0.7400 to 0.7288. Plan recall fell from 0.7500

Table 5.1: Context-aware final fits on 407 reused development units. The Qwen3.8 seed is the earlier optimised Qwen3.6 prompt; these are successive development stages, not independent model comparisons.

Judge	Prompt	Accuracy	Balanced acc.	Macro-F1
Qwen3.6	Seed	0.6683	0.6765	0.6132
Qwen3.6	GEPA candidate	0.7543	0.7769	0.7270
Qwen3.8	Seed	0.7174	0.7400	0.6938
Qwen3.8	GEPA candidate	0.7125	0.7288	0.6743

to 0.5625, exceeding the allowed drop of 0.10. The selection gate therefore retained the imported seed. The internal optimisation reward increased even though exact-label performance declined, illustrating why that reward cannot substitute for independently specified reporting metrics. The stronger Qwen3.6 final-fit numbers and the Qwen3.8 numbers belong to successive deployments and prompt-development stages; they are not a controlled comparison of model capabilities.

The final-fit artifacts explicitly leave the 336-unit locked partition unevaluated. Its responses had nevertheless been inspected during earlier experiments. It cannot serve as a genuinely unseen final assessment merely because later code excludes it. Moreover, success on the released episode corpus does not establish label quality on long solutions sampled from the three benchmark generators.

5.1.2 Deployment sensitivity and annotation coverage

The frozen Qwen3.8 NVFP4 deployment was checked on the same 407 development units under low and medium reasoning effort. Both settings achieved accuracy 0.7101, while balanced accuracy and macro-F1 differed slightly (Table 5.2). Approximately 10.1% of labels changed between settings. Equal accuracy therefore masks disagreement about individual sentences, which matters when those labels define the tokens used in a class-conditioned routing analysis.

Table 5.2: Frozen Qwen3.8 NVFP4 judge on the same 407 development units. Equal aggregate accuracy does not imply identical labels.

Effort	Accuracy	Balanced acc.	Macro-F1
Low	0.7101	0.7415	0.6863
Medium	0.7101	0.7444	0.6832

For the generated benchmark corpus, the refreshed Gemma and GPT-OSS annotations use a common stratified procedure: 100,000 selected sentences per model, with seed 42 and quotas based on dataset, saved correctness, and reasoning-length quartile. One completion per source problem is eligible for annotation. GPT-OSS has 100,000 valid labels on 1,510 problems; Gemma has 99,999 on 1,503 problems, with one missing judge response. The missing sentence is excluded from class features while its trace remains partially labelled. The four exported parts are pieces of one sample, not independent replicates.

Coverage extends to all six benchmarks, but equal sentence budgets do not produce equal cohorts. For example, OlympiadBench contributes 54,556 valid Gemma labels and 70,215 GPT-OSS labels. Qwen retains a separate annotation sampling protocol and deterministic judge deployment. Class proportions and class-conditioned correlations must therefore be read with their actual model-specific denominators. None of these annotations has an independent human accuracy estimate on the generated mathematical solutions. The annotation stage supplies an explicit, inspectable measurement instrument, with unresolved transfer and boundary errors.

5.2 Prospective episode information in hidden states

5.2.1 Within-corpus decodability

All three encoders were evaluated on the same 3,087 labelled units, with 2,469 training and 618 test units. The probes use the hidden state of the prefix

immediately before the labelled unit, rather than a representation computed after reading it. Table 5.3 reports each target’s accuracy-selected hidden-state index and the corresponding F1 and AUROC. The indices denote hidden-state outputs and should not be equated directly with decoder-block indices without the extraction convention described in Chapter 3.

Table 5.3: Prospective boundary probes on the same sentence-level split. Each model reports the accuracy-selected hidden-state index, F1, and AUROC. Selection uses the evaluation split; these are within-corpus estimates.

Target	Index	Qwen		Index	GPT-OSS		Index	Gemma	
		F1	AUROC		F1	AUROC		F1	AUROC
Read	34	0.704	0.934	19	0.686	0.906	21	0.675	0.898
Analyze	36	0.647	0.835	21	0.540	0.767	27	0.611	0.816
Plan	39	0.485	0.855	15	0.434	0.852	25	0.500	0.907
Implement	38	0.771	0.930	21	0.745	0.905	29	0.735	0.922
Explore	40	0.489	0.888	23	0.405	0.842	16	0.447	0.835
Verify	27	0.726	0.912	21	0.667	0.880	19	0.603	0.872
Monitor	35	0.482	0.903	15	0.344	0.774	22	0.382	0.831

There is ranking information for every target in every model: selected-layer AUROC ranges from 0.835 to 0.934 for Qwen, 0.767 to 0.906 for GPT-OSS, and 0.816 to 0.922 for Gemma. Implement has the largest F1 in all three models, at 0.771, 0.745, and 0.735, respectively. Read and Verify are also more accessible at the evaluated decision threshold than several of the rarer classes. Monitor illustrates the difference between ranking and thresholded classification: Qwen reaches AUROC 0.903 but F1 only 0.482, while GPT-OSS has F1 0.344. A high majority-class accuracy is particularly uninformative for these imbalanced one-versus-rest tasks.

The estimates have two sources of optimism. All 38 responses appear in both sentence partitions, so test sentences share response context with training sentences. In addition, the reported index is selected by accuracy on the same test partition used for evaluation. The table consequently supports prospective within-corpus decodability, not response-held-out prediction. It also does not establish that the selected hidden-state indices are the best sites for a causal

Table 5.4: Response-grouped prospective probes on 38 responses and 3,087 units. AUROC is averaged equally across five outer folds and seven targets. Brackets are 95% response-bootstrap intervals (5,000 draws), conditional on saved folds, selected layers, and fits. The paired gain subtracts the prefix-position baseline. Legacy scores select layers by sentence-test accuracy, so the two protocols differ in both splitting and selection.

Model	Legacy AUROC	Grouped AUROC	Position AUROC	Paired gain
Qwen	0.894	0.852 [0.838, 0.868]	0.566 [0.552, 0.594]	+0.286 [0.259, 0.302]
GPT-OSS	0.847	0.780 [0.761, 0.797]	0.571 [0.557, 0.600]	+0.209 [0.179, 0.227]
Gemma	0.869	0.788 [0.772, 0.805]	0.566 [0.553, 0.594]	+0.221 [0.195, 0.237]

intervention. The probe supplies a measurement-location heuristic rather than a validated online controller.

5.2.2 Response-grouped validation and a position baseline

The additional grouped study tests the same representations on complete responses that are absent from both fitting and layer selection. Table 5.4 reports macro-AUROC 0.852 (95% interval [0.838,0.868]) for Qwen, 0.780 ([0.761,0.797]) for GPT-OSS, and 0.788 ([0.772,0.805]) for Gemma. The corresponding legacy means are 0.894, 0.847, and 0.869. These differences change both the split and the layer-selection criterion, so they cannot be attributed solely to response overlap. The grouped results nevertheless establish a stronger claim than the original table: upcoming-label information remains accessible on held-out responses within the released SAT corpus.

The prefix-position baseline reaches macro-AUROC 0.566, 0.571, and 0.566, respectively. Each encoder exceeds its baseline, and all 21 per-target paired AUROC intervals exclude zero; these intervals are not adjusted for multiple comparisons (Table 5.5). Figure 5.1 shows both predictions with response-bootstrap uncertainty. Thus the signal exceeds this particular two-feature positional rule. The control does not exclude more flexible position effects, problem-type or formatting shortcuts, or correlations between episodes and

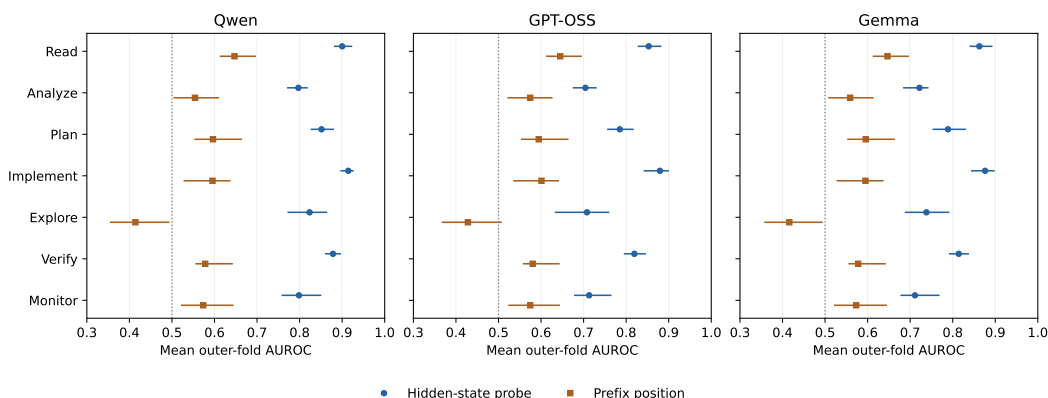


Figure 5.1: Prospective AUROC on response-disjoint outer test folds. Points are means of five fold scores and horizontal bars are 95% conditional response-bootstrap intervals. Both the hidden-state probe and the simple prefix-position baseline are fitted without the test responses.

lexical prefix content.

Ranking performance still does not imply reliable seven-class annotation. Macro binary F1 is 0.521 for Qwen, 0.400 for GPT-OSS, and 0.426 for Gemma at the unchanged decision rule. In Qwen, Explore reaches AUROC 0.823 but F1 only 0.341; Monitor reaches 0.799 and 0.355. The selected layers also vary across folds: Qwen Read uses indices 23–36 and Implement 18–32. This variability argues against interpreting one selected index as a unique reasoning location. The study intentionally produces fold-specific probes; the existing routing experiments retain their original measurement layers.

Across the five folds there are 1,505 Qwen, 945 GPT-OSS, and 1,155 Gemma fits, including 35 selected-layer refits and 35 position baselines per model. The numbers of inner candidate fits reaching the fixed iteration limit are 0, 2, and 2, respectively; the selected-layer refits have 0, 0, and 0 convergence warnings. All position fits converge, and all 5,000 bootstrap draws retain binary support for every reported target. Diagnostics and indices are retained rather than used to change the solver budget after evaluation.

The remaining inferential limits are the small number of responses and the scope of the test. The intervals condition on this single split plan, selected layers, and fitted classifiers; they omit refitting and split-seed variation. Same-

Table 5.5: Response-grouped results by target. AUROC and the paired AUROC gain over the position baseline include 95% conditional response-bootstrap intervals. Balanced accuracy (BA) and binary F1 are mean-fold point estimates at the unchanged decision threshold; their intervals are retained in the result files. Indices list the validation-selected hidden states in outer-fold order, not a globally selected layer.

Model	Target	AUROC	Gain over position	BA	F1	Indices (folds 1–5)
Qwen	Read	0.900 [0.881, 0.924]	+0.253 [0.209, 0.288]	0.805	0.599	34, 29, 36, 32, 23
Qwen	Analyze	0.797 [0.770, 0.819]	+0.243 [0.178, 0.296]	0.725	0.586	22, 26, 35, 22, 22
Qwen	Plan	0.852 [0.826, 0.881]	+0.255 [0.196, 0.292]	0.754	0.440	22, 29, 29, 29, 33
Qwen	Implement	0.914 [0.896, 0.927]	+0.319 [0.277, 0.385]	0.825	0.696	32, 31, 30, 19, 18
Qwen	Explore	0.823 [0.772, 0.865]	+0.409 [0.323, 0.478]	0.674	0.341	35, 31, 29, 34, 24
Qwen	Verify	0.878 [0.860, 0.897]	+0.300 [0.232, 0.325]	0.798	0.634	24, 24, 24, 20, 15
Qwen	Monitor	0.799 [0.758, 0.851]	+0.225 [0.156, 0.292]	0.671	0.355	21, 36, 22, 34, 31
GPT-OSS	Read	0.853 [0.827, 0.883]	+0.208 [0.164, 0.241]	0.724	0.528	21, 18, 17, 16, 13
GPT-OSS	Analyze	0.704 [0.675, 0.731]	+0.130 [0.072, 0.185]	0.609	0.416	16, 20, 17, 20, 20
GPT-OSS	Plan	0.785 [0.755, 0.818]	+0.191 [0.118, 0.232]	0.619	0.286	20, 18, 20, 23, 7
GPT-OSS	Implement	0.879 [0.841, 0.901]	+0.278 [0.233, 0.335]	0.768	0.640	16, 20, 18, 21, 18
GPT-OSS	Explore	0.708 [0.632, 0.760]	+0.280 [0.155, 0.369]	0.561	0.178	23, 22, 19, 11, 22
GPT-OSS	Verify	0.819 [0.794, 0.847]	+0.239 [0.174, 0.264]	0.699	0.517	16, 18, 18, 17, 16
GPT-OSS	Monitor	0.713 [0.678, 0.766]	+0.139 [0.078, 0.211]	0.589	0.233	16, 9, 21, 13, 22
Gemma	Read	0.863 [0.840, 0.893]	+0.216 [0.174, 0.253]	0.747	0.547	16, 20, 16, 19, 21
Gemma	Analyze	0.722 [0.683, 0.743]	+0.163 [0.088, 0.215]	0.653	0.488	6, 20, 20, 23, 22
Gemma	Plan	0.789 [0.753, 0.832]	+0.193 [0.129, 0.241]	0.655	0.337	19, 29, 16, 23, 16
Gemma	Implement	0.876 [0.843, 0.898]	+0.281 [0.244, 0.337]	0.779	0.644	24, 29, 20, 23, 25
Gemma	Explore	0.738 [0.687, 0.792]	+0.322 [0.229, 0.402]	0.572	0.185	17, 19, 17, 29, 19
Gemma	Verify	0.814 [0.791, 0.838]	+0.236 [0.167, 0.262]	0.725	0.543	25, 15, 28, 16, 15
Gemma	Monitor	0.711 [0.677, 0.769]	+0.138 [0.070, 0.217]	0.593	0.240	28, 0, 29, 25, 25

corpus response generalisation does not establish transfer to new benchmarks, annotation validity on the encoders’ own generations, or a causal role for the probed directions. The legacy external Qwen inspection below addresses a different population and decision rule.

5.2.3 External inspection of the Qwen probes

The external inspection covers 80 benchmark examples and 581 sentence boundaries. The seven-way argmax display assigns 286 units to Implement, 126 to Read, and 89 to Monitor, while Explore is never selected. Independent binary probes do not automatically yield calibrated multiclass probabilities: 74 units have no score above 0.5, whereas 180 exceed that threshold for more than one class. Taking an argmax still produces one label in both cases.

A subsequent rule-assisted pragmatic review provides a provisional reference, not independent human consensus or benchmark gold. The probes agree with its primary label on 292/581 units (50.3%) and with any accepted label on 353/581 (60.8%). The difference reflects 187 units for which the review accepts overlapping functions. Implement has the strongest accepted-membership F1, 0.729, while Explore has zero recall. These results identify useful qualitative structure and substantial transfer limitations. They do not validate the probes as a replacement for the contextual sentence judge, and the external review provides no direct evaluation of the Gemma or GPT-OSS probes.

5.3 Routing associations in generated solutions

5.3.1 Population, scoring, and termination

Each model’s observational corpus contains 4,647 attempts from 1,547 source problems. MATH-500, OlympiadBench, and Minerva contribute 500, 675, and 272 single-attempt problems. AIME24 and AIME25 each contribute 30 problems with 32 attempts, and AMC23 contributes 40 with 32 attempts. Consequently, the pooled attempt denominator weights each repeated-attempt problem 32 times as heavily as a single-attempt problem. Benchmark-specific

results are necessary even when the aggregate population is complete.

Table 5.6: All-attempt accuracy in the observational generation corpora. Invalid answers with known gold are counted as incorrect, and token-limit completions remain included. These are generation populations, not the held-out guiding baselines.

Benchmark	Attempts	Qwen	Gemma	GPT-OSS
MATH-500	500	0.690	0.706	0.664
AIME24	960	0.851	0.887	0.709
AIME25	960	0.805	0.824	0.696
OlympiadBench	675	0.477	0.517	0.422
AMC23	1,280	0.967	0.976	0.900
Minerva	272	0.268	0.301	0.114

The accuracy denominator includes missing or invalid extracted answers when gold is known. For Qwen, 3,568 of 4,647 attempts are correct. Of the 412 token-limit completions, only two are correct; accuracy among the remaining 4,235 attempts is 0.842. Excluding limit hits would therefore substantially change the population and apparent accuracy. Likewise, accuracy restricted to parseable answers would answer a different question. The retained failures make length and termination central to the interpretation of routing–correctness associations.

5.3.2 Confidence and other routing measurements

Table 5.7 shows that there is no common sign for the association between router confidence and correctness. Qwen’s coefficients are positive on all six benchmarks. Gemma is negative on AMC23 and Minerva; GPT-OSS is negative on MATH-500, AIME25, AMC23, and Minerva. Confidence summarises the concentration of the router distribution, not the probability that the mathematical answer is correct. Different model architectures, quantisation settings, and distributions of generated solutions can change its association with success.

The Qwen pointwise 95% source-problem bootstrap intervals exclude zero

Table 5.7: Whole-continuation router confidence versus correctness: point-biserial coefficients, with all attempts retained. Each model has the same benchmark counts, but these marginal correlations do not estimate controlled model differences.

Benchmark	Qwen	Gemma	GPT-OSS
MATH-500	+0.199	+0.080	-0.061
AIME24	+0.533	+0.392	+0.210
AIME25	+0.493	+0.313	-0.075
Olympiad	+0.243	+0.027	+0.051
AMC23	+0.175	-0.256	-0.077
Minerva	+0.112	-0.037	-0.160

for confidence on five benchmarks. AIME24 gives $r = 0.533$ with interval $[0.382, 0.678]$, while AIME25 gives $r = 0.493$ with $[0.302, 0.608]$. AMC23 has $r = 0.175$ with the much wider interval $[-0.116, 0.678]$. These intervals use 500 resamples and account for repeated attempts by retaining whole problems. They do not control for difficulty, length, or stopping behaviour, and they are pointwise intervals across an exploratory family of features and scopes.

Other measurements also resist a single interpretation. Qwen’s leading-expert margin is positively associated with correctness across benchmarks, but the selection-boundary margin is approximately zero or negative on AIME24 and AMC23. The former separates the first two experts; the latter concerns the inclusion boundary of the dispatched set. They should not be treated as interchangeable confidence measures. Likewise, selected probability mass and the mean selected-versus-rest probability gap are algebraically related for a fixed expert count and dispatch size; agreement between their correlations is not independent corroboration.

Token count is negatively associated with correctness in all reported model–benchmark combinations except Gemma on AMC23. For Qwen AIME24 and AIME25, its coefficients are -0.726 and -0.727 . This does not imply that shortening a solution would improve it: harder problems, unsuccessful search, and completion limits can jointly increase length and reduce accuracy. Hidden-

state measurements also vary across models. On AIME25, Qwen’s step-distance coefficient is $+0.404$, whereas GPT-OSS gives -0.404 . These descriptive differences argue against transferring a scalar routing or representation rule without model-specific validation.

Figures 5.2 and 5.3 extend the confidence comparison to the other routing and representation measurements, with their uncertainty. Separating the complete continuation from the reasoning span makes the measurement scope explicit. The panels compare descriptive estimates; interval overlap is not a paired test of differences between models.

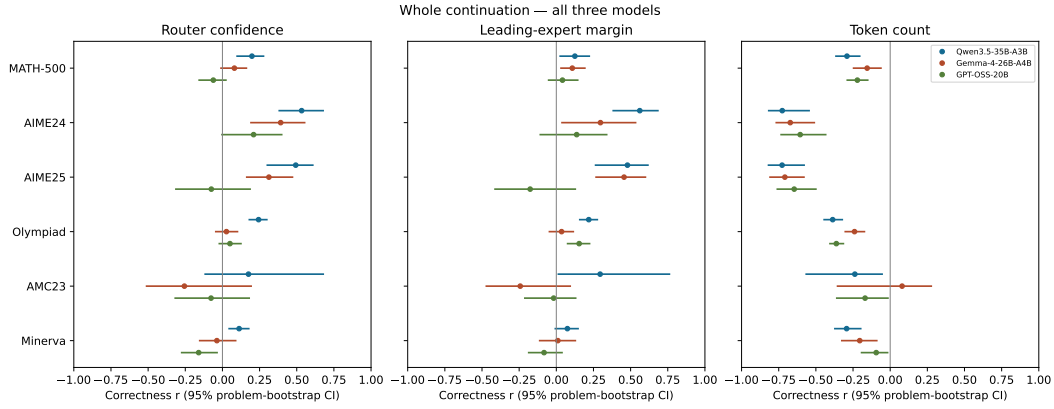


Figure 5.2: Whole continuation: saved correctness correlations and pointwise source-problem bootstrap intervals. Overlap or separation of intervals is not a paired test of model differences.

Figure 5.4 examines dependence among the candidate measurements themselves. This matters because several apparent outcome signals can reflect the same variation in length or router concentration. For notation, T denotes token count, C confidence, M the leading-expert margin, S selected probability mass, B the selection-boundary margin, G the selected/rest mean gap, H hidden-state norm, D hidden-state step distance, R hidden/router geometry, W top-one switch rate, and O adjacent selected-set overlap. Definitions are given in the methods chapter.

A correlation coefficient alone does not identify useful operating values. Figure 5.5 therefore shows accuracy against observed confidence and margin

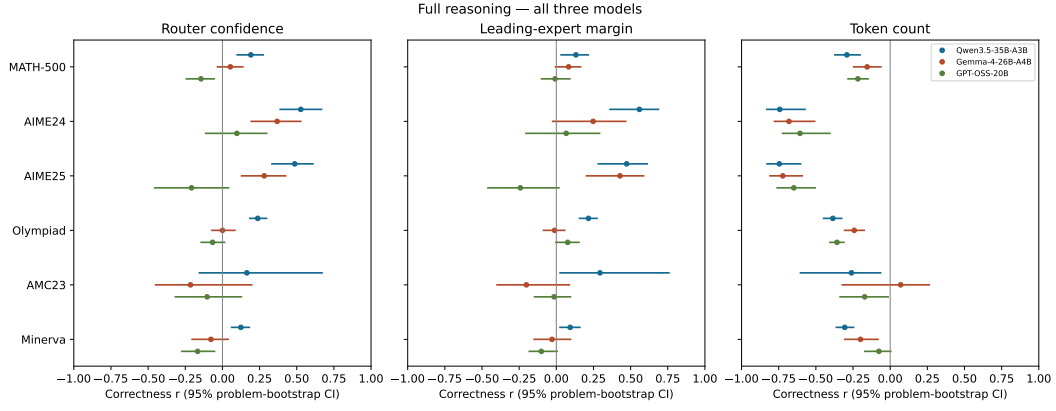


Figure 5.3: Full reasoning: saved correctness correlations and pointwise source-problem bootstrap intervals. Overlap or separation of intervals is not a paired test of model differences.

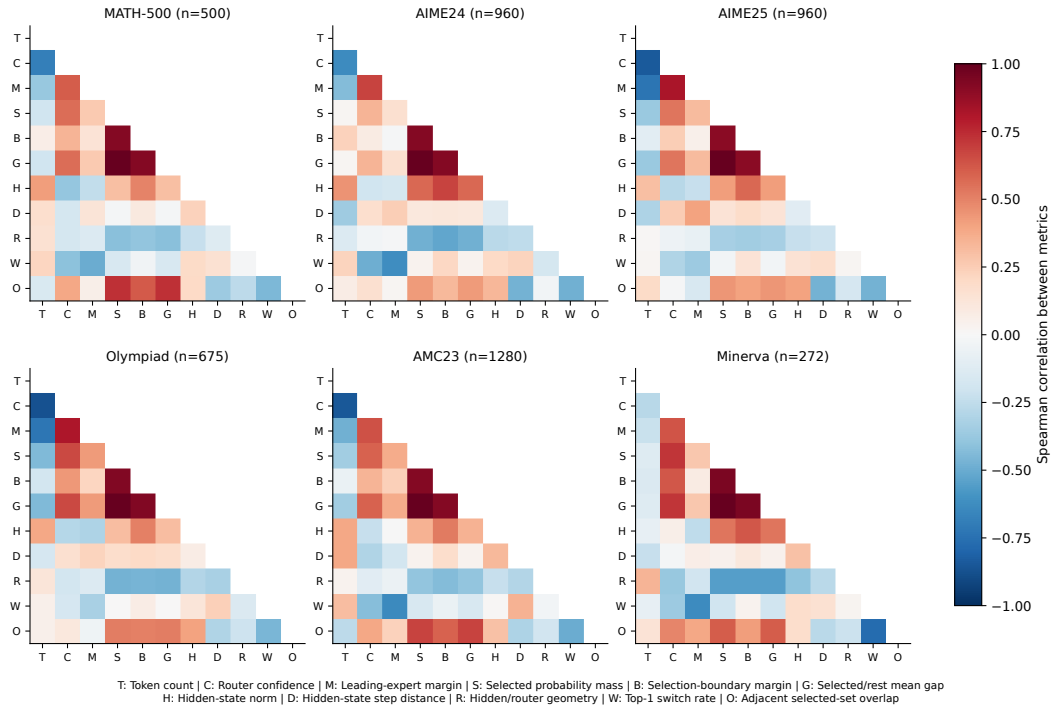


Figure 5.4: Whole-continuation metric-metric Spearman correlations across all six benchmarks. Lower triangles display each association once; diagonals and upper triangles are omitted. Repeated attempts and limit hits remain included. These are descriptive metric relationships, not correctness effects.

ranges for Qwen. These bins describe the saved population and do not define validated intervention thresholds.

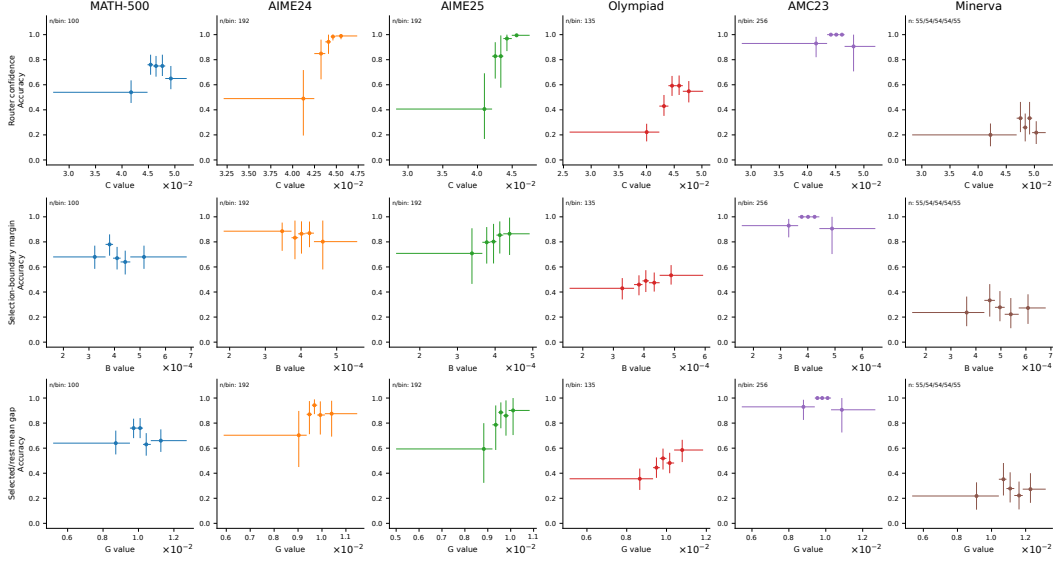


Figure 5.5: Accuracy at observed router-metric values. Each benchmark uses up to five quantile bins; horizontal bars show actual value ranges, points show mean values and accuracy, and panel labels give attempt counts per bin (left to right, or a shared count when equal). Vertical bars are pointwise 95% percentile intervals from 500 whole-source-problem bootstrap resamples within each fixed bin (seed 42). Degenerate intervals can occur in all-correct/all-incorrect bins and do not imply certainty. All attempts and limit hits are retained. Bins can share source problems, so their estimates are not independent; neither bin selection nor length and termination confounding is addressed by these intervals.

5.3.3 Repeated problems, reasoning classes, and position

Aggregation changes the estimand. For Qwen, mean problem-level confidence correlates with avg@32 at Spearman $\rho = 0.632$ on AIME24 and 0.705 on AIME25, but at -0.200 on AMC23, whose interval includes zero. The positive attempt-level AMC23 coefficient and negative problem-level rank coefficient

are therefore not interchangeable evidence. They compare different units and different outcome variation.

Figure 5.6 compares these problem-level associations across the three models. Its units are the 30, 30, and 40 source problems in AIME24, AIME25, and AMC23, respectively; all 32 attempts contribute to each problem’s feature mean and accuracy.

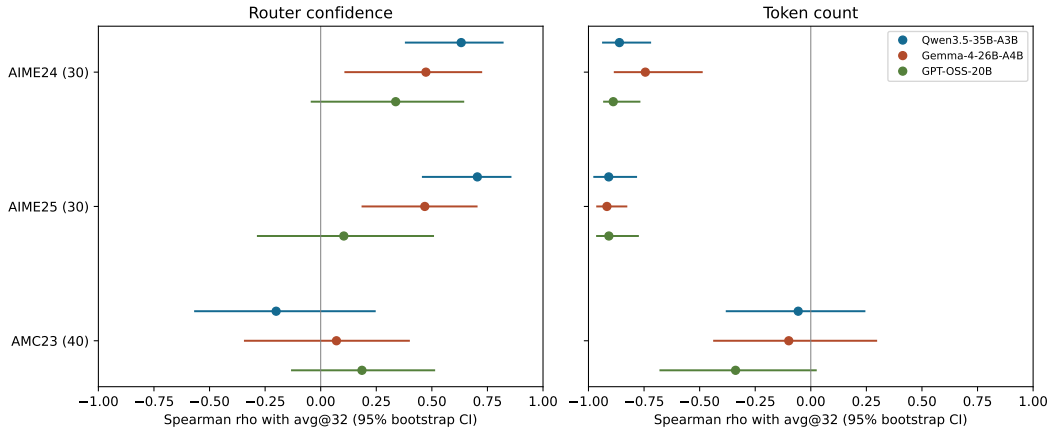


Figure 5.6: All models: whole-continuation problem means versus avg@32, with saved pointwise problem-bootstrap intervals. All 32 attempts enter each problem mean.

Within-problem contrasts provide a complementary view. Restricting Qwen to problems with both correct and incorrect attempts, mean confidence is higher in the correct attempts by 0.00268 on AIME24, 0.00123 on AIME25, and 0.01102 on AMC23. Their saved whole-problem intervals are $[0.00152, 0.00412]$, $[0.00065, 0.00176]$, and $[0.00455, 0.01709]$, respectively. These estimates use only 14, 15, and 6 mixed-outcome problems. They remove differences in problem identity but leave attempt-level length, termination, and trajectory differences unresolved.

Class-conditioned features retain only tokens assigned to a given episode class, while correctness still describes the whole generated answer. Figure 5.7 illustrates the resulting heterogeneity. For Gemma Implement tokens, confidence correlates with correctness at +0.498 on AIME24 (30 attempts) and -0.367 on AMC23 (40 attempts). The corresponding GPT-OSS coefficients

are $+0.164$ and -0.098 . Gemma’s AMC23 Plan, Explore, and Monitor cohorts contain only correct attempts, so their correctness correlations are undefined. Nonzero token coverage is not sufficient to estimate an association when the outcome has no variation. Neither a missing coefficient nor a sparse estimate should be interpreted as evidence that the class has no role.

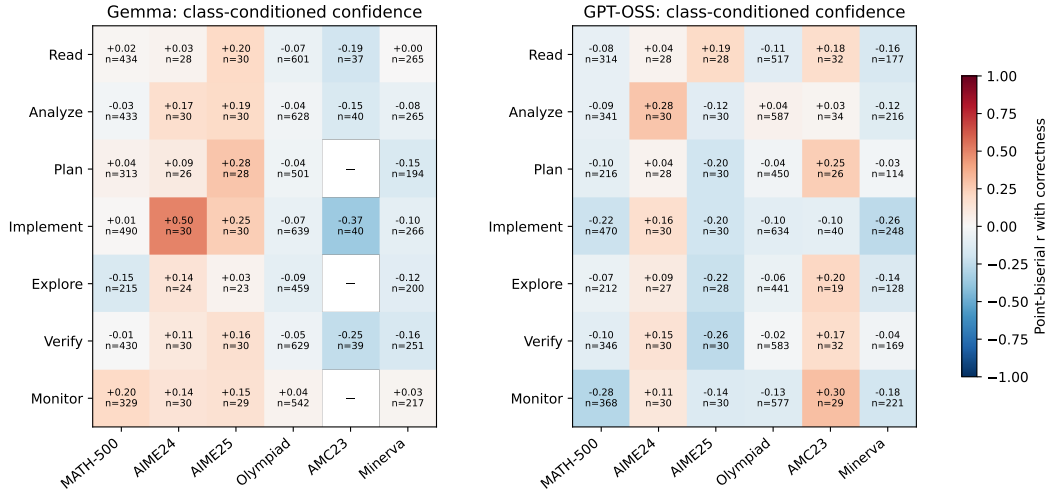


Figure 5.7: Class-conditioned router confidence versus whole-attempt correctness for the stratified Gemma and GPT-OSS cohorts. Cells show point-biserial coefficients and finite-attempt counts. Missing coefficients are undefined, not zero. The models share an annotation procedure but not matched labelled cohorts; this is a descriptive comparison.

The complementary Qwen class-conditioned screen appears in Figure 5.8. In addition to confidence, it includes hidden-state step distance, showing that class-conditioned associations depend on the measurement as well as the benchmark. The counts printed in each cell identify the available cohort rather than a common matched class sample.

Absolute position windows introduce a further selection effect. In Qwen, the mean reasoning length of 12,498.1 tokens defines ten windows of about 1,250 tokens and an overflow region. Coverage falls from 4,647 traces in the first window to 1,789 in overflow. Later-window correlations concern solutions that continued long enough to reach those windows; they are not within-trace changes on a fixed cohort. All available reasoning tokens enter position views

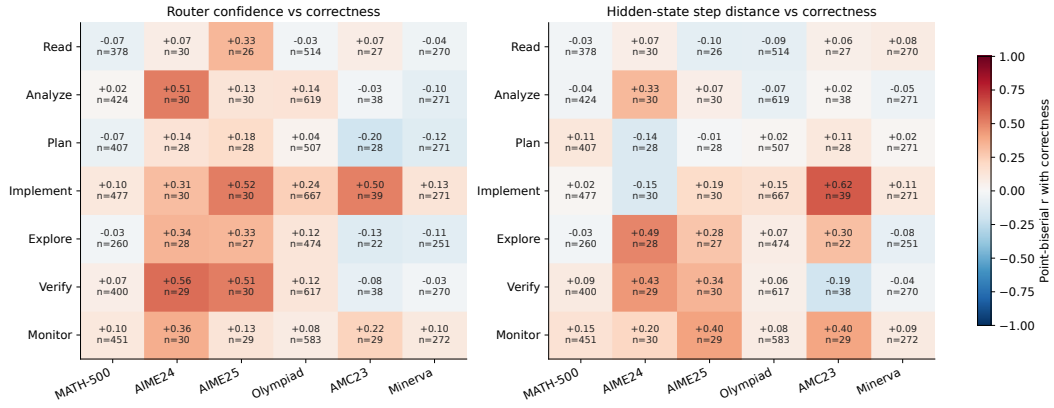


Figure 5.8: Class-conditioned confidence and hidden-state step distance versus whole-attempt correctness. Cells show point-biserial r and finite trace count. Classes use sampled Qwen3.8 labels on Qwen3.5 generations, with different token-bearing cohorts. No significance markings are implied.

irrespective of sentence annotation, so this change in coverage is distinct from missing class labels.

Table 5.8 gives the Qwen reasoning-length distribution used to interpret these windows. Figure 5.9 shows the corresponding position-dependent confidence associations and pointwise intervals. The changing token-bearing population is essential to interpreting the later windows.

Table 5.8: Reasoning length in tokens, all attempts including limit hits. This replaces the position-coverage count plot; lengths are observed under the generation budget.

Dataset	Attempts	Mean	Median	90th percentile
MATH-500	500	6,226.5	4,250	12,772
AIME24	960	15,908.4	14,011	32,766
AIME25	960	17,874.9	16,620	32,768
Olympiad	675	13,402.5	9,150	32,768
AMC23	1280	8,452.8	7,822	14,929
Minerva	272	9,805.0	7,465	19,533

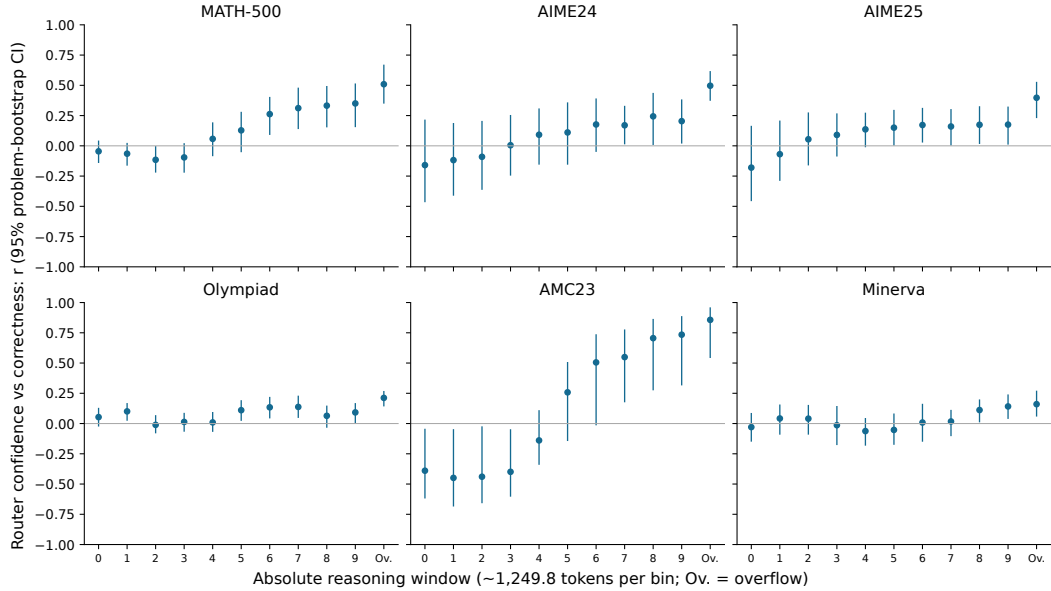


Figure 5.9: Router confidence versus whole-attempt correctness in absolute reasoning windows, with saved pointwise 95% source-problem bootstrap intervals. Each benchmark has its own panel. Missing estimates remain blank. Later windows use changing token-bearing cohorts; these are not within-trace causal effects.

Finally, the observational expert-identity screen is exploratory. It tests many top-one, selected-pool, and pair-use features, and a maximum selected after inspecting them is subject to selection bias. Some pooled screens combine numeric identities across measured layers, whereas an intervention requires a

specific (ℓ, e) pair. A visually strong association cannot be promoted directly to a portable expert target. The calibration procedure therefore retains layer-qualified identities and evaluates its policies on separate source problems.

5.3.4 Sampled class composition and outcome associations

The frequency of a predicted reasoning activity is distinct from the router features measured within that activity. Figure 5.10 shows the sampled label and benchmark composition for Gemma and GPT-OSS. These models share a stratified annotation procedure but retain different selected problems and traces; equal label budgets do not imply matched populations.

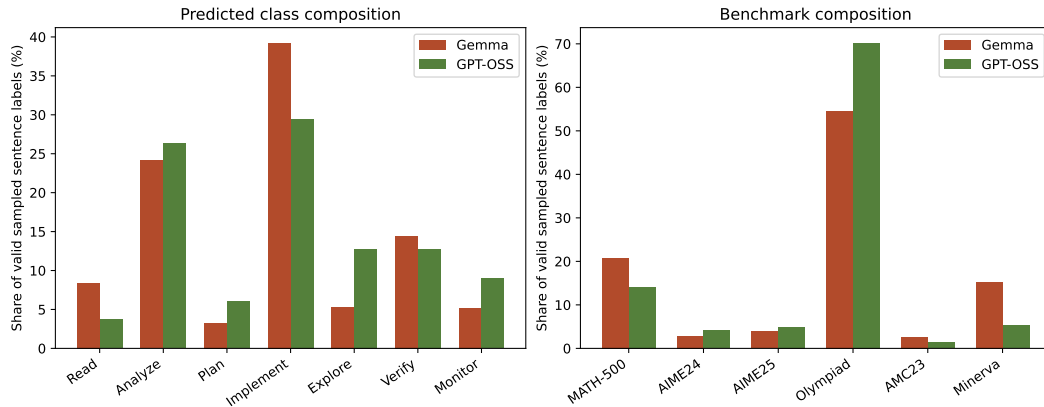


Figure 5.10: Gemma and GPT-OSS stratified label composition. Denominators are 99,999 and 100,000 valid sampled sentences. Equal sample budgets do not imply equal class or benchmark proportions. These sentence shares are neither complete-corpus token shares nor adjusted model effects.

For Qwen, Table 5.9 reports mean sentence shares, with each labelled sample-zero attempt receiving equal weight. An observed sample without a class contributes zero for that class; attempts without labels are excluded. These quantities describe the annotation sample, not complete-trace class prevalence. Figure 5.11 locates the sampled sentences along relative reasoning progress, while Figure 5.12 relates their shares to correctness and output length.

Because class shares sum to one, their correlations cannot be interpreted as independent class effects.

Table 5.9: Mean sampled sentence share (percent), with equal weight per labelled attempt. Empty-label attempts are excluded; absent classes in an observed sample have zero share. These are not full-corpus class proportions.

Dataset	Attempts	Read	Analyze	Plan	Implement	Explore	Verify	Monitor
MATH-500	499	8.9	19.5	9.3	29.3	4.1	11.8	17.1
AIME24	30	6.7	33.9	2.9	28.3	7.0	14.4	6.8
AIME25	30	5.7	33.2	4.8	27.9	6.7	13.4	8.3
Olympiad	675	6.9	26.8	6.0	29.7	6.4	13.2	11.0
AMC23	40	6.6	25.9	5.8	32.0	3.8	15.2	10.6
Minerva	272	9.4	19.8	8.4	21.8	7.7	10.3	22.6

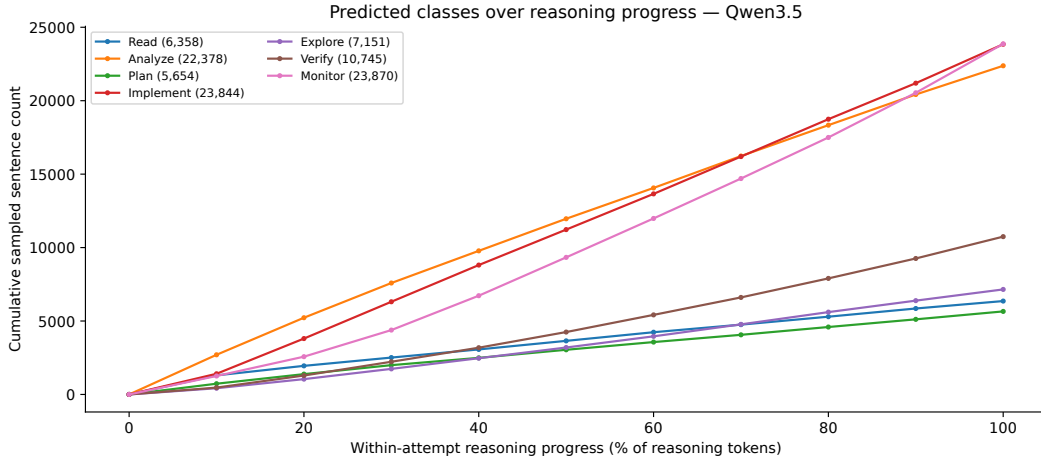


Figure 5.11: Cumulative counts of sampled sentences by predicted class, pooled across benchmarks. Counts start at zero; a sentence is counted when its last reasoning token is reached. Progress is normalized by each attempt’s full reasoning-token length, unlike the absolute windows in Figure 5.9. Legend totals equal the rightmost endpoints. These are raw sampled counts, not cumulative percentages or estimates of full-corpus counts; longer traces and benchmark sampling quotas affect them.

Appendix A retains the benchmark-specific class-share outcome curves, row-normalised transition matrix, and complete expert-ID screens. Those views

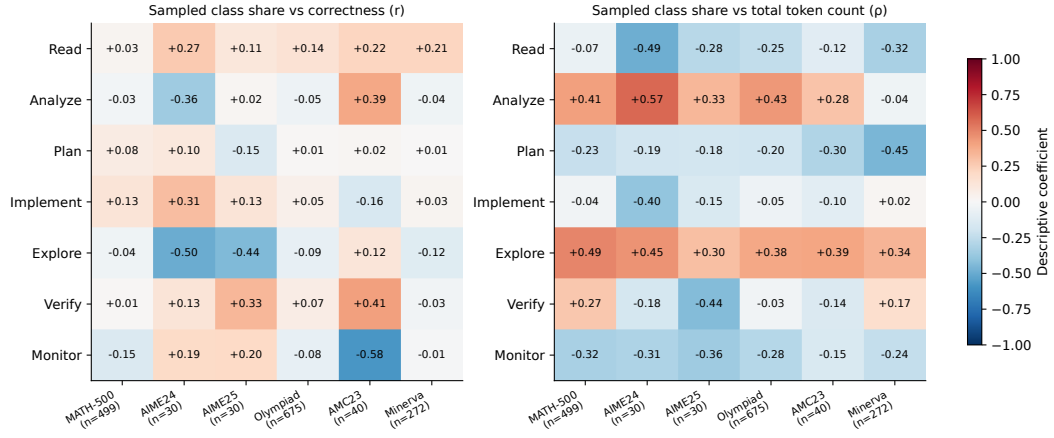


Figure 5.12: Sampled class proportions versus whole-attempt correctness (point-biserial r) and total continuation-token count (Spearman ρ). All labelled attempts enter each class, including observed zero proportions. These differ from correlations of router features within class-selected tokens. No significance claim is made; class shares sum to one and cannot be interpreted as independent effects.

give the denominators and population detail behind the compact comparisons without selecting only the largest observed associations.

5.3.5 Directed class transitions by answer correctness

For each model, we separate labelled attempts by whole-answer correctness and count directed transitions between originally adjacent labelled sentences. Rows are source classes and columns are destination classes. Self-transitions are retained; missing labels and sampling gaps are never bridged. Each cell is the total observed count divided by the number of labelled attempts in that outcome group: equivalently, the mean per-attempt count, including zero-pair attempts. Empty-label attempts are excluded. These are counts per attempt, not row-normalized probabilities or full-trace transition estimates.

The two panels per model pool the six benchmarks and all six panels share one colour scale. Benchmark-specific counts and denominators are saved in the accompanying statistics JSON. Outcome groups differ in benchmark mix,

reasoning length, and label coverage; Qwen also retains a different annotation protocol from Gemma and GPT-OSS. Differences are descriptive associations, not evidence that a transition causes correctness or a controlled comparison between models.

5.4 Held-out routing interventions

5.4.1 Matched comparisons and uncertainty

The global intervention split assigns 1,082 source problems to calibration and 465 to evaluation. The evaluation set contains 150 MATH-500, 9 AIME24, 9 AIME25, 203 OlympiadBench, 12 AMC23, and 82 Minerva problems. Keeping 32 attempts for each AIME and AMC problem produces 1,395 matched attempts. All attempts from one problem remain on the same side of the split. For the original Qwen policies, calibration uses 3,014 scored traces from 1,001 problems after excluding 238 traces with missing saved correctness. This exclusion differs from counting invalid known-gold answers as incorrect in the observational analysis, and can affect the selected expert targets.

Each row in the complete-comparison tables compares baseline and guided runs with matching problem IDs, prompts, prompt token IDs, per-attempt sampling settings, engine configuration, and recorded software versions. Baselines are not interchanged between rows. Stochastic generation uses temperature 0.6, top- $p = 0.95$, and a 32,768-token output limit. The primary accuracy summaries are attempt-weighted. Section 5.4.8 adds benchmark-level results and a secondary analysis giving equal weight to each source problem; neither aggregation is a benchmark macro-average.

Accuracy-change intervals use 5,000 paired, dataset-stratified source-problem bootstrap resamples with seed 42. A sampled problem carries all of its attempts and both conditions together. This preserves the paired contrast and dependence between repeated attempts. The intervals are conditional on the fitted policy and saved decoding runs. They do not incorporate policy refitting, independent decoding seeds, or adjustment for the multiple tested strengths and methods. In particular, each AIME evaluation has only nine

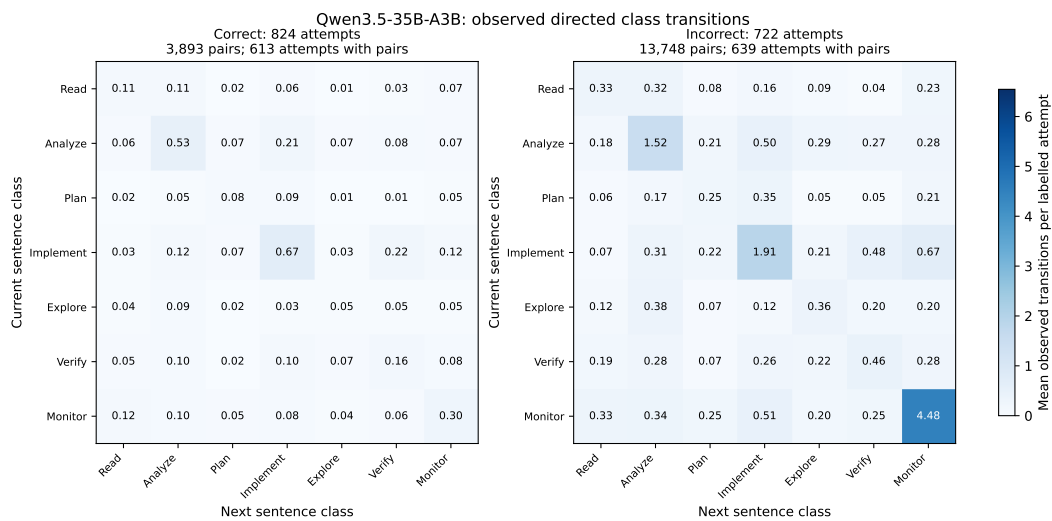


Figure 5.13: Qwen3.5-35B-A3B: directed class transitions for correct and incorrect answers. Cells show mean observed counts per labelled attempt, including self-transitions on the diagonal. Panel headings give denominators and adjacent-pair coverage; zero-pair attempts remain in the mean. The shared colour scale is identical across all three models. Sampling and population differences limit interpretation.

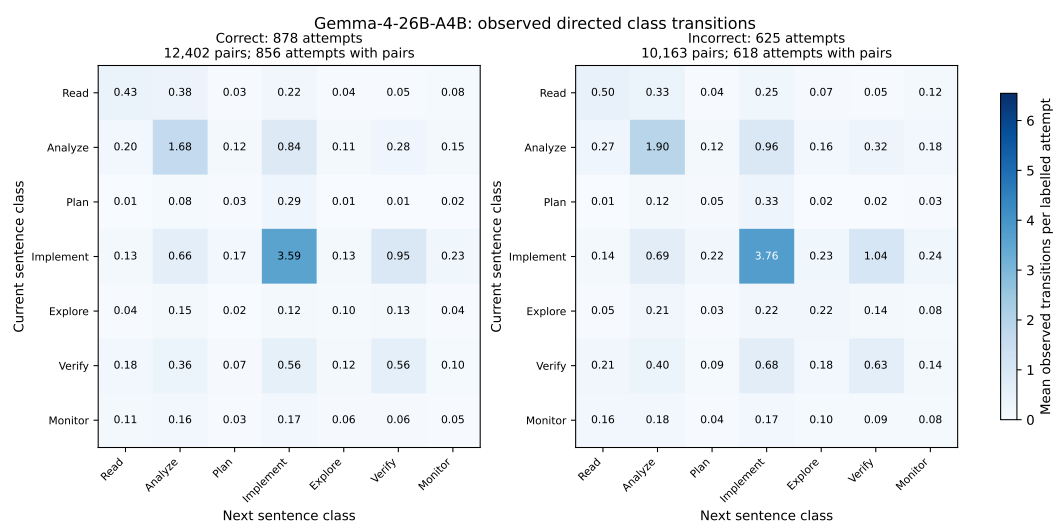


Figure 5.14: Gemma-4-26B-A4B: directed class transitions for correct and incorrect answers. Cells show mean observed counts per labelled attempt, including self-transitions on the diagonal. Panel headings give denominators and adjacent-pair coverage; zero-pair attempts remain in the mean. The shared colour scale is identical across all three models. Sampling and population differences limit interpretation.

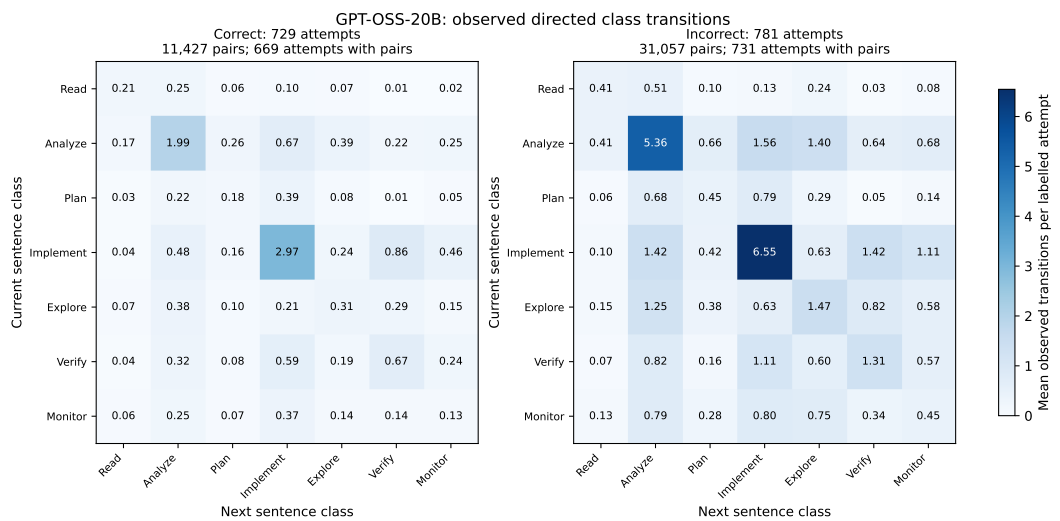


Figure 5.15: GPT-OSS-20B: directed class transitions for correct and incorrect answers. Cells show mean observed counts per labelled attempt, including self-transitions on the diagonal. Panel headings give denominators and adjacent-pair coverage; zero-pair attempts remain in the mean. The shared colour scale is identical across all three models. Sampling and population differences limit interpretation.

source problems. An interval containing zero does not establish equivalence, and an interval excluding zero is interpreted within this exploratory design.

5.4.2 Expert-selection comparison with SteerMoE

The comparison separates expert detection from the intervention applied to those experts. SteerMoE uses paired contrastive inputs and counts top- k expert inclusions separately in the two populations. Its score is the token-pooled activation-rate difference

$$\Delta_{\ell e}^{\text{paper}} = \frac{A_{\ell e}^+}{N^+} - \frac{A_{\ell e}^-}{N^-}, \quad (5.1)$$

where A counts tokens selecting the layer-expert identity and N counts all tokens in the corresponding population. Detection therefore uses routing frequencies, not hidden-state similarity. Positive and negative differences identify opposite behavioral associations; steering changes the selected experts' log-softmax scores to the original maximum plus ϵ or minimum minus ϵ [22], Sections 3.1–3.2. The authors' released custom-steering notebook ranks layer-expert identities together by absolute risk difference¹. Our policy instead assigns a separate budget to each of a preselected set of layers.

Implemented estimator. For calibration attempt r , let $y_r \in \{0, 1\}$ be correctness, $f_{r\ell e}$ the fraction of saved routing tokens selecting expert e at layer ℓ , and $w_r = 1/m_{p(r)}$, where $m_{p(r)}$ is the number of usable attempts for its source problem. With $W = \sum_r w_r$, the implemented score is

$$\pi = \frac{\sum_r w_r y_r}{W}, \quad L_{\ell e} = \frac{\sum_r w_r f_{r\ell e} y_r}{\sum_r w_r f_{r\ell e}} - \pi. \quad (5.2)$$

Thus calibration gives every problem total weight one and first normalizes routing counts within each trace. It contrasts observed correct and incorrect completions; it does not construct paired counterfactual behavioral inputs. Experts need at least four supporting problems in the audited policies. Within each selected layer, eligible experts are ordered by signed lift, then support,

¹https://github.com/adobe-research/SteerMoE/blob/main/custom_steering.ipynb

then expert ID, and capped at native top- k (eight for Qwen and four for GPT-OSS). Fixed bias scales each retained lift by the largest retained magnitude; paper-style steering uses only membership in the selected set, not those relative magnitudes.

Why activation-based scores need not select the same experts. Define $\mu_1 = E_w[f_{rle} \mid y_r = 1]$, $\mu_0 = E_w[f_{rle} \mid y_r = 0]$, and $\bar{f} = E_w[f_{rle}]$. Algebra applied to our estimator gives

$$L_{le} = \frac{\pi(1-\pi)}{\bar{f}}(\mu_1 - \mu_0), \quad \bar{f} > 0. \quad (5.3)$$

Under these same weights, lift and activation-rate difference have the same sign but can have different rankings: lift additionally divides by the expert’s mean activation frequency. This identity does not equate either score to the paper’s token-pooled estimate on different, paired inputs.

The saved frequency masses, activation-weighted accuracies, and calibration baseline suffice to reconstruct $\mu_1 - \mu_0$ without replaying a model. Table 5.10 changes only the ranking score and retains our weights, sign filter, support threshold, layers, and target budget. The audit also reconstructs every saved selected set and normalized score. Only 14 of 48 Qwen positive layer–expert identities remain selected; the four-target GPT-OSS policies retain 6 of 16 positive identities and 7 of 16 negative identities. This demonstrates selection sensitivity even before changing the data construction or token weighting. It does not measure intervention performance for these alternative sets.

Table 5.10: Selection sensitivity using saved calibration sufficient statistics. RD here uses the same problem-balanced trace weights, support filter, polarity, layer set, and per-layer budget as lift; it is not a reproduction of the paper’s token-pooled contrastive estimator. Overlap counts layer–expert identities.

Model	Polarity	Layers	Overlap	Overlap per layer
GPT-OSS	negative	4	7/16	15: 1/4, 19: 2/4, 21: 1/4, 23: 3/4
GPT-OSS	positive	4	6/16	15: 2/4, 19: 1/4, 21: 1/4, 23: 2/4
Qwen	positive	6	14/48	27: 4/8, 34: 3/8, 35: 2/8, 36: 1/8, 38: 3/8, 39: 1/8

Interpretation of the intervention comparison. The completed Qwen fixed runs share selection statistics and target sets. The completed GPT-OSS fixed and paper-style conditions use four targets per layer, with the same selected identities within each polarity. These comparisons retain our selector and do not evaluate the full SteerMoE detection and steering pipeline.

The completed OSS comparison already holds the selected identities fixed within each polarity while changing the intervention rule. A selector comparison should instead fix the split, generation contract, layers, budget, and intervention, changing only the ranking estimator. The present OSS budget is four targets per layer, matching native dispatch; smaller budgets of one or two targets would test less extensive promotion. Matched random sets would help separate selector information from generic perturbation. A faithful detection replication additionally requires paired contrastive inputs and token-pooled activation counts. The analysis uses saved calibration statistics and does not evaluate new intervention runs for the alternative selected sets.

5.4.3 Consistent offline answer-equivalence audit

The qualitative inspection identified equivalent final answers rejected by the historical normalized-string/numeric fallback. A uniform offline pass therefore rescores both conditions in every saved, manifested identity and margin evaluation: 13 complete full stochastic comparisons, two complete greedy comparisons, the 39-problem Minerva comparison, and the interrupted Qwen subset. It reads 40,097 stored condition records, including reused baseline copies; this is not a count of independent generated solutions. The original text, tokens, manifests, policies, and judgments are preserved. Revised decisions and all changed answers are recorded separately in `report/guiding_rescoring.json`.

The rule first extracts the last nonempty balanced boxed candidate, then uses the existing answer-marker and last-number fallbacks when no such box is present. Only that candidate is compared with the gold answer. It normalizes display notation, plain-text labels, common units, currency, and explicit list separators. Mathematical comparison uses Math-Verify 0.9.0 with la-

tex2sympy2_extended 1.11.0, ANTLR 4.13.2, and SymPy 1.14.0². Variables and structured objects use strict comparison; exact-number answers require equality, while decimal/scientific scalar answers allow relative tolerance 10^{-6} with no absolute floor. Thus zero does not match a small nonzero answer merely because its magnitude is below 10^{-6} . Scientific notation is parsed numerically, rather than as a symbolic expression involving e . Text denoting multiple answers is not silently reduced to its first number. Standard unit annotations follow the parser’s normalization conventions; this is not a dimensional-analysis check.

The pass changes 4,054 stored judgments from incorrect to correct and one from correct to incorrect. It also changes the extracted candidate in 2,065 records. The single downward change concerns a capped Qwen baseline whose extracted box contains the literal placeholder “answer” rather than its earlier saved numeric fallback. These counts include repeated baseline copies. Changes occur on every benchmark, mainly OlympiadBench, MATH-500, and Minerva. The analysis reruns the same 5,000 paired, dataset-stratified source-problem bootstrap resamples (seed 42), including the equal-problem sensitivity statistic. The 494-ID interrupted comparison remains separate and its accuracy counts are unchanged.

Table 5.11: Selected changes after applying the same revised scoring rule to both conditions. Changes and 95% intervals are percentage points. Attempt weighting is primary; problem weighting is secondary. Historical values are retained here only to document the audit and are not mixed into current result tables.

Condition	Weighting	Historical change	Revised change	Revised 95% CI
Qwen margin (c=25)	Attempt	+1.08	+0.93	[0.00, +1.86]
GPT-OSS margin	Attempt	-3.15	-0.86	[-2.87, +1.15]
GPT-OSS fixed positive +2	Attempt	-1.43	-3.30	[-6.16, -0.93]
GPT-OSS fixed positive +2	Problem	+4.87	-0.10	[-3.12, +3.08]
GPT-OSS fixed negative +1	Problem	+4.94	+2.35	[-0.58, +5.44]
Qwen greedy margin	Attempt	+2.15	+0.86	[-1.29, +3.01]

²<https://github.com/huggingface/Math-Verify>

No revised positive accuracy interval excludes zero in the complete comparisons under either reported weighting. In particular, the earlier positive secondary OSS findings and greedy Qwen margin interval do not survive the common scoring rule. The Qwen concurrency-25 margin lower endpoint is exactly zero. Strong negative effects under paper-style promotion remain. Generation costs and termination flags are unchanged.

The checker is tested on equivalent and distinct fractions, text labels, scientific notation, ordered answers, units, currency, multiple-answer text, and very small numerical values, as well as final-candidate extraction. It does not provide independent human adjudication. There are 699 records with an unparsed candidate or reference, 332 with unsupported embedded text, and 30 parser/comparison errors; these receive an incorrect decision unless normalized display equality already succeeds. Error categories and changed judgments remain available for further review. A two-second parser/comparison timeout and a 2,048-character limit for nonnumeric expressions bound degenerate inputs. Calibration and observational correlation labels retain their original values: the revised evaluations test the existing frozen policies, not policies refitted with revised labels.

5.4.4 Identity selection and intervention strength

Table 5.12 distinguishes polarity, which defines the selected expert set, from signed strength, which defines whether that set is promoted or suppressed. The fixed method uses the lift-weighted additive bias. The implementation also supports the paper-style extreme-score replacement described in Section 3.4.2; the completed comparisons in this table evaluate fixed bias.

For Qwen fixed positive-polarity guiding, strength +1 changes accuracy by -0.29 points with interval $[-1.15, +0.65]$, and strength -1 gives $+1.08$ with $[-0.14, +2.37]$. Increasing promotion to strength +2 gives -2.80 points with $[-5.16, -0.72]$. Thus stronger promotion reduces primary attempt-weighted accuracy under the revised scorer.

All seven GPT-OSS identity conditions use four targets at each of layers 15, 19, 21, and 23, matching native routing top- $k = 4$. Each evaluates 1,395

Table 5.12: Complete stochastic identity comparisons. B/G denotes baseline/guided accuracy in percent; changes and intervals are percentage points. Each row uses its own matched baseline and 1,395 attempts on 465 problems. Intervals use 5,000 paired, dataset-stratified problem resamples, seed 42.

Model	Method	Polarity	Strength	B accuracy	G accuracy	Change	95% CI
GPT-OSS	Fixed	Negative	+1	77.85	78.42	+0.57	[-1.58, +2.65]
GPT-OSS	Paper	Negative	+1	77.85	18.28	-59.57	[-67.24, -51.40]
GPT-OSS	Paper	Positive	-1	77.85	75.63	-2.22	[-4.16, -0.43]
GPT-OSS	Paper	Positive	+1	77.85	16.20	-61.65	[-69.32, -53.41]
GPT-OSS	Fixed	Positive	-1	77.85	76.85	-1.00	[-3.44, +1.22]
GPT-OSS	Fixed	Positive	+1	77.85	77.85	+0.00	[-2.15, +2.01]
GPT-OSS	Fixed	Positive	+2	77.85	74.55	-3.30	[-6.16, -0.93]
Qwen	Fixed	Positive	+1	84.95	84.66	-0.29	[-1.15, +0.65]
Qwen	Fixed	Positive	-1	84.09	85.16	+1.08	[-0.14, +2.37]
Qwen	Fixed	Positive	+2	84.09	81.29	-2.80	[-5.16, -0.72]

matched attempts from 465 held-out problems at concurrency 25, with a frozen chat-template date of 22 September 2026. Rescored baseline accuracy is 77.85% (1,086/1,395) in every condition. Fixed positive strengths +1, +2, and -1 give changes of 0.00, -3.30, and -1.00 points. Only the strength+2 interval excludes zero: [-6.16, -0.93]. Fixed negative-polarity promotion gives +0.57 points with [-1.58, +2.65] and does not establish a benefit.

Paper-style promotion at strength +1 reduces accuracy to 16.20% (226/1,395) for positive targets and 18.28% (255/1,395) for negative targets. Changes are -61.65 points with [-69.32, -53.41] and -59.57 with [-67.24, -51.40], respectively. Mean generated length rises from 6,497 to 22,489 and 23,639 tokens; token-limit hits increase from 74 to 670 and 815. Suppressing the positive set with paper-style strength -1 gives 75.63% accuracy and a smaller but below-zero primary change of -2.22 points ([-4.16, -0.43]); mean length is 6,698 tokens with 106 limit hits.

Paper-style steering adapts the extreme-score intervention with $\epsilon = 0.01$ while retaining our accuracy-lift selector [22]; it is not a full reproduction of SteerMoE. The four-target budget does not prevent severe degradation from promotion. Promoting a full top- k set fixes membership and equalises its hook

scores; this is a possible mechanism, not a verified explanation of termination.

5.4.5 Interrupted Qwen paper-style intervention: partial results

The Qwen positive-expert paper-style intervention at strength +1 was interrupted after 494 of 1,395 planned attempts (35.4%). 901 attempts have no saved guided result. This policy targets eight experts at each of six layers, matching the model routing top- $k = 8$. The intervention uses epsilon 0.01, concurrency 25, and a 32,768-token completion budget. Only saved completed attempts are analysed.

On the 494 exact matched baseline/guided IDs (16 source problems), baseline accuracy is 70.45% (348/494), and guided accuracy is 0.81% (4/494), a change of -69.64 percentage points. There are 4 wrong-to-right and 348 right-to-wrong flips. Mean generated length changes from 19,352 to 32,768 tokens (medians 17,252 and 32,768); token-limit hits change from 110/494 to 494/494.

Table 5.13: Partial Qwen paper-style intervention. Accuracy and token counts compare only the same completed IDs in each condition. B/G denotes baseline/guided. A dash indicates no completed guided attempts, not zero accuracy.

Benchmark	Completed/planned	Problems	Accuracy B/G (%)	Change (pp)	Mean tokens B/G	Limit hits B/G
AIME24	288/288	9	61.81/0.00	-61.81	20,947/32,768	74/288
AIME25	206/288	7	82.52/1.94	-80.58	17,121/32,768	36/206
AMC23	0/384	0	—	—	—	—
MATH-500	0/150	0	—	—	—	—
Minerva	0/82	0	—	—	—	—
Olympiad	0/203	0	—	—	—	—

These are descriptive partial results, excluded from the complete-run comparison and weighting tables. The completed subset is not a random sample: benchmark submission order, scheduling, and generation length can affect which requests have finished. Multiple attempts from one problem are also dependent. Consequently the pooled difference is not an estimate for the full 1,395-attempt evaluation, and no population confidence interval or significance claim is reported. Missing attempts are not counted as wrong or filled from the

baseline. Increased token-limit incidence does not by itself establish repetitive looping.

All saved guided records have unique IDs, scored boolean outcomes, matching baseline inputs, prompt token IDs, and per-attempt sampling settings. Policy, engine, software, scoring, and template-date contracts also match; hook diagnostics confirm activity at all selected layers. Source hashes and benchmark denominators are saved in `report/partial_guiding_statistics.json`.

5.4.6 Qualitative inspection of paper-style failures

A reproducible, assistant-assisted excerpt inspection examines 22 saved historically baseline-correct/guided-incorrect pairs from strength-+1 paper-style promotion. For each complete GPT-OSS polarity, four cases are sampled without replacement from outputs ending at the token limit and four from outputs ending normally. Six additional capped cases are sampled from the interrupted Qwen positive run. Sampling uses sorted attempt IDs and seed 20260924, with separate streams per condition and termination stratum. The OSS samples cover eight and seven distinct problems; the Qwen sample covers four. Selection conditions on recorded correctness loss and termination, and repeated attempts can share a problem. The resulting counts describe this sample, not the prevalence of failure modes.

The review examines fixed beginning, middle, and ending excerpts, together with the most frequent exact 16-word span, against the corresponding baseline answer. Repetition is described from the text, not inferred from output length alone. The whole-text diagnostic counts case-sensitive, whitespace-delimited spans, allowing overlap; it does not capture every form of repetition, particularly unspaced symbol chains. This is neither independent human annotation nor exhaustive verification of the derivations.

Repetition and termination. All 14 sampled capped outputs contain repeated arithmetic or verbal cycles and lack a boxed-answer marker. Five of the eight normally ended OSS outputs also contain cycles before returning an incorrect boxed answer. Across these 19 repetitive outputs, the most frequent

Table 5.14: Descriptions of 22 cases sampled as correctness losses under the original scorer. Capped/stopped gives the sampled termination strata. Repetition can coexist with mathematical errors; the arithmetic-error column describes the remaining case without an observed long cycle. Formatting disagreement denotes an equivalent final answer rejected by the saved scorer. Counts are conditional on the sampling design.

Condition	Capped/stopped	Repetition	Arithmetic error	Formatting	No box
GPT-OSS positive	4/4	6	0	2	4
GPT-OSS negative	4/4	7	1	0	4
Qwen positive (partial)	6/0	6	0	0	6

exact 16-word span occurs 62–9,906 times; the matched baseline maxima range from one to four. For example, positive OSS case P03 repeats 390? while its baseline returns 49. Qwen case Q06 repeatedly emits “Wait, wait.” while its baseline returns 82. Negative OSS case N06 repeats an unresolved sum and eventually returns 112 instead of 144. Thus repetition can accompany both failure to finish and a normally terminated wrong answer. The separate non-cyclic arithmetic example N05 correctly enumerates three, two, and two choices but evaluates $3 \times 2 \times 2 = 6$, returning 6 instead of 12.

Scoring disagreement. Two normally ended positive OSS cases expose formatting sensitivity in the saved fallback scorer. Case P05 returns 14/15, equivalent to the gold $\frac{14}{15}$; case P08 returns **East**, equivalent to the gold `\text{east}`. Both are stored as incorrect, whereas their baselines use the gold LaTeX representation and are stored as correct. The recorded method is `normalized_exact_numeric_fallback`; its normalized-string or floating-point comparison rejects these pairs. The classification concerns final-answer equivalence, not the validity of every intermediate step. The subsequent offline audit in Section 5.4.3 corrects both examples and applies the same rule to all saved baseline and guided evaluations. Current accuracy and bootstrap tables use those revised decisions. This qualitative sample retains its original IDs and selection based on historical judgments; P05 and P08 are consequently no longer correctness losses under the revised rule. Resampling after correction

would answer a different qualitative question.

Scope of interpretation. The review supplies direct text examples of repetition, incorrect arithmetic, and scoring artefacts. It does not isolate why the routing intervention causes a particular trajectory, quantify population-level looping, or extend the interrupted Qwen result to its missing attempts. Full-population termination counts remain those in the generation-diagnostic tables. The sampled IDs, source hashes, excerpts, text diagnostics, and case-level notes are stored in `report/paper_failure_inspection.json` and `report/paper_failure_review.json`.

5.4.7 Margin guiding and generation cost

All three complete stochastic margin comparisons have intervals containing zero after consistent rescoring. Qwen at concurrency 16 gives +0.07 points ($[-0.86, +1.01]$); Qwen at concurrency 25 gains 13 net correct attempts, or +0.93 points ($[0.00, +1.86]$). GPT-OSS loses 12 net correct attempts, or -0.86 points ($[-2.87, +1.15]$). The Qwen concurrency-25 lower endpoint is zero, so its positive point estimate does not establish a direction under this interval criterion.

Table 5.15: Complete stochastic margin-guiding comparisons at strength 1. B/G denotes baseline/guided accuracy in percent; changes and intervals are percentage points. Concurrency is the maximum number of active sequences. Each row has its own matched baseline; differences between rows are not a controlled concurrency experiment.

Model	Concurrency	B	G	Change	95% CI
Qwen	16	84.30	84.37	+0.07	$[-0.86, +1.01]$
Qwen	25	84.09	85.02	+0.93	$[+0.00, +1.86]$
GPT-OSS	25	77.85	76.99	-0.86	$[-2.87, +1.15]$

The small margin effects do not establish benefit or harm after rescoring. The two Qwen configurations have different baseline contracts, so their

point-estimate difference cannot be attributed specifically to concurrency. No independent configuration-selection set was used.

Table 5.16 reports generation cost and limit hits. In the positive Qwen margin comparison, mean generated tokens change from 13,830 to 13,810 and limit hits from 154 to 146. GPT-OSS margin guiding changes mean tokens from 6,497 to 6,409 with an accuracy interval containing zero. A reduction in average tokens is therefore not sufficient to claim useful efficiency. These are descriptive token statistics; no paired token-cost confidence interval or equal-accuracy compute comparison is available here.

Table 5.16: Generation diagnostics for the stochastic comparisons. Means count all generated tokens, including final answers; limit hits are counts among 1,395 attempts. B/G denotes baseline/guided, and c denotes concurrency. Token differences are descriptive and have no uncertainty interval here.

Condition	Mean tokens B	Mean tokens G	Limit hits B	Limit hits G
Qwen fixed positive +1	13863	13706	140	146
Qwen fixed positive +2	13830	14092	154	169
Qwen fixed positive -1	13830	13919	154	138
GPT-OSS fixed positive +1	6497	6580	74	72
GPT-OSS fixed positive +2	6497	8629	74	164
GPT-OSS fixed positive -1	6497	6371	74	60
GPT-OSS fixed negative +1	6497	6900	74	87
GPT-OSS paper positive +1	6497	22489	74	670
GPT-OSS paper positive -1	6497	6698	74	106
GPT-OSS paper negative +1	6497	23639	74	815
Qwen margin (c=16)	13848	13840	144	143
Qwen margin (c=25)	13830	13810	154	146
GPT-OSS margin (c=25)	6497	6409	74	83

The original Qwen margin run also contains full hook diagnostics. Across selected layers, the top-eight identity set changes on 32.7–41.8% of 20,657,039 hook token evaluations per layer. Mean distance to the target interval decreases, from $2.19\text{--}4.71 \times 10^{-4}$ to $0.285\text{--}0.902 \times 10^{-4}$. The implementation moves the requested margin, but finite-precision conversion and casting do not preserve expert membership in this run. These diagnostics concern hooked logits, not an

independent record of the kernel’s dispatched experts, and their denominator can include prefill, padding, and recomputation. The outcome cannot be attributed solely to a membership-preserving gap adjustment.

The separate greedy comparisons use 465 attempts on 465 problems. Qwen identity guiding gives -0.22 points $([-2.58, +2.15])$, and margin guiding gives $+0.86$ $([-1.29, +3.01])$. The 39-problem Minerva-only identity comparison changes from 24 to 25 correct answers, or $+2.56$ points $([-5.13, +12.82])$. All three intervals contain zero. These distinct decoding protocols and populations remain separate from the full stochastic comparisons.

5.4.8 Benchmark contributions and weighting sensitivity

Table 5.17 separates the benchmark contributions to the thirteen completed stochastic comparisons reported here. The primary pooled statistic weights attempts equally, so a problem with 32 attempts contributes 32 times as much as a problem with one. As a secondary sensitivity analysis, let n_q be the number of attempts for evaluation problem q and define

$$\hat{\Delta}_{\text{problem}} = \frac{1}{P} \sum_{q=1}^P \frac{1}{n_q} \sum_{i:q(i)=q} (Y_i^\pi - Y_i^0), \quad P = 465. \quad (5.4)$$

This estimates the change in mean per-problem correctness over the saved attempts. It is not pass@32 or a benchmark macro-average: a benchmark’s weight remains proportional to its number of evaluation problems. The attempt-weighted statistic remains the primary analysis; the secondary statistic tests how the interpretation depends on the evaluation unit.

Both statistics use the same 5,000 paired bootstrap samples of whole problems within each benchmark. Only the aggregation of a sampled problem’s outcomes changes. Recomputed attempt-level and benchmark intervals agree with the original saved comparisons. The complete benchmark counts, intervals, and problem-weighted results are recorded in `report/guiding_sensitivity.json` and the thesis source snapshot. All intervals remain exploratory and unadjusted for multiple comparisons.

The Qwen margin benchmark breakdown in Table 5.19 uses corrected judgments for both conditions. Its positive point estimate comes from AIME24

Table 5.17: Benchmark-level accuracy changes in percentage points for the thirteen reported stochastic interventions. Positive/negative denotes expert polarity; c denotes concurrency. Entries are point estimates, not significance decisions. Benchmark counts (problems/attempts) are shown in the header. Full benchmark intervals and correct counts are saved in `guiding_sensitivity.json`.

Condition	MATH-500 150/150	AIME24 9/288	AIME25 9/288	Olympiad 203/203	AMC23 12/384	Minerva 82/82
Qwen fixed positive +1	+0.00	-2.78	+0.00	-0.49	+0.52	+3.66
Qwen fixed positive +2	+0.00	-5.90	-1.74	-0.99	-3.91	+0.00
Qwen fixed positive -1	-1.33	+1.39	+2.08	+1.97	+0.00	+3.66
GPT-OSS fixed positive +1	+5.33	+1.39	-0.35	-2.46	-1.30	-1.22
GPT-OSS fixed positive +2	+2.00	-8.33	-7.64	-2.46	-0.26	+3.66
GPT-OSS fixed positive -1	+3.33	-2.08	-3.12	-2.96	+0.78	-1.22
GPT-OSS fixed negative +1	+4.67	-1.04	-1.39	+0.00	+1.04	+4.88
GPT-OSS paper positive +1	-47.33	-56.60	-76.39	-60.10	-68.75	-24.39
GPT-OSS paper positive -1	+2.67	-3.47	-5.21	-3.45	-0.78	+0.00
GPT-OSS paper negative +1	-47.33	-56.60	-74.31	-56.16	-63.80	-29.27
Qwen margin (c=16)	+0.67	+1.39	+1.39	-3.45	+0.26	-2.44
Qwen margin (c=25)	-0.67	+2.43	+2.43	+0.00	+0.00	+0.00
GPT-OSS margin (c=25)	+2.67	-1.04	-0.69	-3.45	-1.30	+1.22

Table 5.18: Sensitivity to the evaluation unit: accuracy changes and 95% intervals in percentage points. Attempt weighting is the primary statistic; equal-problem weighting first averages correctness over each problem’s attempts and then averages the 465 problems. Both use the same 5,000 paired, dataset-stratified problem resamples (seed 42). Equal problem weights are not equal benchmark weights. This is a secondary exploratory analysis without multiplicity adjustment.

Condition	Attempt change	95% CI	Problem change	95% CI
Qwen fixed positive +1	-0.29	[-1.15, +0.65]	+0.39	[-1.51, +2.31]
Qwen fixed positive +2	-2.80	[-5.16, -0.72]	-0.68	[-2.76, +1.33]
Qwen fixed positive -1	+1.08	[-0.14, +2.37]	+1.14	[-0.75, +2.94]
GPT-OSS fixed positive +1	+0.00	[-2.15, +2.01]	+0.42	[-2.37, +3.12]
GPT-OSS fixed positive +2	-3.30	[-6.16, -0.93]	-0.10	[-3.12, +3.08]
GPT-OSS fixed positive -1	-1.00	[-3.44, +1.22]	-0.51	[-3.41, +2.49]
GPT-OSS fixed negative +1	+0.57	[-1.58, +2.65]	+2.35	[-0.58, +5.44]
GPT-OSS paper positive +1	-61.65	[-69.32, -53.41]	-50.15	[-54.88, -45.45]
GPT-OSS paper positive -1	-2.22	[-4.16, -0.43]	-0.83	[-3.76, +2.23]
GPT-OSS paper negative +1	-59.57	[-67.24, -51.40]	-49.13	[-53.80, -44.56]
Qwen margin (c=16)	+0.07	[-0.86, +1.01]	-1.66	[-3.67, +0.45]
Qwen margin (c=25)	+0.93	[+0.00, +1.86]	-0.12	[-2.07, +1.80]
GPT-OSS margin (c=25)	-0.86	[-2.87, +1.15]	-0.50	[-3.49, +2.46]

and AIME25, offset by one loss on MATH-500. The primary interval includes zero.

Table 5.19: Benchmark contributions to Qwen margin guiding at concurrency 25. B/G denotes baseline/guided correct counts; net is guided minus baseline correct attempts. Changes and pointwise 95% intervals are percentage points, with 5,000 paired problem resamples (seed 42).

Benchmark	Problems	Attempts	Correct B/G	Net	Change	95% CI
MATH-500	150	150	148/147	-1	-0.67	[-2.00, +0.00]
AIME24	9	288	178/185	+7	+2.43	[+0.00, +5.56]
AIME25	9	288	242/249	+7	+2.43	[+1.04, +4.17]
Olympiad	203	203	171/171	+0	+0.00	[-3.45, +3.45]
AMC23	12	384	384/384	+0	+0.00	[+0.00, +0.00]
Minerva	82	82	50/50	+0	+0.00	[-6.10, +6.10]

Qwen margin guiding at concurrency 25 gains seven correct attempts on AIME24 and seven on AIME25, with zero net change on AMC23. MATH-500 loses one correct attempt; Minerva and OlympiadBench each have zero net change. The AIME improvements involve nine improved problems and one worsened problem. Equal-problem weighting gives -0.12 points with $[-2.07, +1.80]$, compared with the primary $+0.93$ and $[0.00, +1.86]$. Neither weighting establishes a beneficial direction.

For GPT-OSS fixed positive strength $+2$, the revised equal-problem estimate is -0.10 points $[-3.12, +3.08]$, while fixed negative-polarity promotion gives $+2.35$ $[-0.58, +5.44]$. The previous secondary positive intervals do not survive rescoring. For positive strength $+2$, MATH-500 and Minerva gain three correct attempts each, OlympiadBench loses five, and AIME24, AIME25, and AMC23 lose 24, 22, and one. Qwen fixed strength $+2$ gives -0.68 equal-problem points $[-2.76, +1.33]$, and GPT-OSS paper-style suppression gives -0.83 $[-3.76, +2.23]$; their below-zero primary intervals do not persist under this secondary weighting. GPT-OSS paper-style positive and negative promotion remain strongly negative: -50.15 $[-54.88, -45.45]$ and -49.13 $[-53.80, -44.56]$ points. No complete stochastic comparison has a positive

interval excluding zero under either weighting.

5.5 What the combined evidence establishes

The four stages do not support a single unqualified claim that reasoning can be improved by increasing the use of apparently successful experts. The judge supplies usable but imperfect labels on a development corpus; the probes reveal prospective information on held-out responses within their SAT corpus, beyond the simple position baseline; and the routing analysis identifies associations that depend on model, benchmark, scope, and aggregation. The intervention stage provides the strongest test of actionability. After consistent offline rescoring, no complete intervention establishes a beneficial accuracy direction using the reported 95% interval criterion. Qwen fixed positive strength +2 loses 2.80 attempt-weighted points ($[-5.16, -0.72]$); GPT-OSS fixed positive strength +2 loses 3.30 ($[-6.16, -0.93]$). GPT-OSS paper-style suppression loses 2.22 points ($[-4.16, -0.43]$), and positive/negative paper-style promotion loses 61.65 and 59.57 points with intervals well below zero. All stochastic margin intervals contain zero: Qwen concurrency 25 gives +0.93 points ($[0.00, +1.86]$), while GPT-OSS gives -0.86 ($[-2.87, +1.15]$). Equal-problem weighting removes the below-zero intervals for fixed strength +2 and paper-style suppression, but paper-style promotion remains strongly negative. The earlier positive OSS secondary intervals and greedy Qwen margin interval do not survive rescoring. These exploratory intervals remain conditional on saved attempts and frozen policies and are not adjusted for multiple comparisons.

This pattern separates the existence of measurable routing structure from the usefulness of a particular control rule. The evidence supports continued testing of carefully specified policies, with attention to numerical behaviour and evaluation contracts. It does not establish a transferable routing heuristic, nor does it rule out improvements from policies, selection procedures, and controls that were not evaluated.

Chapter 6

Conclusions

This thesis investigated whether reasoning annotations and internal routing measurements can support interventions in sparse mixture-of-experts models without changing model weights. The implemented pipeline connects automatic sentence annotation, prospective representation probing, observational routing analysis, and matched free-generation experiments. Its main empirical finding is a gap between observable structure and reliable control: reasoning-related information and routing–correctness associations are measurable, but promoting associated experts does not generally improve held-out mathematical accuracy under the tested policies.

6.1 Answers to the research questions

The first question concerned automatic episode annotation and the accessibility of upcoming episode information in prefix representations. The annotation experiments demonstrate that context-aware prompts can achieve useful agreement on the available labelled corpus and that prompt selection benefits from explicit class-level safeguards. The final alignment retained its seed when an optimised candidate damaged Plan recall, even though its internal optimisation reward increased. The NVFP4 deployment check further showed that unchanged aggregate accuracy can conceal changes in individual labels. Automatic annotations are therefore informative measurements with uncer-

tainty, not independently established ground truth for the generated benchmark solutions.

All three frozen encoders contain linearly accessible information about upcoming episode labels on held-out responses from the SAT corpus. With response-disjoint outer folds and inner-validation layer selection, macro-AUROC is 0.852 for Qwen, 0.780 for GPT-OSS, and 0.788 for Gemma; all exceed the simple prefix-position baseline. This strengthens the answer to the decodability question beyond the original sentence split. It does not establish transfer to other domains or to the models’ own generations, and thresholded classification remains uneven across labels. The external Qwen inspection reinforces this distinction: Implement transfers more coherently than several other classes, while the provisional review does not support a robust seven-class online detector.

The second question concerned relationships between routing, reasoning activity, and correctness. The three-model corpus shows substantial but heterogeneous structure. Confidence is positively associated with correctness across Qwen benchmarks, but its sign varies in Gemma and GPT-OSS. Class conditioning does not remove that variation, and problem-level aggregation can yield a different association from pooling attempts. The analysis therefore identifies hypotheses for model-specific interventions rather than a common scalar measure of successful reasoning. Length, termination, problem difficulty, and annotation coverage remain plausible contributors to the observed patterns.

Directed class-transition matrices additionally separate correct and incorrect attempts for all three models while retaining original adjacency and sampling gaps. The expert-selection audit shows that activation-based lift and risk difference need not choose the same targets, even with identical calibration weights and budgets: overlap is 14/48 layer–expert identities for Qwen and 6/16 for GPT-OSS positive and 7/16 for GPT-OSS negative policies.

The third question asked whether calibrated policies improve held-out accuracy or change token use. After consistent offline rescoring, no complete intervention establishes a beneficial accuracy direction using the reported 95% interval criterion. Qwen fixed positive strength +2 loses 2.80 attempt-weighted points ($[-5.16, -0.72]$); GPT-OSS fixed positive strength +2 loses

3.30 $([-6.16, -0.93])$. GPT-OSS paper-style suppression loses 2.22 points $([-4.16, -0.43])$, and positive/negative paper-style promotion loses 61.65 and 59.57 points with intervals well below zero. All stochastic margin intervals contain zero: Qwen concurrency 25 gives +0.93 points $([0.00, +1.86])$, while GPT-OSS gives -0.86 $([-2.87, +1.15])$. Equal-problem weighting removes the below-zero intervals for fixed strength +2 and paper-style suppression, but paper-style promotion remains strongly negative. The earlier positive OSS secondary intervals and greedy Qwen margin interval do not survive rescoring. These exploratory intervals remain conditional on saved attempts and frozen policies and are not adjusted for multiple comparisons.

The interrupted Qwen paper-style run shows severe degradation on its completed subset: 4/494 correct answers and 494/494 token-limit hits. Its coverage is restricted to AIME24 and part of AIME25, so it remains separate from the complete-run evidence. Promoting a full top- k target set fixes the selected identities and equalises their hook scores; this is a possible mechanism, not a verified causal explanation of the observed termination behaviour.

The qualitative review supplies direct examples of repetition in both capped and normally ended outputs, while also identifying two formatting disagreements with the saved scorer. It does not estimate failure-mode prevalence or establish the routing mechanism.

The revised evaluations provide evidence of harm for specific interventions and no positive accuracy interval excluding zero. A change of evaluation unit cannot replace an unfavourable primary result; both scoring rules and objectives should be fixed before independent replication. Token counts also change, but lower token use without established preservation of accuracy is not a demonstrated efficiency improvement.

6.2 Contributions and interpretation

The methodological contribution is an integrated and auditable experimental path from functional descriptions of reasoning to routing perturbations. The completed control experiments test the actionability of calibrated routing associations. They do not evaluate a controller driven by online episode predic-

tions: annotation and probing support measurement and target exploration, and their usefulness as control inputs remains to be tested. The workflow preserves problem identities, generation settings, sentence spans, replay provenance, and frozen calibration policies. It separates sentence-level annotation quality from prefix-state decodability, and both from the effect of intervening during generation. This makes it possible to identify where a promising signal fails to support the next inference.

The observational contribution is a comparison of routing and representation measurements across three sparse models, six benchmarks, multiple reasoning classes, and response positions. Its most consequential lesson is heterogeneity: an association that appears strong in one population may be weak, undefined, or oppositely signed in another. The distinction between attempt-level, problem-level, and within-problem evidence is necessary for interpreting repeated mathematical sampling.

The intervention contribution is a set of paired held-out evaluations that expose both weak effects and substantial degradation. The large losses under GPT-OSS paper-style promotion do not support a monotonic reading of the calibration association under that intervention rule. They do not show that the selected experts are intrinsically harmful: forcing a fixed target set can disrupt coordinated, input-dependent computation. The changes after rescoring also show why conclusions require validated answer equivalence as well as an explicit evaluation objective. Acting on a routing statistic changes an entire generation trajectory, and the measured effect belongs to that complete policy.

6.3 Limitations

The annotation reference contains few responses and ambiguous functional boundaries. Repeated prompt development on its validation responses limits generalisation claims, and the generated benchmark labels lack an independent human assessment. Class-conditioned results inherit these errors and the sampling scheme. Although Gemma and GPT-OSS share a stratified procedure, their labelled problem cohorts differ; Qwen uses a different deployment and sampling protocol.

The original probe scores and routing-layer choices remain exploratory because responses cross the sentence split and layer selection uses test outcomes. The separate grouped study removes those two evaluation overlaps, but has only 38 responses from one corpus and one split seed. Its bootstrap conditions on the fitted classifiers and selected layers; neither external transfer nor online detection is established by that result. Exceeding a two-feature position baseline also does not eliminate richer shortcuts. The correlation study retains all attempts and respects problem clusters in the main bootstrap intervals, but clustering does not remove confounding. Late-position views condition on continued generation, class views condition on available labels and class tokens, and missing-outcome exclusions in policy calibration differ from the observational scoring denominator. Generation-to-replay transfer across quantisations adds uncertainty about the measured router state.

The failure inspection identified answer-formatting sensitivity, addressed by consistent offline rescoring of the saved intervention evaluations. The revised scorer still has a defined extraction hierarchy, numerical tolerance, unit conventions, and unsupported formats; it is not independent human adjudication. The observational analysis and frozen policy calibration retain their original correctness labels. The current results therefore evaluate those existing policies under a revised outcome rule; they do not estimate what a policy refitted using revised calibration labels would do.

Intervention uncertainty is conditional on frozen policies and saved attempts. The confidence intervals do not include variation from recalibration, new decoding seeds, or configuration search, and no multiple-comparison adjustment is applied. Nine held-out problems in each AIME benchmark limit benchmark-specific precision. The Qwen comparisons with different baseline contracts and concurrency settings do not isolate one experimental factor across rows. The seven OSS identity conditions share those settings and retain the same targets within polarity, allowing a more direct comparison of intervention rules. Finally, margin diagnostics reveal membership changes in finite precision, while identity interventions change routing weights as well as selected sets. These experiments do not isolate all of the mechanisms through which the policies affect generation.

6.4 Directions for further work

The next measurement step is an independent, response-grouped annotation assessment on generated mathematical solutions. It should include examples from every model and benchmark, explicit treatment of ambiguous units, and adjudication of a representative sample. The judge and its decoding settings should be frozen before that assessment. The completed response-grouped probe study should be extended to new domains and the encoders' own generations, with repeated response splits and thresholds and abstention rules selected on development data. More flexible position controls would further test shortcut explanations. These extensions would assess whether the available episode information supports a reliable online measurement.

For observational modelling, matched problem cohorts and common annotation protocols would make cross-model comparisons more interpretable. Analyses should separate within-problem variation from between-problem difficulty and account for length and termination without simply discarding failed completions. Candidate features and hypotheses should be chosen before a fresh evaluation, reducing the temptation to treat the strongest result in a broad screen as a confirmed mechanism.

The offline answer-equivalence audit is complete for the saved intervention evaluations. A further validation study should adjudicate residual parser failures and ambiguous or structured answers, assess robustness to tolerance and extraction choices, and examine calibration-label sensitivity before refitting policies on development data.

The next generation study should replicate the Qwen margin configuration and test GPT-OSS identity guiding with independent decoding seeds and an untouched evaluation set. The attempt-weighted or problem-weighted objective must be specified in advance, with benchmark-level reporting retained. Strength, target-selection rules, and other settings should be selected on a separate validation partition. The OSS negative-lift controls and target budgets matching native dispatch are already complete. Matched random-expert controls and smaller budgets of one or two targets per layer remain useful for distinguishing selector information from generic perturbation and from forcing a full top- k

target set. Applying hooks separately during prefill and decoding would localise when their effects arise. Recording actual dispatched identities and mixture weights would also clarify the numerical behaviour of the margin operation.

A later extension could condition guidance on a validated prediction of the current reasoning activity. The completed interventions do not yet test such a controller: they use fixed identities or current router margins, without an online episode prediction. Any class-conditioned policy would need to account for detector errors and forbid access to the future continuation used to define its reference labels. Its evaluation should jointly report accuracy, token use, and failure modes under a matched budget.

The evidence thus supports routing as a measurable and consequential part of mathematical generation, while leaving reliable guidance an open empirical problem. Progress depends on validating the measurements, specifying what an intervention changes, and testing its benefit on data that did not determine the policy.

Appendix A

Additional descriptive routing plots

This appendix supplements the main results with benchmark-specific outcome curves and expert screens from the same saved populations. These are exploratory descriptions; they do not supply independent replications of the aggregate analyses.

A.1 Class shares and observed outcomes

Figures A.1 and A.2 show observed accuracy and generation cost across sampled class-share bins. The attempt counts make sparse bins visible. Lines connect occupied bins and do not fit an optimal class composition or establish a causal response.

A.2 Benchmark-specific directed transitions

Figure A.3 complements the correctness-stratified mean-count matrices in the main chapter. It pools outcomes within each benchmark and normalises each source-class row to percentages, describing the observed destination distribution conditional on an adjacent labelled pair. These percentages have a different denominator from mean counts per attempt; a missing row is undefined, not a

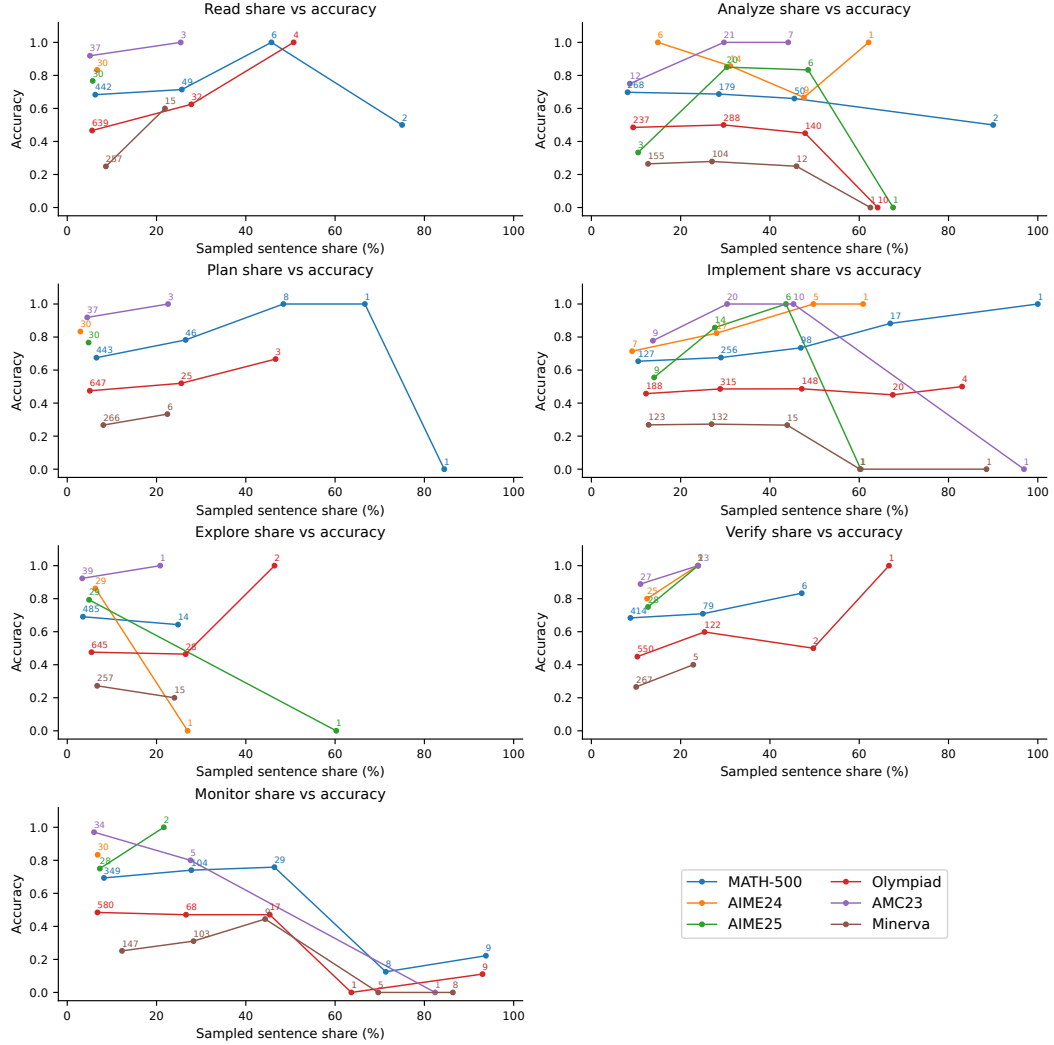


Figure A.1: Actual outcome levels versus sampled class proportions. Fixed share bins are $[0, 20)$, $[20, 40)$, $[40, 60)$, $[60, 80)$, and $[80, 100]$ percent. Each point is at the mean observed share and shows accuracy; small labels give attempt counts. Empty bins are omitted. Colours distinguish benchmarks; lines only connect occupied bins. Sparse bins, sentence sampling, length, and difficulty can explain apparent trends; these descriptive plots do not identify optimal class proportions or a validated accuracy–length Pareto frontier.

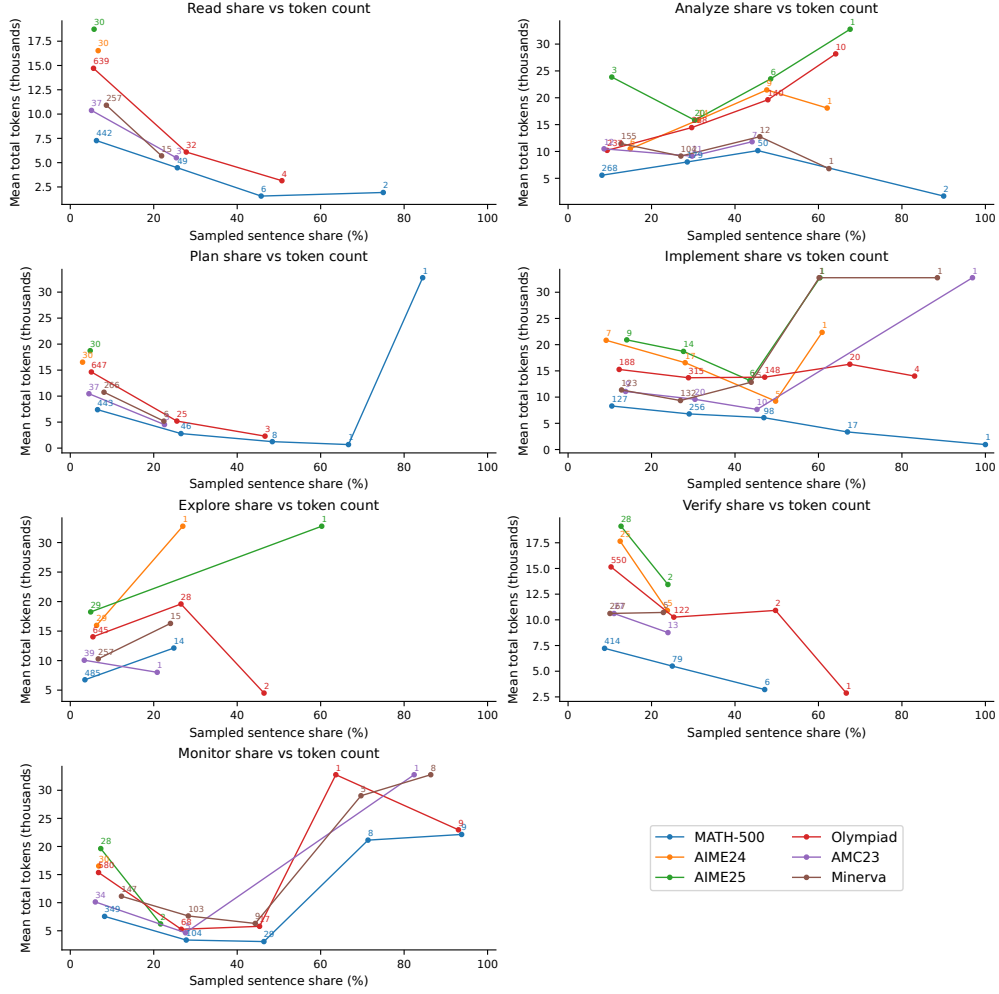


Figure A.2: Mean total continuation tokens versus sampled class share. Bin boundaries, colours, sample-count labels, and sampling limitations are as in Figure A.1. Token count includes the final answer and token-limit completions; lines are descriptive, not intervention effects.

zero transition probability.

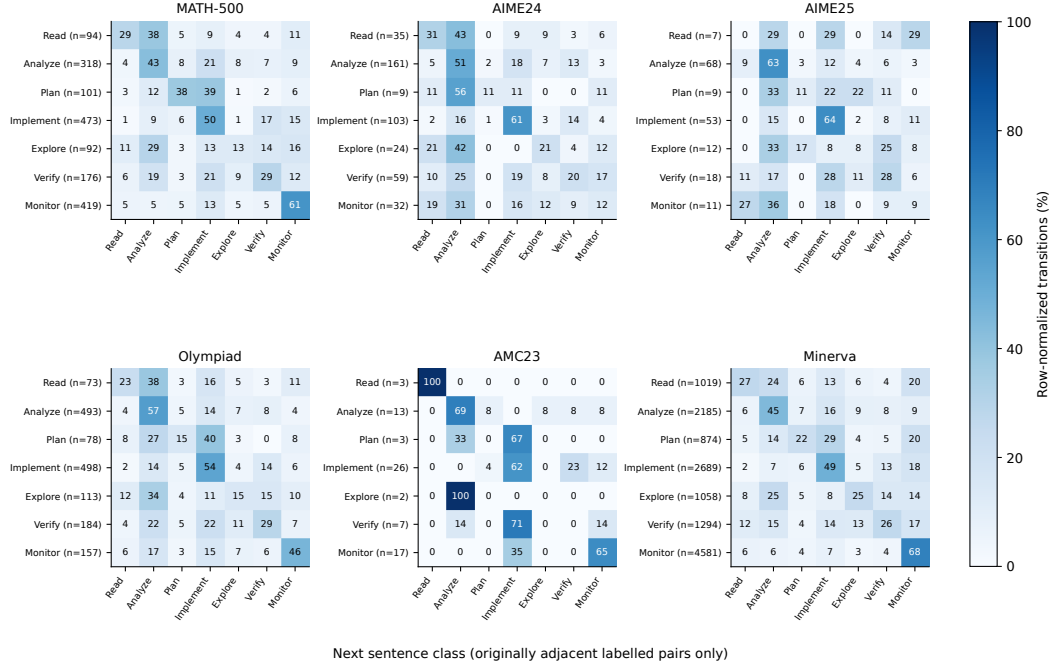


Figure A.3: Observed class-state transitions by benchmark. Rows are the current class, columns the next class; cells are row percentages and row labels give pair counts. Only originally consecutive sentence indices with both labels are counted. Gaps in sampling are never bridged. Self-transitions are retained; empty rows are undefined. This describes the observed adjacent-pair sample, not the complete transition process or an effect of changing state.

A.3 Expert-frequency screens across models

Figures A.4, A.5, and A.6 show every model-local expert ID in its original order. Pooling a numeric ID across measured layers does not turn it into a single shared module. The steering policies instead target layer-qualified identities. These screens show association with correctness and token count; they neither rank causal usefulness nor identify matching experts across architectures.

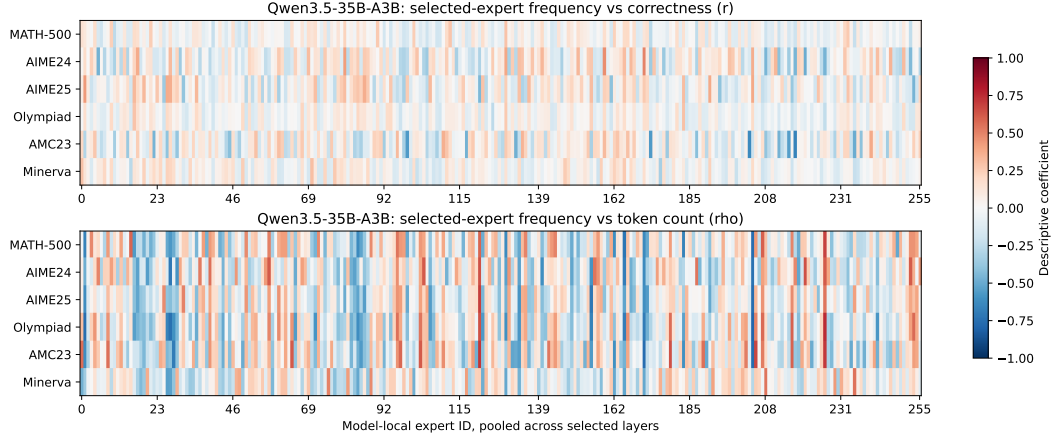


Figure A.4: Qwen3.5-35B-A3B: selected-pool expert frequency correlations with correctness and token count. IDs are model-local and pooled across selected layers; they do not identify matching experts across models. Missing coefficients are blank.

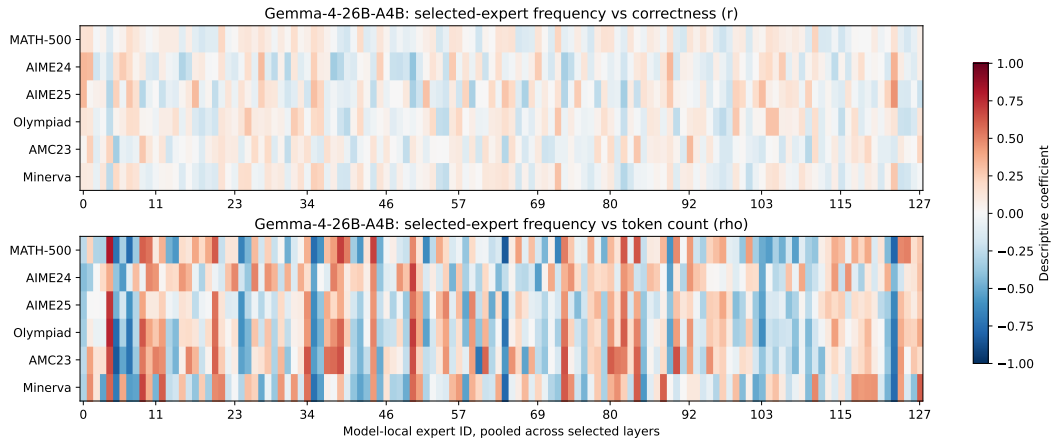


Figure A.5: Gemma-4-26B-A4B: selected-pool expert frequency correlations with correctness and token count. IDs are model-local and pooled across selected layers; they do not identify matching experts across models. Missing coefficients are blank.

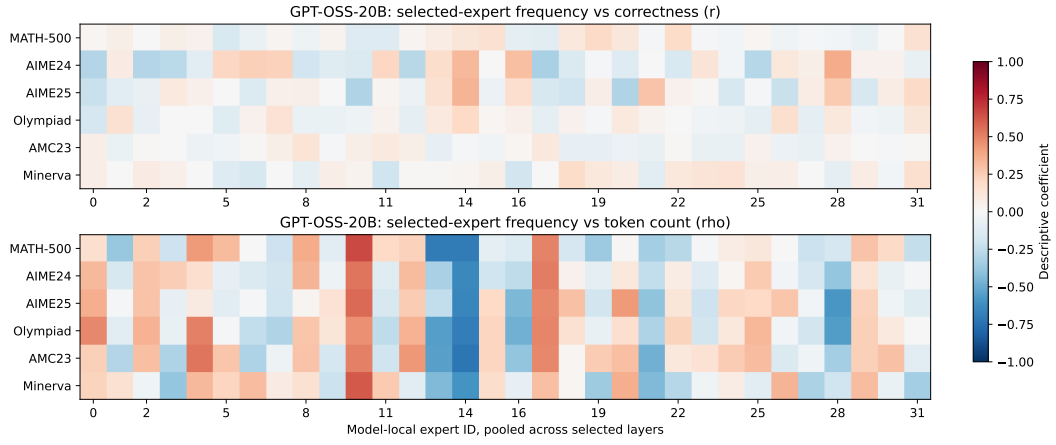


Figure A.6: GPT-OSS-20B: selected-pool expert frequency correlations with correctness and token count. IDs are model-local and pooled across selected layers; they do not identify matching experts across models. Missing coefficients are blank.

Bibliography

- [1] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 24824–24837. DOI: 10.52202/068431–1800. URL: <https://proceedings.neurips.cc/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html>.
- [2] Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. “Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023, pp. 74952–74965. DOI: 10.52202/075280–3275. URL: https://proceedings.neurips.cc/paper_files/paper/2023/hash/ed3fea9033a80fea1376299fa7863f4a-Abstract-Conference.html.
- [3] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer”. In: *International Conference on Learning Representations*. 2017. URL: <https://arxiv.org/abs/1701.06538>.
- [4] William Fedus, Barret Zoph, and Noam Shazeer. “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity”. In: *Journal of Machine Learning Research* 23.120 (2022), pp. 1–39. URL: <https://jmlr.org/papers/v23/21-0998.html>.

- [5] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, L  lio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Th  ophile Gervet, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. *Mixtral of Experts*. Preprint. 2024. DOI: 10.48550/arXiv.2401.04088. arXiv: 2401.04088 [cs.LG].
- [6] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. “DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024, pp. 1280–1297. DOI: 10.18653/v1/2024.acl-long.70. URL: <https://aclanthology.org/2024.acl-long.70/>.
- [7] Qwen Team. *Qwen3.5-35B-A3B Model Card*. 2026. URL: <https://huggingface.co/Qwen/Qwen3.5-35B-A3B> (visited on 09/23/2026).
- [8] Alan H. Schoenfeld. *Mathematical Problem Solving*. Academic Press, 1985. ISBN: 978-0-12-628870-4. DOI: 10.1016/C2013-0-05012-8. URL: <https://www.sciencedirect.com/book/monograph/9780126288704/mathematical-problem-solving>.
- [9] Ming Li, Nan Zhang, Chenrui Fan, Hong Jiao, Yanbin Fu, Sydney Peters, Qingshu Xu, Robert Lissitz, and Tianyi Zhou. “Understanding the Thinking Process of Reasoning Models: A Perspective from Schoenfeld’s Episode Theory”. In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2025, pp. 18267–18288. DOI: 10.18653/v1/2025.emnlp-main.922. URL: <https://aclanthology.org/2025.emnlp-main.922/>.

- [10] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. “GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning”. In: *International Conference on Learning Representations*. 2026. URL: https://proceedings.iclr.cc/paper_files/paper/2026/hash/0e9e708b6f48e14fd0ac29e167413f76-Abstract-Conference.html.
- [11] John Hewitt and Percy Liang. “Designing and Interpreting Probes with Control Tasks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, 2019, pp. 2733–2743. DOI: 10.18653/v1/D19-1275. URL: <https://aclanthology.org/D19-1275/>.
- [12] Emmy Liu, Chenxuan Cui, Kenneth Zheng, and Graham Neubig. “Testing the Ability of Language Models to Interpret Figurative Language”. In: *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2022, pp. 4437–4452. DOI: 10.18653/v1/2022.naacl-main.330. URL: <https://aclanthology.org/2022.naacl-main.330/>.
- [13] Yoav Benjamini and Yosef Hochberg. “Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing”. In: *Journal of the Royal Statistical Society: Series B (Methodological)* 57.1 (1995), pp. 289–300. DOI: 10.1111/j.2517-6161.1995.tb02031.x.
- [14] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. “Let’s Verify Step by Step”. In: *International Conference on Learning Representations*. 2024. URL: https://proceedings.iclr.cc/paper_files/paper/2024/hash/aca97732e30bcf1303bc22ac3924fd16-Abstract-Conference.html.

- [15] Bo Liu, Simon Yu, Zichen Liu, Leon Guertler, Penghui Qi, Daniel Balcells, Mickel Liu, Cheston Tan, Weiyan Shi, Min Lin, Wee Sun Lee, and Natasha Jaques. “SPIRAL: Self-Play on Zero-Sum Games Incentivizes Reasoning via Multi-Agent Multi-Turn Reinforcement Learning”. In: *International Conference on Learning Representations*. 2026. URL: https://proceedings.iclr.cc/paper_files/paper/2026/hash/3ba7c8c36b71b2fc31fae3361aefefbf-Abstract-Conference.html.
- [16] John Hewitt and Christopher D. Manning. “A Structural Probe for Finding Syntax in Word Representations”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, 2019, pp. 4129–4138. DOI: 10.18653/v1/N19-1419. URL: <https://aclanthology.org/N19-1419/>.
- [17] Lihao Sun, Hang Dong, Bo Qiao, Qingwei Lin, Dongmei Zhang, and Saravan Rajmohan. “LLM Reasoning as Trajectories: Step-Specific Representation Geometry and Correctness Signals”. In: *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2026, pp. 26872–26887. DOI: 10.18653/v1/2026.acl-long.1237. URL: <https://aclanthology.org/2026.acl-long.1237/>.
- [18] Kang Chen, Sihan Zhao, Yixin Cao, and Yu-Gang Jiang. *Beyond the Trace: Coupling an Interpretable Reasoning-State Readout to Native MoE Routing*. Preprint, version 2. 2026. DOI: 10.48550/arXiv.2608.17638. arXiv: 2608.17638. URL: <https://arxiv.org/abs/2608.17638v2>.
- [19] Matthew Lyle Olson, Neale Ratzlaff, Musashi Hinck, Man Luo, Sungduk Yu, Chendi Xue, and Vasudev Lal. “Probing Semantic Routing in Large Mixture-of-Expert Models”. In: *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, 2025, pp. 18263–18278. DOI: 10.18653/v1/2025.findings-emnlp.991. URL: <https://aclanthology.org/2025.findings-emnlp.991/>.

- [20] Charles Ye, Bo Yuan, and Lee Sharkey. *Polysemantic Experts, Monosemantic Paths: Routing as Control in MoEs*. Preprint. 2026. DOI: 10.48550/arXiv.2604.17837. arXiv: 2604.17837.
- [21] Mengru Wang, Xingyu Chen, Yue Wang, Zhiwei He, Jiahao Xu, Tian Liang, Qiuzhi Liu, Yunzhi Yao, Wenxuan Wang, Ruotian Ma, Haitao Mi, Ningyu Zhang, Zhaopeng Tu, Xiaolong Li, and Dong Yu. “Two Experts Are All You Need for Steering Thinking: Reinforcing Cognitive Effort in MoE Reasoning Models Without Additional Training”. In: *Advances in Neural Information Processing Systems*. Vol. 38. 2025, pp. 140699–140724. DOI: 10.52202/085713-4701. URL: https://proceedings.neurips.cc/paper_files/paper/2025/hash/cdcfd67988fea858bb22317497105053-Abstract-Conference.html.
- [22] Mohsen Fayyaz, Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Ryan Rossi, Trung Bui, Hinrich Schütze, and Nanyun Peng. “Steering MoE LLMs via Expert (De)Activation”. In: *International Conference on Learning Representations*. 2026. URL: https://proceedings.iclr.cc/paper_files/paper/2026/hash/0d61c5f5ef91e7e8a091b7b8f72b853c-Abstract-Conference.html.
- [23] Youngsik Yoon, Siwei Wang, Wei Chen, and Jungseul Ok. *When Are Experts Misrouted? Counterfactual Routing Analysis in Mixture-of-Experts Language Models*. Preprint. 2026. DOI: 10.48550/arXiv.2605.07260. arXiv: 2605.07260.
- [24] Guinan Su, Yanwu Yang, Li Shen, Lu Yin, Shiwei Liu, and Jonas Geiping. *Rewiring Experts on the Fly: Continuous Rerouting for Better Online Adaptation in Mixture-of-Expert Models*. Preprint. 2025. DOI: 10.48550/arXiv.2510.14853. arXiv: 2510.14853.
- [25] Nina Rimskey, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Turner. “Steering Llama 2 via Contrastive Activation Addition”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024, pp. 15504–15522. DOI: 10.18653/v1/

- 2024.acl-long.828. URL: <https://aclanthology.org/2024.acl-long.828/>.
- [26] Yihuai Hong, Meng Cao, Dian Zhou, Lei Yu, and Zhijing Jin. “The Reasoning-Memorization Interplay in Language Models Is Mediated by a Single Direction”. In: *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, 2025, pp. 21565–21585. DOI: 10.18653/v1/2025.findings-acl.1111. URL: <https://aclanthology.org/2025.findings-acl.1111/>.
- [27] Ningyu Xu, Qi Zhang, Chao Du, Qiang Luo, Xipeng Qiu, Xuanjing Huang, and Menghan Zhang. “Revealing Emergent Human-Like Conceptual Representations from Language Prediction”. In: *Proceedings of the National Academy of Sciences* 122.44 (2025), e2512514122. DOI: 10.1073/pnas.2512514122. URL: <https://doi.org/10.1073/pnas.2512514122>.
- [28] Tianhang Zhang, Lin Qiu, Qipeng Guo, Cheng Deng, Yue Zhang, Zheng Zhang, Chenghu Zhou, Xinbing Wang, and Luoyi Fu. “Enhancing Uncertainty-Based Hallucination Detection with Stronger Focus”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 915–932. DOI: 10.18653/v1/2023.emnlp-main.58. URL: <https://aclanthology.org/2023.emnlp-main.58/>.
- [29] University of Bologna NLP Group. *UniBoNLP GPU Cluster: Guide*. URL: <http://137.204.107.40:37339/slurm/guide> (visited on 09/23/2026).
- [30] University of Bologna NLP Group. *UniBoNLP Cluster Resource Allocation Dashboard*. URL: <http://137.204.107.40:37339/slurm/> (visited on 09/13/2026).
- [31] Hugging Face H4. *MATH-500: Dataset Release*. URL: <https://huggingface.co/datasets/HuggingFaceH4/MATH-500> (visited on 09/23/2026).
- [32] math-ai. *AIME 2024: Dataset Release*. URL: <https://huggingface.co/datasets/math-ai/aime24> (visited on 09/23/2026).

-
- [33] math-ai. *AIME 2025: Dataset Release*. URL: <https://huggingface.co/datasets/math-ai/aime25> (visited on 09/23/2026).
 - [34] Hothan. *OlympiadBench: Dataset Release*. URL: <https://huggingface.co/datasets/Hothan/OlympiadBench> (visited on 09/23/2026).
 - [35] math-ai. *AMC 2023: Dataset Release*. URL: <https://huggingface.co/datasets/math-ai/amc23> (visited on 09/23/2026).
 - [36] math-ai. *Minerva Math: Dataset Release*. URL: <https://huggingface.co/datasets/math-ai/minervamath> (visited on 09/23/2026).